Differential fuzzing of the ZML API

zmlsmith findings report

Bugs, crashes and risks found in ZML and upstream XLA, ranked from most to least critical; every bug has a standalone reproducer verified on the commit below.

Code under test	ZML at commit `74c590b0` (`master`)
Platform	CPU PJRT plugin: one device, a 4-device mesh, and a 2-level (bus of rings) mesh
Tool	`tools/zmlsmith` (random programs, reference interpreter, automatic reduction, parser fuzzing)
Findings	85 entries covering 117 reproducer files
Date	2026-10-25, verified 2026-10-26

2	32	40	7	4
Critical	High	Medium	Low	Design

Contents

1 Executive summary ... 4
1.1 Recommended order of work ... 4
2 Method, rubric and reproduction ... 5
2.1 How the bugs were found ... 5
2.2 Severity rubric ... 5
2.3 How each finding was verified ... 5
2.4 Reproducing a finding ... 6
3 Findings index ... 7
4 Critical findings ... 10
C01 Loader.loadBuffer hands a dangling stack pointer to an async task ... 10
C02 DynamicBufferPool hands the same block to two owners under contention ... 11
5 High findings ... 13
H01 Any malformed tokenizer.json smashes the stack in default (non-NDEBUG) builds (IREE backend) ... 13
H02 SentencePiece streaming decode drops text after repeated tokens ... 14
H03 Long __metadata__ keys crash the safetensors parser (1024-byte stack buffer, unchecked append) ... 15
H04 Any HTTP redirect with a relative Location panics: the request URL it resolves against is overwritten ... 16
H05 SentencePiece Decoder keeps the ids of the previous call ... 17
H06 Tensor data offsets are not validated ... 18
H07 scatterSlices ignores the order of update axes ... 19
H08 Chained padded reduce_window gives wrong values ... 20
H09 Nucleus sampling drops the token that crosses top_p ... 21
H10 SentencePiece encode of an empty text panics ... 23
H11 Paged attention doesn't compile with fewer queries than page_size ... 23
H12 Malformed safetensors headers panic instead of failing (length, schema, dims) ... 25
H13 HTTP redirects are followed with no limit ... 26
H14 Answers without Content-Length panic (HTTP, S3, GCS and HF backends) ... 27
H15 HF repository listings ignore pagination ... 30
H16 Malformed Content-Range panics (HTTP and parallel_read) ... 31
H17 3xx without a valid Location panics ... 33
H18 resizeBilinear / Bicubic drop ResizeOpts.precision ... 35
H19 bf16 Rng.uniform falls outside [min, max) ... 36
H20 linspace doesn't end at end ... 37
H21 Slice .len is ignored ... 38
H22 Paged attention: wrong outputs for batches with an empty sequence or a partial prefill chunk ... 39
H23 Homemade tokenizer encode returns a slice that can't be freed ... 40
H24 FNUZ formats use the IEEE exponent bias ... 41
H25 scatterSlices with rank-0 indices ignores update_fn ... 42
H26 ops.while with the same tensor in two state slots ... 43
H27 broadcast ignores axes when dims already match ... 44
H28 Convolution with negative padding and permuted layouts gives wrong values ... 45
H29 SPMD reduce_window with base dilation and padding returns shifted values ... 46
H30 resizeBicubic permutes axes for axis >= 2 ... 47
H31 maximum(clamp(lo, x, hi), lo) is simplified to the clamp ... 49
H32 f4e2m1 to integer saturates +-6.0 ... 50
6 Medium findings ... 52
M01 TensorStore.View.hasKey matches by prefix ... 52
M02 maximum / minimum and max / min reductions don't propagate NaN on CPU ... 53
M03 Resize sample coordinates are computed in f16 / bf16 ... 54
M04 zml.floats conversions don't round to nearest even and mishandle NaN ... 55
M05 Integer Rng.uniform is biased ... 56
M06 Quantized Linear / scaledDot don't compile on CPU ... 57
M07 pjrt struct helpers exceed the eval branch quota on long root module names ... 58
M08 Partitioning metadata survives ops that change the partitioned dim ... 59
M09 Exe.call passes undefined pointers when arguments are missing ... 60
M10 top_p = 1 still filters: the token whose f32 cumulative sum rounds above 1 is never sampled ... 61

M11 Homemade tokenizer: crashes on empty input, unknown UTF-8 and id == len ... 62
M12 Homemade streaming decode drops text ... 64
M13 Tokenizer.fromBytes indexes an empty file and misroutes JSON with leading whitespace ... 65
M14 Homemade Normalizer.fromHfJson overflows its 8-byte whitespace buffer ... 65
M15 RoPE yarn config without factor panics ... 66
M16 Tensor.print aborts for sub-byte, e8m0, e3m4, e4m3 and f4 dtypes ... 67
M17 Sub-byte Shape byte strides count whole bytes per element ... 68
M18 slices() with mixed static and dynamic slices produces invalid MLIR ... 69
M19 Sub-byte tensors: empty, odd last dim, rank 0 and rank 8 panic ... 70
M20 rope assumes .hd is the last axis ... 72
M21 XLA compiler aborts and verifier errors ... 73
M22 XLA CPU runtime failures: conv with reversal + stride + padding, FFT layout ... 77
M23 HF revisions containing a slash are mis-parsed ... 78
M24 RoPE scaling configs using `type` are rejected ... 79
M25 Crashes on valid inputs in small Tensor helpers ... 80
M26 Tensor.any, Tensor.all and pad with a slice don't compile ... 84
M27 broad / broadcastLeft / Right and nearest / upsample lose tags ... 85
M28 Homemade tokenizer loaders don't compile on Zig 0.16 ... 86
M29 HF tree sizes overflow on large listings ... 87
M30 Tensor.print of an empty tensor doesn't compile ... 88
M31 GatedDeltaNet.forward only accepts one axis order ... 89
M32 Integer mean over a long axis panics while tracing ... 90
M33 Reading back an empty sharded buffer panics ... 90
M34 gather / scatter reject valid batching layouts ... 91
M35 Bicubic resize of axes shorter than 4 asserts ... 93
M36 GSPMD: partitionId inside manualComputation doesn't compile ... 94
M37 maxPool indices are wrong when the max is the dtype minimum ... 95
M38 Rank-0 Slice views print the first element of the buffer ... 96
M39 leakyReLU is wrong for negative slopes ... 97
M40 Simplifier rewrites ignore IEEE zeros: -0 + 0, and convert(bool) * x with x = -3, inf or NaN ... 97
7 Low findings ... 100
L01 Constant additions are reassociated ... 100
L02 reuseBuffer doesn't check that input and output share a sharding ... 101
L03 Shape helpers: reshape(-1) of an empty shape, and `._N` past the rank ... 102
L04 DataType.Value.as and mantissaSize miss f8 / f4 / 2-bit dtypes ... 102
L05 zml.testing.expectClose panics on bool slices and on identical f4e2m1, f8e3m4, f8e4m3, f8e8m0 slices ... 104
L06 DataType.resolvePeerType is not commutative and can lose values ... 105
L07 customCall array outputs don't compile ... 105
8 Design notes ... 107
D01 .coord is a reserved tag in gather ... 107
D02 Quantized scales are tagged differently ... 108
D03 maxPool2d indices hold only the column of the max ... 108
D04 Buffer.toSlice fails on buffers sharded on any axis but the first ... 108
9 Appendix A: robustness audit ... 110
10 Appendix B: withdrawn after verification ... 111
11 Appendix C: coverage and limits ... 112

1 Executive summary

zmlsmith generated random programs over the ZML API, checked every op against an independent reference interpreter, and reduced every failure automatically to a small standalone Zig test. Parsers and network clients were fuzzed separately in isolated processes. After the campaign, every finding was verified again: its reproducer was rerun on a pristine checkout of `74c590b0` and had to fail with the specific error the finding describes, every cited code line was checked against the commit, and each claim was reviewed against the code and the relevant spec. 8 entries of the first draft did not survive and are listed in Appendix B.

The report keeps 85 findings: 72 bugs in ZML, 9 in upstream XLA (reachable through ZML), and 4 design notes. 2 are critical and 32 high.

- **Critical.** C01: `Loader.loadBuffer` passes a pointer to its own stack frame to an async task; with the documented `--config=release` the load completes and silently fills the buffer with wrong bytes. C02: `DynamicBufferPool` hands one block to two threads under contention (a lock-free stack with an ABA race), which corrupts the Loader's staging buffers.
- **Crashes on untrusted input.** 10 high findings are reached from files or network answers that ZML does not control: checkpoints, tokenizer files and HTTP servers. Most turn a bad or merely unusual input into a panic or a hang instead of an error; the safetensors offsets finding can also read the wrong bytes silently. H01 is stack corruption, but only in Bazel's default (non-NDEBUG) build mode; release builds were checked and return an error. H04 panics on every redirect with a relative Location.
- **Silent wrong values.** 19 high findings return wrong values with no error, among them streaming detokenization (H02, H05), nucleus sampling at the usual `top_p` values (H09), paged attention on CPU (H22), several Tensor ops, and five XLA miscompilations on valid StableHLO.

Critical findings

`C01` Loader.loadBuffer hands a dangling stack pointer to an async task ZML

`C02` DynamicBufferPool hands the same block to two owners under contention ZML

1.1 Recommended order of work

1. Fix the two critical bugs (C01, C02): both are a few lines (keep the tensor alive until the task ran; guard the pool's pop with its existing mutex). The patch check in each finding shows the reproducer passing with that change.
2. Make every parser and network client return errors instead of panicking on untrusted input: the safetensors, tokenizer and VFS entries marked *untrusted input*. Most fixes replace a `.?`, `catch unreachable` or unchecked append.
3. Fix the silent wrong values in the inference path: streaming detokenizers, nucleus sampling, paged attention.
4. Add the reproducers of fixed bugs to CI: move each file from `regressions/found/` to `regressions/`, and delete its `gen.KnownBug` entry so zmlsmith explores that input space again.
5. File the upstream XLA issues with their reproducers, the wrong-value ones first.

2 Method, rubric and reproduction

2.1 How the bugs were found

- **Graph API.** The generator builds random programs over 160 Tensor functions, 27 nn functions, the `zml.ops` control flow and the quantization helpers. It evaluates an independent reference interpreter while building, with bounds per element: correctly rounded ops on exact inputs must be bit exact, sign of zero included. The tool checks shape and tags of each op, runs each program twice for determinism, and on a failure shrinks the program (node removal, bypassing nodes inside loop and branch bodies, input simplification, dimension bisection) until the signature changes.
- **Parsers and IO.** `zmlsmith parse-fuzz` mutates valid safetensors files, `tokenizer.json` documents, sentencepiece models and rope configs. It also drives the HTTP, S3 and GCS backends against a local socket server that sends mutated answers, and the HF backend, whose URLs are hardcoded to `https://huggingface.co`, through a local https proxy that answers as huggingface.co with a test certificate trusted only by the test's own client. Every input runs in a child process. Crashes are bucketed by panic message and first ZML frame, then reduced.
- **Runtime and host APIs.** Property tests cover executables and donation, concurrency, buffers on meshes, Slice views, `zml.meta`, dtypes, tokenizers, `TensorStore`, paged attention and the KV cache. `zmlsmith robust` runs invalid API uses, one per process.
- **Oracle validation.** Injected bugs (`--mutate`) are caught: sqrt off by one ulp fails 15 of 600 seeds, and dropping the sign of a zero fails 3 of 600 (both rerun at `74c590b0`). A 1,000-seed default campaign at `74c590b0` had no failures, so no false positives.

2.2 Severity rubric

Each finding gets an **impact** score and an **exposure** score from 1 to 5. Their product is the **risk** score, which orders findings within a tier. Findings reached from untrusted input (files, network answers, user text) are marked, and that raises their tier.

CRITICAL	Memory corruption or undefined behavior reached from untrusted input or a common path, or silent wrong results on a default or common inference path, or a data race.
HIGH	Silent wrong values in a public API, a crash or hang triggered by untrusted input (denial of service), or a failure that blocks a common use case.
MEDIUM	A crash or compile failure on a valid but less common program, wrong metadata (tags, shapes), or an XLA compiler abort.
LOW	Missing validation of API misuse, host helpers, test utilities, or differences that only bit-exact code can see.
DESIGN	Surprising behavior that may be intended and needs a decision.

2.3 How each finding was verified

Every finding in this report went through the following checks after the fuzzing campaign. Findings that did not survive are listed in Appendix B with the reason.

1. **Run on the pristine commit.** `git archive #commit` was extracted into a fresh directory with only `tools/zmlsmith/` added, so no local change of the working tree can be involved, and every reproducer was run there with Bazel on CPU.
2. **Failure for the stated reason.** Each reproducer has a pattern (a panic message, an assertion text printed by the test, an XLA error, or a compiler error at the cited ZML line) that its log must contain. A test that fails for another reason does not count. The pattern and the matching log line are printed under each finding.
3. **Adversarial code review.** Every location, root cause and expected value was checked against the source at `74c590b0`, trying to refute the finding: is the call valid per the API's signature and doc comment, where does the expected value come from (a doc comment, the StableHLO spec, the safetensors or HTTP spec, the reference implementation in HF transformers or numpy), is the behavior documented or intended. The source of the expected behavior and the code lines are listed under each finding; the 542 cited code and spec lines were checked by script against the commit, and the 31 the script could not match literally (ranges, wrapped lines, paraphrased quotes) were checked by hand.
4. **Release build.** The memory findings were also run with the documented `--config=release` (ReleaseSafe, NDEBUG). What changes between the two modes is stated in each finding.
5. **Repeat runs.** The timing- and sampling-dependent reproducers (buffer pool race, IREE stack smash, nucleus sampling, `loadBuffer`, missing `Exe.call` arguments) were run 5 times each unpatched: each failed 5 of 5.

6. **Patch check.** For the critical findings and several high ones, the suggested fix was applied to the scratchpad copy of the commit and the reproducer was rerun: it passes with the fix (3 of 3 runs), which confirms the cause. The patch is shown under each finding; the copy was reverted afterwards.

Limits of the assessment.

- **CPU only.** Every result is from the CPU PJRT plugin. GPU and TPU behavior was not tested, and some findings (NaN min/max, the XLA CPU ones) may be CPU specific.
- **Debug builds.** The reproducers run in Bazel's default mode. Memory findings were observed as Debug safety panics or crashes; their release behavior is stated only where it was run.
- **Line numbers** refer to commit `74c590b0`.

2.4 Reproducing a finding

Every finding names one or more files in `tools/zmlsmith/regressions/found/`. Each file is its own Bazel test target, because most of them panic. They are expected to fail until the bug is fixed.

```
# one reproducer (file found/<name>.zig)
bazel test //tools/zmlsmith/regressions:found_<name> --test_output=errors
# every reproducer
bazel test //tools/zmlsmith/regressions:found --keep_going
# re-explore a fixed bug's input space: delete its gen.KnownBug entry, then
bazel build //tools/zmlsmith
bazel-bin/tools/zmlsmith/zmlsmith fuzz --known --count 400 --jobs 8 --out /tmp/zs
```

A compile bug's reproducer fails to build by design. 82 of 82 findings were reproduced at `74c590b0` with the expected failure.

3 Findings index

ID	Severity	Finding	Owner	Risk	Status
C01	CRITICAL	Loader.loadBuffer hands a dangling stack pointer to an async task	ZML	15	reproduced
C02	CRITICAL	DynamicBufferPool hands the same block to two owners under contention	ZML	15	reproduced
H01	HIGH	Any malformed tokenizer.json smashes the stack in default (non-NDEBUG) builds (IREE backend)	ZML	16	reproduced
H02	HIGH	SentencePiece streaming decode drops text after repeated tokens	ZML	16	reproduced
H03	HIGH	Long __metadata__ keys crash the safetensors parser (1024-byte stack buffer, unchecked append)	ZML	12	reproduced
H04	HIGH	Any HTTP redirect with a relative Location panics: the request URL it resolves against is overwritten	ZML	12	reproduced
H05	HIGH	SentencePiece Decoder keeps the ids of the previous call	ZML	12	reproduced
H06	HIGH	Tensor data offsets are not validated	ZML	12	reproduced
H07	HIGH	scatterSlices ignores the order of update axes	ZML	12	reproduced
H08	HIGH	Chained padded reduce_window gives wrong values	XLA	12	reproduced
H09	HIGH	Nucleus sampling drops the token that crosses top_p	ZML	12	reproduced
H10	HIGH	SentencePiece encode of an empty text panics	ZML	12	reproduced
H11	HIGH	Paged attention doesn't compile with fewer queries than page_size	ZML	12	reproduced
H12	HIGH	Malformed safetensors headers panic instead of failing (length, schema, dims)	ZML	9	reproduced
H13	HIGH	HTTP redirects are followed with no limit	ZML	9	reproduced
H14	HIGH	Answers without Content-Length panic (HTTP, S3, GCS and HF backends)	ZML	9	reproduced
H15	HIGH	HF repository listings ignore pagination	ZML	9	reproduced
H16	HIGH	Malformed Content-Range panics (HTTP and parallel_read)	ZML	9	reproduced
H17	HIGH	3xx without a valid Location panics	ZML	9	reproduced
H18	HIGH	resizeBilinear / Bicubic drop ResizeOpts.precision	ZML	9	reproduced
H19	HIGH	bf16 Rng.uniform falls outside [min, max)	ZML	9	reproduced
H20	HIGH	linspace doesn't end at `end`	ZML	9	reproduced
H21	HIGH	Slice .len is ignored	ZML	9	reproduced
H22	HIGH	Paged attention: wrong outputs for batches with an empty sequence or a partial prefill chunk	ZML	8	reproduced
H23	HIGH	Homemade tokenizer encode returns a slice that can't be freed	ZML	8	reproduced
H24	HIGH	FNUZ formats use the IEEE exponent bias	ZML	8	reproduced
H25	HIGH	scatterSlices with rank-0 indices ignores update_fn	ZML	8	reproduced
H26	HIGH	ops.while with the same tensor in two state slots	ZML	8	reproduced
H27	HIGH	broadcast ignores axes when dims already match	ZML	8	reproduced
H28	HIGH	Convolution with negative padding and permuted layouts gives wrong values	XLA	8	reproduced
H29	HIGH	SPMD reduce_window with base dilation and padding returns shifted values	XLA	8	reproduced
H30	HIGH	resizeBicubic permutes axes for axis >= 2	ZML	6	reproduced
H31	HIGH	maximum(clamp(lo, x, hi), lo) is simplified to the clamp	XLA	6	reproduced
H32	HIGH	f4e2m1 to integer saturates +-6.0	XLA	6	reproduced
M01	MEDIUM	TensorStore.View.hasKey matches by prefix	ZML	12	reproduced
M02	MEDIUM	maximum / minimum and max / min reductions don't propagate NaN on CPU	ZML	12	reproduced
M03	MEDIUM	Resize sample coordinates are computed in f16 / bf16	ZML	12	reproduced
M04	MEDIUM	zml.floats conversions don't round to nearest even and mishandle NaN	ZML	12	reproduced
M05	MEDIUM	Integer Rng.uniform is biased	ZML	9	reproduced
M06	MEDIUM	Quantized Linear / scaledDot don't compile on CPU	ZML	9	reproduced

ID	Severity	Finding	Owner	Risk	Status
M07	MEDIUM	pjrt struct helpers exceed the eval branch quota on long root module names	ZML	9	reproduced
M08	MEDIUM	Partitioning metadata survives ops that change the partitioned dim	ZML	9	reproduced
M09	MEDIUM	Exe.call passes undefined pointers when arguments are missing	ZML	8	reproduced
M10	MEDIUM	top_p = 1 still filters: the token whose f32 cumulative sum rounds above 1 is never sampled	ZML	6	reproduced
M11	MEDIUM	Homemade tokenizer: crashes on empty input, unknown UTF-8 and id == len	ZML	6	reproduced
M12	MEDIUM	Homemade streaming decode drops text	ZML	6	reproduced
M13	MEDIUM	Tokenizer.fromBytes indexes an empty file and misroutes JSON with leading whitespace	ZML	6	reproduced
M14	MEDIUM	Homemade Normalizer.fromHfJson overflows its 8-byte whitespace buffer	ZML	6	reproduced
M15	MEDIUM	RoPE yarn config without factor panics	ZML	6	reproduced
M16	MEDIUM	Tensor.print aborts for sub-byte, e8m0, e3m4, e4m3 and f4 dtypes	ZML	6	reproduced
M17	MEDIUM	Sub-byte Shape byte strides count whole bytes per element	ZML	6	reproduced
M18	MEDIUM	slices() with mixed static and dynamic slices produces invalid MLIR	ZML	6	reproduced
M19	MEDIUM	Sub-byte tensors: empty, odd last dim, rank 0 and rank 8 panic	ZML	6	reproduced
M20	MEDIUM	rope assumes .hd is the last axis	ZML	6	reproduced
M21	MEDIUM	XLA compiler aborts and verifier errors	XLA	6	reproduced
M22	MEDIUM	XLA CPU runtime failures: conv with reversal + stride + padding, FFT layout	XLA	6	reproduced
M23	MEDIUM	HF revisions containing a slash are mis-parsed	ZML	6	reproduced
M24	MEDIUM	RoPE scaling configs using `type` are rejected	ZML	6	reproduced
M25	MEDIUM	Crashes on valid inputs in small Tensor helpers	ZML	6	reproduced
M26	MEDIUM	Tensor.any, Tensor.all and pad with a slice don't compile	ZML	6	reproduced
M27	MEDIUM	broad / broadcastLeft / Right and nearest / upsample lose tags	ZML	6	reproduced
M28	MEDIUM	Homemade tokenizer loaders don't compile on Zig 0.16	ZML	4	reproduced
M29	MEDIUM	HF tree sizes overflow on large listings	ZML	4	reproduced
M30	MEDIUM	Tensor.print of an empty tensor doesn't compile	ZML	4	reproduced
M31	MEDIUM	GatedDeltaNet.forward only accepts one axis order	ZML	4	reproduced
M32	MEDIUM	Integer mean over a long axis panics while tracing	ZML	4	reproduced
M33	MEDIUM	Reading back an empty sharded buffer panics	ZML	4	reproduced
M34	MEDIUM	gather / scatter reject valid batching layouts	ZML	4	reproduced
M35	MEDIUM	Bicubic resize of axes shorter than 4 asserts	ZML	4	reproduced
M36	MEDIUM	GSPMD: partitionId inside manualComputation doesn't compile	ZML	4	reproduced
M37	MEDIUM	maxPool indices are wrong when the max is the dtype minimum	ZML	4	reproduced
M38	MEDIUM	Rank-0 Slice views print the first element of the buffer	ZML	4	reproduced
M39	MEDIUM	leakyReLU is wrong for negative slopes	ZML	3	reproduced
M40	MEDIUM	Simplifier rewrites ignore IEEE zeros: -0 + 0, and convert(bool) * x with x = -3, inf or NaN	XLA	3	reproduced
L01	LOW	Constant additions are reassociated	XLA	8	reproduced
L02	LOW	reuseBuffer doesn't check that input and output share a sharding	ZML	4	reproduced
L03	LOW	Shape helpers: reshape(-1) of an empty shape, and `._N` past the rank	ZML	4	reproduced
L04	LOW	DataType.Value.as and mantissaSize miss f8 / f4 / 2-bit dtypes	ZML	4	reproduced
L05	LOW	zml.testing.expectClose panics on bool slices and on identical f4e2m1, f8e3m4, f8e4m3, f8e8m0 slices	ZML	3	reproduced
L06	LOW	DataType.resolvePeerType is not commutative and can lose values	ZML	2	reproduced
L07	LOW	customCall array outputs don't compile	ZML	2	reproduced
D01	DESIGN	.coord is a reserved tag in gather	ZML	1	reproduced

ID	Severity	Finding	Owner	Risk	Status
D02	DESIGN	Quantized scales are tagged differently	ZML	1	no target
D03	DESIGN	maxPool2d indices hold only the column of the max	ZML	1	no target
D04	DESIGN	Buffer.toSlice fails on buffers sharded on any axis but the first	ZML	1	no target

4 Critical findings

C01 Loader.loadBuffer hands a dangling stack pointer to an async task

CRITICAL ZML memory · IO · risk 15 (impact 5 × exposure 3)

API io.Loader.loadBuffer
Location zml/io.zig:428
Platform CPU

What happens. Loading one tensor through `loadBuffer` panics in the default build with "Tags, dims and partitioning have diverged!": the shape read by the task is garbage. With `--config=release` there is no panic: the load completes and the buffer holds wrong bytes (the reproducer's comparison fails at index 0). The same round trip through `Loader.load` passes.

Root cause. `loadBuffer` passes `&tensor`, the address of its by-value parameter, to the async load task. The task runs after `loadBuffer` returned and reads a dead stack slot.

> **Risk assessment.** Use after return, reached by any caller of the public single-buffer loader. In the documented release build it silently loads wrong weights (verified), which is the worst outcome for a loader.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_load_buffer --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

```
load_buffer: fails (matches Tags, dims and partitioning have diverged)
1/1 load_buffer.test.loader_rt_load_buffer...thread 3155437 panic: Tags, dims and partitioning have diverged! dims=1 tags=1
partitioning=140723376573480
```

Verification: confirmed as reported. **Expected behavior, from:** Io.Group.async may run the task after the caller returns, so a pointer to a by-value parameter of loadBuffer is dangling in the task. The public signature takes `tensor: Tensor` by value (zml/io.zig:427). Expected behaviour is the Loader.load round trip, which the reproducer already checks against the same data.
Root cause checked by patching: `loadBuffer` calls `defaultCallback` directly instead of handing `&tensor` to `group.async` (zml/io.zig:428). `found_load_buffer` passes (3 of 3 runs); unpatched it fails 5 of 5.

- zml/io.zig:427: pub fn loadBuffer(loader: *Loader, io: std.Io, buffer: *Buffer, tensor: Tensor, store: *const TensorStore, shardings: []const Sharding, opts: LoadOpts) void {
- zml/io.zig:428: loader.group.async(io, defaultCallback, .{ loader, io, &tensor, buffer, store, shardings, opts });
- zml/io.zig:431: fn defaultCallback(self: *Loader, io: std.Io, tensor: *const Tensor, buffer: *Buffer, store: *const TensorStore, shardings: []const Sharding, opts: LoadOpts) void {

Suggested fix. Copy the tensor into the task's own state, or await the task before returning.

tools/zmlsmith/regressions/found/load_buffer.zig (94 lines, first 70 shown)

```
const std = @import("std");
const zml = @import("zml");

// io.Loader.loadBuffer passes `&tensor`, the address of its by-value parameter, to the
// async load task (zml/io.zig:428 at 74c590b0). The task runs after loadBuffer returned
// and reads a dead stack slot: the tensor shape is garbage ("Tags, dims and partitioning
// have diverged! ... partitioning=140728831883192") and it panics. The same round trip
// through Loader.load passes (regressions/loader.zig). Found while auditing zmlsmith
// coverage of zml/io.zig.

const Entry = struct { name: []const u8, dtype: []const u8, dims: []const i64, bytes: usize };
const entries = [_]Entry{
    .{ .name = "w", .dtype = "F32", .dims = &.{ 3, 5 }, .bytes = 60 },
    .{ .name = "b", .dtype = "BF16", .dims = &.{7}, .bytes = 14 },
    .{ .name = "s", .dtype = "I8", .dims = &.{}, .bytes = 1 },
    .{ .name = "big", .dtype = "U8", .dims = &.{1000}, .bytes = 1000 },
};

fn writeCheckpoint(io: std.Io, dir: std.Io.Dir, data: []u8) !void {
    var header: std.Io.Writer.Allocating = .init(std.testing.allocator);
    defer header.deinit();
    try header.writer.writeAll("{");
    var off: usize = 0;
    for (entries, 0..) |e, i| {
        if (i > 0) try header.writer.writeAll(",");
        try header.writer.print("\"{s}\":{{\"dtype\":\"{s}\",\"shape\":[", .{ e.name, e.dtype });
        for (e.dims, 0..) |d, j| try header.writer.print("{s}{d}", .{ if (j > 0) "," else "", d });
        try header.writer.print("],\"data_offsets\":[{d},{d}]}}", .{ off, off + e.bytes });
        off += e.bytes;
    }
    try header.writer.writeAll("}");
    for (data, 0..) |*x, i| x.* = @truncate(i *% 131 +% 7);
    var file: std.Io.Writer.Allocating = .init(std.testing.allocator);
    defer file.deinit();
    try file.writer.writeInt(u64, header.written().len, .little);
    try file.writer.writeAll(header.written());
```

```
    try file.writer.writeAll(data);
    try dir.writeFile(io, .{ .sub_path = "m.safetensors", .data = file.written() });
}

const Model = struct { w: zml.Tensor, b: zml.Tensor, s: zml.Tensor, big: zml.Tensor };

fn roundTrip(parallelism: usize, chunk: usize, use_load_buffer: bool) !void {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    var tmp = std.testing.tmpDir(.{});
    defer tmp.cleanup();
    var data: [1075]u8 = undefined;
    try writeCheckpoint(io, tmp.dir, &data);

    var path_buf: [1024]u8 = undefined;
    const dir_len = try tmp.dir.realPath(io, &path_buf);
    const path = try std.fmt.allocPrint(allocator, "{s}/m.safetensors", .{path_buf[0..dir_len]});
    defer allocator.free(path);
    var registry: zml.safetensors.TensorRegistry = try .fromPath(allocator, io, path);
    defer registry.deinit();
    var store: zml.io.TensorStore = .fromRegistry(allocator, &registry);
    defer store.deinit();
    const v = store.view();
    const model: Model = .{
        .w = v.createTensor("w", null, .replicated),
        .b = v.createTensor("b", null, .replicated),
        .s = v.createTensor("s", null, .replicated),
        .big = v.createTensor("big", null, .replicated),
    };
    var buffers = try zml.mem.bufferize(allocator, Model, &model);
    var loader: zml.io.Loader = try .init(allocator, platform, .{ .parallelism = parallelism, .dma_chunks = 2, .dma_chunk_size = chunk });
    defer loader.deinit();
    const shardings: []const zml.Sharding = &.{platform.replicated_sharding};
```

C02 DynamicBufferPool hands the same block to two owners under contention

CRITICAL ZML race · Memory · risk 15 (impact 5 × exposure 3)

API mem.DynamicBufferPool get/put
Location `zml/mem.zig:282-310 (DynamicBufferPool.put 282-298, pop 300-310; the stale read is zml/mem.zig:303)`
Platform CPU

What happens. With 8 threads each holding two blocks and checking a per-thread stamp, blocks are handed to two owners at once: 1,117,138 foreign stamps in the run at 74c590b0, 578,111 with `--config=release`.

Root cause. Classic Treiber-stack ABA in DynamicBufferPool.pop (zml/mem.zig:300-310). pop loads head X, reads X.next (Y) at zml/mem.zig:303 before the compare-exchange, then is preempted. Other threads pop X and Y, then push X back. The CAS head X -> Y then succeeds although Y is in use, so Y is handed out a second time. The stamp-based detection in the reproducer shows exactly this double ownership. A second, separate defect: Node.next is a plain `?*Node` (zml/mem.zig:227), read in pop (zml/mem.zig:303) while put writes it (zml/mem.zig:284-287) on a block that may be concurrently popped. That is a non-atomic data race, which is undefined behavior in Zig independent of the ABA.

> **Risk assessment.** Two owners of one buffer means silent data corruption in whatever the pool backs (the Loader's pinned staging buffers), and it only shows under concurrent load. The reproducer repeats rounds until the first collision because a quiet machine can pass a round.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_pool_aba --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `pool_aba`: *fails* (matches `expected 0, found [1-9][0-9]*`)
   `1/1 pool_aba.test.pool_aba...expected 0, found 1117138`

Verification: confirmed, report corrected. **Expected behavior, from:** A lock-free stack pop must not install a successor read before the head is confirmed unchanged. A block must have at most one owner (the pool contract, zml/mem.zig:250-280 get/put), so a second owner is a violation of the pool's own contract.
Root cause checked by patching: `DynamicBufferPool.get` takes `self.mutex` around `pop()` (zml/mem.zig:259). `found_pool_aba` passes (3 of 3 runs, 20 rounds of 8 threads each); unpatched it fails 5 of 5.

- zml/mem.zig:227: `const Node = struct { next: ?*Node };` (plain, non-atomic)
- zml/mem.zig:284-287: `node.next = head;` then `self.free_stack.cmpxchgWeak(head, node, .release, .acquire)`
- zml/mem.zig:303: `if (self.free_stack.cmpxchgWeak(head, if (head) |h| h.next else null, .acquire, .acquire)) |actual| {`
- zml/mem.zig:296-297: `_ = self.in_flight.fetchSub(1, .release); self.condition.signal(io);` (put's bookkeeping happens after the push)

Suggested fix. Make the free list safe under contention: either protect pop with the existing self.mutex (put already takes it at zml/mem.zig:294, so the list can be guarded by one lock), or use a tagged head (version counter, double-width CAS) and an atomic Node.next. The mutex option is the simplest correct fix.

tools/zmlsmith/regressions/found/pool_aba.zig (46 lines)

```
const std = @import("std");
const zml = @import("zml");

// mem.DynamicBufferPool keeps free blocks in a lock-free stack whose pop reads
// `head.next` before the compare-exchange, without an ABA guard (zml/mem.zig:303 at
// 74c590b0). When another thread pops head and its successor and pushes head back in
// between, the stale `next` is installed and a block still in use is handed out twice.
// Each thread here stamps the blocks it holds and checks the stamp before returning them.
// The race needs threads running at the same time: under load (a whole test suite in
// parallel) a round can pass, so rounds are repeated until the first collision.
// Found by reading the code while auditing zmlsmith coverage.

const threads = 8;
const iters = 100_000;

var pool: zml.mem.DynamicBufferPool = .init(threads * 2, 64);
var double_owned: std.atomic.Value(u32) = .init(0);

fn worker(id: u8) void {
    const io = std.testing.io;
    var held: [2][]u8 = undefined;
    for (0..iters) |_| {
        for (&held) |*b| {
            b.* = pool.get(std.heap.page_allocator, io) catch unreachable;
            @memset(b.*[8..], id);
        }
        for (held) |b| {
            for (b[8..]) |x| if (x != id) {
                _ = double_owned.fetchAdd(1, .monotonic);
                break;
            };
            pool.put(io, b);
        }
    }
}

test "pool_aba" {
    for (0..20) |_| {
        var ts: [threads]std.Thread = undefined;
        for (&ts, 0..) |*t, i| t.* = try std.Thread.spawn(.{}, worker, .{@as(u8, @intCast(i + 1))});
        for (ts) |t| t.join();
        if (double_owned.load(.monotonic) > 0) break;
    }
    try std.testing.expectEqual(@as(u32, 0), double_owned.load(.monotonic));
}
```

5 High findings

H01 Any malformed tokenizer.json smashes the stack in default (non-NDEBUG) builds (IREE backend)

HIGH ZML memory · Tokenizer · risk 16 (impact 4 × exposure 4) · **untrusted input**

API tokenizer.Tokenizer.fromBytes / fromFile (IREE)
Location `zml/tokenizer/iree/tokenizer.zig:67-74 (checkOk; buffer at :70, iree_status_format call at :72-74); <output_base>/external/+non_module_deps+iree/runtime/src/iree/base/status_stack_trace.c:60-81 (append helpers), :358-455 (stack-trace frame formatter); .../iree/base/status.c:909-917 (payload loop)`
Platform CPU

What happens. Loading an invalid tokenizer.json, even `{}`, kills the process with a general protection fault on return from `checkOk` in Bazel's default build mode (`--compilation_mode=dbg` in .bazelrc, no NDEBUG, so IREE_STATUS_MODE 3). With the documented `--config=release` (NDEBUG, mode 2) the same reproducer gets a proper error back and logs it: verified by running both.

Root cause. `checkOk` formats the IREE error status into a 1024-byte stack buffer. In IREE_STATUS_MODE 3 the status carries a stack-trace payload, and its frame formatter (status_stack_trace.c) adds the untruncated snprintf length to buffer_length, so once the buffer is full `buffer_capacity - buffer_length` wraps around and `snprintf` writes past the end of the stack buffer. IREE_STATUS_MODE defaults to 3 when NDEBUG is not defined and to 2 otherwise (iree/base/config.h:114-122); the payload is compiled only in mode 3 (iree/base/status.h:62-65).

> **Risk assessment.** The tokenizer file comes from the model repository, so it is an out-of-process input. Every developer and test build (the default mode) turns a malformed or unsupported file into stack corruption instead of an error. Release builds as documented are not affected. The LLM example loads tokenizer.json through this path (examples/llm/main.zig:183-197).

Reproduction.
1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_tok_iree_error --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `tok_iree_error`: *fails* (matches `General protection exception|Illegal instruction|SIGILL|SIGSEGV`)
   `General protection exception (no address available)`

Verification: confirmed, report corrected. **Expected behavior, from:** IREE runtime source in the bazel external repo (status.h, config.h, status.c, status_stack_trace.c). The buffer wrap is a plain unsigned-arithmetic consequence of the helpers, not an assumption.
- zml/tokenizer/iree/tokenizer.zig:70: var buffer: [1024]u8 = undefined;
- zml/tokenizer/iree/tokenizer.zig:72: if (c.iree_status_format(status, buffer.len, &buffer, &buffer_length)) {
- status_stack_trace.c:60-66: iree_string_buffer_append_cstr ... return IREE_UNLIKELY(n < 0) ? 0 : buffer_length + n; (untruncated n added)
- status_stack_trace.c:78: buffer ? buffer_capacity - buffer_length : 0, format, varargs); (unsigned wrap once buffer_length > capacity)
- config.h:114-122: #if !defined(IREE_STATUS_MODE) ... #ifdef NDEBUG #define IREE_STATUS_MODE 2 #else #define IREE_STATUS_MODE 3
- status.h:62-67: #elif defined(IREE_STATUS_MODE) && IREE_STATUS_MODE == 3 ... IREE_STATUS_FEATURE_STACK_TRACE
- status.c:911-913: payload->formatter(payload, buffer ? buffer_capacity - buffer_length : 0, buffer ? buffer + buffer_length : NULL, &payload_buffer_length);
- external IREE BUILD: build_tools/third_party/libbacktrace/BUILD.bazel:31 defines IREE_HAVE_LIBBACKTRACE=1, so the Linux stack-trace formatter is compiled in

Suggested fix. Size the buffer from a first `iree_status_format` call with no buffer, or use `iree_status_format_to` with an allocating writer.

`tools/zmlsmith/regressions/found/tok_iree_error.zig` (23 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Any malformed tokenizer.json, even "{}", smashes the stack: iree.checkOk formats the
// error status into a 1024 byte stack buffer (zml/tokenizer/iree/tokenizer.zig:70-72 at
// 74c590b0), and the stack trace payload formatter of IREE keeps adding the untruncated
// length, so once the buffer is full `capacity - length` wraps and snprintf writes past it
// (iree/base/status_stack_trace.c:60-81). The process dies with SIGILL or a general
// protection fault on return from checkOk instead of fromBytes returning an error.
// Found by zmlsmith parse-fuzz.
test "tok_iree_error" {
    const inputs = [_][]const u8{
        "{\"\":\"\",\"\":{\"\":\"\",\"\":n",
        "{}",
    };
    for (inputs) |in| {
```

```
        if (zml.tokenizer.Tokenizer.fromBytes(std.testing.allocator, in)) |t| {
            var tok = t;
            tok.deinit();
        } else |_| {}
    }
}
```

H02 SentencePiece streaming decode drops text after repeated tokens

HIGH ZML wrong values · Tokenizer · risk 16 (impact 4 × exposure 4)

API tokenizer sentencepiece Decoder.feedOne
Location `zml/tokenizer/sentencepiece/sentencepiece.zig:140-165 (Decoder.next; window of 4 at :87 and reserve at :96; suffix match at :154-163)`
Platform CPU

What happens. Ids {6, 3, 3, 3, 6, 4} stream "a " where batch decoding gives "a aa".

Root cause. The streamer decodes a sliding window of the last 4 ids and emits what wasn't in the previous output, matched by suffix. Once the window slides over repeated tokens the new decode equals the old one and text is lost.

> **Risk assessment.** This is the path used to print generated tokens, so user-visible output is silently missing text whenever the model repeats tokens (spaces, punctuation, code indentation).

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_spm_stream --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

```
spm_stream: fails (matches streamed \"a  \", batch \"a  aa\")
1/1 spm_stream.test.spm_stream...ids { 6, 3, 3, 3, 6, 4 }: streamed "a  ", batch "a  aa"
```

Verification: confirmed, report corrected. **Expected behavior, from:** Streaming contract: the concatenation of feedOne outputs plus finalize must equal a batch decode of the same ids (the streamer's own design, tokenizer.zig:111-124 documents feedOne as incremental output). The batch decode (decode, sentencepiece.zig:120-128) is the reference.

- zml/tokenizer/sentencepiece/sentencepiece.zig:87: const TokenIdsBufferSize = 4; / :96: c.std_vector_int_reserve(vec, TokenIdsBufferSize);
- zml/tokenizer/sentencepiece/sentencepiece.zig:140-146: pub fn next(...) { const current_ids = self.ids(); if (current_ids.len >= c.std_vector_int_capacity(self.vec)) { std.mem.copyForwards(...) ... } else { c.std_vector_int_push_back(...) }
- zml/tokenizer/sentencepiece/sentencepiece.zig:158-162: if (std.mem.startsWith(u8, new_string, self.last_string.constSlice()[start..])) { const chunk = new_string[self.last_string.len - start ..]; ... return chunk;
- zml/tokenizer/sentencepiece/sentencepiece.zig:164: return &.{};

Suggested fix. Track the decoded byte length of the prefix instead of diffing windows, or decode incrementally and emit only complete UTF-8.

`tools/zmlsmith/regressions/found/spm_stream.zig` (42 lines)

```
const std = @import("std");
const zml = @import("zml");

// Streaming decode with the sentencepiece backend (Decoder.feedOne then finalize) against
// a batch decode of the same ids with a fresh decoder, on random id sequences. next()
// decodes a sliding window of the last 4 ids and emits the part of the result that wasn't
// in the previous one, matched by suffix (zml/tokenizer/sentencepiece/sentencepiece.zig:
// 137-163 at 74c590b0): once the window slides over repeated tokens the new decode equals
// the old one and the next token's text is never emitted: ids {6, 3, 3, 3, 6, 4} stream
// "a  " where the batch decode is "a  aa". Found by zmlsmith tokenizer
// property tests.
/// A minimal sentencepiece ModelProto: <unk>, <s>, </s>, then _ a b _a ab.
const model = [_]u8{ 10, 14, 10, 5, 60, 117, 110, 107, 62, 21, 0, 0, 0, 0, 24, 2, 10, 12, 10, 3, 60, 115, 62, 21, 0, 0, 0, 0, 24, 3, 10, 13, 10, 4, 60, 47, 115,
62, 21, 0, 0, 0, 0, 24, 3, 10, 12, 10, 3, 226, 150, 129, 21, 0, 0, 128, 191, 24, 1, 10, 10, 10, 1, 97, 21, 0, 0, 0, 192, 24, 1, 10, 10, 10, 1, 98, 21, 0, 0, 0,
192, 24, 1, 10, 13, 10, 4, 226, 150, 129, 97, 21, 0, 0, 192, 191, 24, 1, 10, 11, 10, 2, 97, 98, 21, 0, 0, 128, 191, 24, 1 };

test "spm_stream" {
    const allocator = std.testing.allocator;
    var tok = try zml.tokenizer.Tokenizer.fromBytes(allocator, &model);
    defer tok.deinit();
    var prng: std.Random.DefaultPrng = .init(5);
    const rng = prng.random();
    for (0..300) |_| {
        var ids: [12]u32 = undefined;
        const n = rng.intRangeAtMost(usize, 1, ids.len);
        for (ids[0..n]) |*id| id.* = rng.intRangeAtMost(u32, 3, 7);
        var batch_dec = try tok.decoder();
        defer batch_dec.deinit();
        var want = try batch_dec.decodeAlloc(allocator, ids[0..n]);
        defer want.deinit(allocator);
        var dec = try tok.decoder();
        defer dec.deinit();
        var got: std.ArrayList(u8) = .empty;
```

```
        defer got.deinit(allocator);
        var buf: [256]u8 = undefined;
        for (ids[0..n]) |id| try got.appendSlice(allocator, try dec.feedOne(id, &buf));
        try got.appendSlice(allocator, try dec.finalize(&buf));
        if (!std.mem.eql(u8, got.items, want.items)) {
            std.debug.print("ids {any}: streamed \"{s}\", batch \"{s}\"\n", .{ ids[0..n], got.items, want.items });
            return error.TestUnexpectedResult;
        }
    }
}
```

H03 Long __metadata__ keys crash the safetensors parser (1024-byte stack buffer, unchecked append)

HIGH ZML crash · Safetensors · risk 12 (impact 4 × exposure 3) · **untrusted input**

API safetensors.parseSafetensors
Location `zml/safetensors.zig:783-847`
Platform CPU

What happens. A metadata key of 1100 characters, or deep enough nesting, trips the capacity assert of `appendSliceAssumeCapacity` instead of failing the load: a safety panic in the default build and also with the documented `--config=release` (ReleaseSafe), both verified. Only ReleaseFast or ReleaseSmall builds, which the repository does not configure, would write past the 1024-byte stack buffer.

Root cause. Metadata keys are flattened into a 1024-byte stack buffer with `appendSliceAssumeCapacity`, which does not check capacity.

> **Risk assessment.** A checkpoint file is untrusted input (downloaded from a hub). In the configured build modes a malicious or corrupt header takes the process down (denial of service); in an unchecked build mode it would be a stack buffer overflow.

Reproduction.
1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_st_meta_key --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `st_meta_key`: *fails* (matches `panic: reached unreachable code`)
   `1/1 st_meta_key.test.st_meta_key...thread 3163549 panic: reached unreachable code`

Verification: confirmed, report corrected. **Expected behavior, from:** Zig std ArrayListUnmanaged.appendSliceAssumeCapacity asserts that the new length fits capacity (lib/std/array_list.zig:300-306 in the Zig 0.16.0 std of the bazel output base). The safetensors format only requires a JSON header with string metadata and states no key length limit, so a 1100 character key should load or fail with an error, not panic.
Root cause checked by patching: populateMetadata returns `error.InvalidMetadata` when `prefix.items.len + 1 + key.len` exceeds the buffer (zml/safetensors.zig:844). `found_st_meta_key` passes (3 of 3 runs).
- zml/safetensors.zig:783: var prefix_buf: [1024]u8 = undefined;
- zml/safetensors.zig:784: var prefix = StringBuilder.initBuffer(&prefix_buf);
- zml/safetensors.zig:7: const StringBuilder = std.ArrayListUnmanaged(u8);
- zml/safetensors.zig:847: prefix.appendSliceAssumeCapacity(entry.key_ptr.*);
- lib/std/array_list.zig:303: assert(new_len <= self.capacity);

Suggested fix. Use a growable writer, or return an error when a key exceeds the buffer.

`tools/zmlsmith/regressions/found/st_meta_key.zig` (31 lines)

```
const std = @import("std");
const zml = @import("zml");

// parseSafetensors flattens __metadata__ keys into a 1024 byte stack buffer with
// appendSliceAssumeCapacity (zml/safetensors.zig:783-847 at 74c590b0): a metadata key of
// 1100 characters, or deep enough nesting, writes past the buffer (a panic in Debug,
// memory corruption in release) instead of failing the load. Found while auditing
// zmlsmith coverage of zml/safetensors.zig.
test "st_meta_key" {
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    var tmp = std.testing.tmpDir(.{});
    defer tmp.cleanup();
    var header: std.Io.Writer.Allocating = .init(allocator);
    defer header.deinit();
    try header.writer.writeAll("{\"__metadata__\":{\"");
    try header.writer.splatByteAll('k', 1100);
    try header.writer.writeAll("\":\"v\"},\"w\":{\"dtype\":\"F32\",\"shape\":[1],\"data_offsets\":[0,4]}}");
```

```
    var file: std.Io.Writer.Allocating = .init(allocator);
    defer file.deinit();
    try file.writer.writeInt(u64, header.written().len, .little);
    try file.writer.writeAll(header.written());
    try file.writer.writeAll(&.{ 0, 0, 0, 0 });
    try tmp.dir.writeFile(io, .{ .sub_path = "m.safetensors", .data = file.written() });
    const f = try tmp.dir.openFile(io, "m.safetensors", .{});
    defer f.close(io);
    var registry: zml.safetensors.TensorRegistry = .init(allocator);
    defer registry.deinit();
    zml.safetensors.parseSafetensors(allocator, io, &registry, f) catch {};
}
```

H04 Any HTTP redirect with a relative Location panics: the request URL it resolves against is overwritten

HIGH ZML memory · VFS · risk 12 (impact 4 × exposure 3) · **untrusted input**

API vfs HTTP open (fetchSize)
Location `vfs/http.zig:317, :321-323, :345-346`
Platform CPU

What happens. Opening a file whose server answers `302` with `Location: g.bin` panics with `attempt to unwrap error: InvalidHostName` from the `catch unreachable` at vfs/http.zig:346: the copy of the Location has overwritten the scheme and host of the base URL. With other Location lengths the overlapping buffers trip `@memcpy arguments alias` inside std.Uri instead (the parse-fuzz finding). Release builds were not run for this finding.

Root cause. fetchSize writes the request URL into aux_buffer, which is &redirect_buffer (vfs/http.zig:317, :321), and parses it (:323), so the base `uri` points into that buffer. On a 3xx the Location is copied to aux_buffer[0..len] (:345) because Uri.resolveInPlace expects the relative reference at the start of the buffer (Uri.zig:395-397). That copy overwrites the base scheme and host bytes. merge_paths (Uri.zig:510-516) then formats the base path into the same buffer, where it overlaps its own source, and the std alias check fires. The Location bytes are not in redirect_buffer: std Client.zig:1146-1148 parses the head from the connection reader, and HEAD responses return before redirect handling (Client.zig:1174), so the copy at :345 does not itself overlap.

> **Risk assessment.** Triggered by an ordinary server response: relative redirects are legal (RFC 9110 section 10.2.2) and common behind proxies and CDNs. Every such redirect takes the process down instead of being followed. The input comes from the network.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_vfs_redirect_alias --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `vfs_redirect_alias`: *fails* (matches `attempt to unwrap error: InvalidHostName|@memcpy arguments alias`)
   `1/1 vfs_redirect_alias.test.vfs_redirect_alias...thread 3175566 panic: attempt to unwrap error: InvalidHostName`

Verification: confirmed, report corrected. **Expected behavior, from:** std.Uri.resolveInPlace (Uri.zig:395) requires the relative reference at aux_buf[0..new_len] and reads the base URI afterwards, so the base URI must stay intact. Overlapping @memcpy is illegal in Zig and checked for in Debug.
Root cause checked by patching: fetchSize formats the request URL into its own `url_buffer` instead of `aux_buffer` (vfs/http.zig:321). `found_vfs_redirect_alias` passes (3 of 3 runs): the redirect to g.bin is followed and the 4 bytes are read.

- vfs/http.zig:317: var aux_buffer: []u8 = &redirect_buffer;
- vfs/http.zig:321: const full_url = try std.fmt.bufPrint(aux_buffer, "{s}://{s}", .{ scheme, url });
- vfs/http.zig:323: var uri = std.Uri.parse(full_url) catch return std.Io.File.OpenError.BadPathName;
- vfs/http.zig:344-345: const location = res.head.location.?; @memcpy(aux_buffer[0..location.len], location);
- vfs/http.zig:346: uri = uri.resolveInPlace(location.len, &aux_buffer) catch unreachable;
- lib/std/Uri.zig:395-397: pub fn resolveInPlace(base: Uri, new_len: usize, aux_buf: *[]u8) ResolveInPlaceError!Uri { const new = aux_buf.*[0..new_len];
- lib/std/Uri.zig:510-516: fn merge_paths(base: Component, new: []u8, aux_buf: *[]u8) ... base.formatPath(&aux) ...
- lib/std/http/Client.zig:1148: .head = Response.Head.parse(head_buffer) catch return error.HttpHeadersInvalid,
- lib/std/http/Client.zig:1174: if (r.method == .HEAD or head.status.class() == .informational or

Suggested fix. Keep the request URL and the relative reference in separate buffers, so the base URI is not overwritten before it is read. A plain @memmove is not enough: it still overwrites the base URL bytes that resolveInPlace reads.

`tools/zmlsmith/regressions/found/vfs_redirect_alias.zig` (73 lines, first 70 shown)

```
const std = @import("std");
const zml = @import("zml");

// vfs HTTP fetchSize writes the request URL into aux_buffer (== &redirect_buffer) and parses
// it there, so the base URI points into that buffer (vfs/http.zig:317, :321-323 at
// 74c590b0). On a 3xx a relative Location is copied to the start of the same buffer (:345)
// before Uri.resolveInPlace reads the base, which overwrites it. With `Location: g.bin` the
// base host is then garbage and the `catch unreachable` at :346 panics (InvalidHostName);
// other lengths trip "@memcpy arguments alias" inside std.Uri. Found by zmlsmith parse-fuzz
// (vfs target). Once fixed, the redirect to g.bin is followed and the 4 bytes are read.

/// Answers connection i with `answers[i]`, recording each request line.
const Server = struct {
    server: std.Io.net.Server,
    answers: []const []const u8,
    lines: [4][64]u8 = undefined,
    lens: [4]usize = .{ 0, 0, 0, 0 },

    fn run(s: *Server) void {
        const io = std.testing.io;
        for (s.answers, 0..) |answer, i| {
            const conn = s.server.accept(io) catch return;
            defer conn.close(io);
            var rbuf: [4096]u8 = undefined;
            var r = conn.reader(io, &rbuf);
            const first = r.interface.takeDelimiterExclusive('\n') catch return;
            r.interface.toss(1);
            s.lens[i] = @min(first.len, 64);
            @memcpy(s.lines[i][0..s.lens[i]], first[0..s.lens[i]]);
            while (true) {
                const line = r.interface.takeDelimiterExclusive('\n') catch return;
                r.interface.toss(1);
                if (line.len <= 1) break;
            }
            var wbuf: [4096]u8 = undefined;
            var w = conn.writer(io, &wbuf);
            w.interface.writeAll(answer) catch return;
            w.interface.flush() catch return;
        }
    }
};

test "vfs_redirect_alias" {
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const addr = try std.Io.net.IpAddress.parseIp4("127.0.0.1", 0);
    var s: Server = .{ .server = try addr.listen(io, .{ .reuse_address = true }), .answers = &.{
        "HTTP/1.1 302 Found\r\nConnection: close\r\nLocation: g.bin\r\nContent-Length: 0\r\n\r\n",
        "HTTP/1.1 200 OK\r\nConnection: close\r\nContent-Length: 4\r\n\r\n",
        "HTTP/1.1 200 OK\r\nConnection: close\r\nContent-Length: 4\r\n\r\nabcd",
    } };
    defer s.server.deinit(io);
    const t = try std.Thread.spawn(.{}, Server.run, .{&s});
    defer t.join();

    var client: std.http.Client = .{ .allocator = allocator, .io = io };
    defer client.deinit();
    var http: zml.io.VFS.HTTP = try .init(allocator, io, &client, .http);
    defer http.deinit();
    var vfs: zml.io.VFS = try .init(allocator, io);
    defer vfs.deinit();
    try vfs.register("http", http.io());
    const vio = vfs.io();
    var url_buf: [64]u8 = undefined;
    const url = try std.fmt.bufPrint(&url_buf, "http://127.0.0.1:{d}/d/f.bin", .{s.server.socket.address.getPort()});
    var file = try std.Io.Dir.openFile(.cwd(), vio, url, .{ .mode = .read_only });
    defer file.close(vio);
    try std.testing.expectStringStartsWith(s.lines[1][0..s.lens[1]], "HEAD /d/g.bin ");
    var buf: [4]u8 = undefined;
    const n = try file.readPositional(vio, &.{&buf}, 0);
```

H05 SentencePiece Decoder keeps the ids of the previous call

HIGH ZML wrong values · Tokenizer · risk 12 (impact 4 × exposure 3)

API tokenizer sentencepiece Decoder.decode
Location `zml/tokenizer/sentencepiece/sentencepiece.zig:120-128`
Platform CPU

What happens. Decoding [a] then [b] with the same decoder gives "ab".

Root cause. `decode` pushes the ids into its vector without clearing it first.

> **Risk assessment.** Silent wrong text whenever a decoder is reused, e.g. one per request in a server.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_spm_decode_stale --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `spm_decode_stale`: *fails* (matches `found \"ab\"|'\\x61'\)`)
   `^ ('\x61')`

Verification: confirmed as reported. **Expected behavior, from:** Decoder API contract: decode(ids) returns the text of exactly those ids. Stateful reuse must not carry previous ids; reset() exists for that purpose (sentencepiece.zig:56-59 clears the vector). The reproducer's expectation 'b' follows from that contract.
Root cause checked by patching: `Decoder.decode` clears the id vector first (zml/tokenizer/sentencepiece/sentencepiece.zig:120). `found_spm_decode_stale` passes (3 of 3 runs).
- zml/tokenizer/sentencepiece/sentencepiece.zig:120-127: pub fn decode(self: *Decoder, ids_: []const u32) ![]const u8 { c.std_vector_int_reserve(self.vec, ids_.len); ... for (ids_) |id| { c.std_vector_int_push_back(self.vec, @intCast(id)); } ... return self.string();
- zml/tokenizer/sentencepiece/sentencepiece.zig:115-118: pub fn reset(self: *Decoder) void { c.std_vector_int_clear(self.vec); c.std_string_clear(self.str); }
- zml/tokenizer/tokenizer.zig:106: inline else => |*d| .fromOwnedSlice(try allocator.dupe(u8, try d.decode(token_ids))), (decodeAlloc reaches the stale path)

Suggested fix. Clear the vector at the start of `decode`.

`tools/zmlsmith/regressions/found/spm_decode_stale.zig` (23 lines)

```
const std = @import("std");
const zml = @import("zml");

// sentencepiece Decoder.decode pushes the ids into its std::vector without clearing it
// first (zml/tokenizer/sentencepiece/sentencepiece.zig:120-128 at 74c590b0): a second
// decode with the same decoder also decodes the ids of the first call. Found by reading
// the code while auditing zmlsmith, confirmed with a model built by the spm target.
/// A minimal sentencepiece ModelProto: <unk>, <s>, </s>, then _ a b _a ab.
const model = [_]u8{ 10, 14, 10, 5, 60, 117, 110, 107, 62, 21, 0, 0, 0, 0, 24, 2, 10, 12, 10, 3, 60, 115, 62, 21, 0, 0, 0, 0, 24, 3, 10, 13, 10, 4, 60, 47, 115,
62, 21, 0, 0, 0, 0, 24, 3, 10, 12, 10, 3, 226, 150, 129, 21, 0, 0, 128, 191, 24, 1, 10, 10, 10, 1, 97, 21, 0, 0, 0, 192, 24, 1, 10, 10, 10, 1, 98, 21, 0, 0, 0,
192, 24, 1, 10, 13, 10, 4, 226, 150, 129, 97, 21, 0, 0, 192, 191, 24, 1, 10, 11, 10, 2, 97, 98, 21, 0, 0, 128, 191, 24, 1 };

test "spm_decode_stale" {
    const allocator = std.testing.allocator;
    var tok = try zml.tokenizer.Tokenizer.fromBytes(allocator, &model);
    defer tok.deinit();
    var dec = try tok.decoder();
    defer dec.deinit();
    var first = try dec.decodeAlloc(allocator, &.{4}); // "a"
    defer first.deinit(allocator);
    var second = try dec.decodeAlloc(allocator, &.{5}); // "b"
    defer second.deinit(allocator);
    try std.testing.expectEqualStrings("b", second.items);
}
```

H06 Tensor data offsets are not validated

HIGH ZML wrong values · Safetensors · risk 12 (impact 4 × exposure 3) · **untrusted input**

API safetensors.parseSafetensors
Location `zml/safetensors.zig:445-458, :105, :142`
Platform CPU

What happens. A truncated checkpoint (f32[4] claiming 1000 bytes, holding 4) registers without error. A start offset near i64 max makes the reader pread at a negative offset, which std reports as a programmer bug (errno INVAL) and panics in Debug. The start offset case has no reproducer.

Root cause. The reader is sized from the shape (`remaining = tensor.byteSize()`, safetensors.zig:105) and starts at `data_start_offset + start`. data_offsets[1] is never read, and neither offset is checked against the file size or against dtype x shape.

Risk assessment. A truncated download fails at load time with end of stream. On the async loader path that error reaches `unreachable` (zml/io.zig:438-440, defaultCallback), so the process panics instead of reporting an error. If end - start is smaller than byteSize, the read silently takes bytes from the next tensor and the model runs on misaligned weights. The input is untrusted.

Reproduction.
1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_st_offsets --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `st_offsets`: *fails* (matches `found/st_offsets\.zig:27:`)
   `/home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/4993/execroot/_main/tools/zmlsmith/regressions/found/st_offsets.zig:27:9: 0x56842949559d in test.st_offsets (too`

Verification: confirmed, report corrected. **Expected behavior, from:** safetensors format: data_offsets are relative to the byte buffer after the header, and end - start must equal the dtype x shape byte size (safetensors format documentation; not checked offline). Zig std: pread with an INVAL errno goes to errnoBug, which panics in Debug (lib/std/Io/Threaded.zig:14054-14055).
- zml/safetensors.zig:105: .remaining = tensor.byteSize(),

- zml/safetensors.zig:141-142: const combined_limit = ...; const n = try self.file_reader.interface.stream(w, combined_limit);
- zml/safetensors.zig:458: .offset = data_start_offset + start,
- zml/safetensors.zig:446: const start: u64 = @intCast(offset_field.array.items[0].integer);
- zml/io.zig:438-440: self.loadSingleInner(...) catch |e| { log.err("Errors are not handled in defaultCallback, got {}", .{e}); unreachable; };
- lib/std/Io/Threaded.zig:9849: const rc = posix.pread(file.handle, dest[0].ptr, @intCast(dest[0].len), @bitCast(offset));
- lib/std/Io/Threaded.zig:9869: .INVAL => |err| return syscall.errnoBug(err),

Suggested fix. Check `end - start == byteSize(shape)` and `end <= file_size - header_size` when parsing.

tools/zmlsmith/regressions/found/st_offsets.zig (30 lines)

```
const std = @import("std");
const zml = @import("zml");

// parseSafetensors never checks data_offsets: the end offset is ignored, and neither
// offset is compared with the file size or with the dtype and shape. A truncated or
// corrupted checkpoint registers fine and the loader later reads garbage or past the end
// (zml/safetensors.zig:445-458 at 74c590b0). Found by reading the code while auditing
// zmlsmith.
test "st_offsets" {
    const io = std.testing.io;
    var tmp = std.testing.tmpDir(.{});
    defer tmp.cleanup();

    // f32[4] is 16 bytes, but the offsets claim 0..1000 and the file holds 4 data bytes.
    const header = "{\"w\":{\"dtype\":\"F32\",\"shape\":[4],\"data_offsets\":[0,1000]}}";
    var bytes: [8 + header.len + 4]u8 = undefined;
    std.mem.writeInt(u64, bytes[0..8], header.len, .little);
    @memcpy(bytes[8..][0..header.len], header);
    @memset(bytes[8 + header.len ..], 0);
    try tmp.dir.writeFile(io, .{ .sub_path = "m.safetensors", .data = &bytes });

    const file = try tmp.dir.openFile(io, "m.safetensors", .{});
    defer file.close(io);
    var registry: zml.safetensors.TensorRegistry = .init(std.testing.allocator);
    defer registry.deinit();
    if (zml.safetensors.parseSafetensors(std.testing.allocator, io, &registry, file)) |_| {
        return error.TestUnexpectedResult;
    } else |_| {}
}
```

H07 scatterSlices ignores the order of update axes

HIGH ZML wrong values · Tensor · risk 12 (impact 4 × exposure 3)

API Tensor.scatterSlices
Location `zml/ops.zig:1865, :1870, :1920 (updates_transpose computed), :1929 (stored), :1795-1812 (updates passed to stablehlo.scatter untransposed)`
Platform CPU
KnownBug `scatter_update_order`

What happens. When the update axes are in a different order than the operand's, updates are applied without being transposed: values land in the wrong place with no error.

Root cause. scatterConfig computes `updates_transpose` (zml/ops.zig:1865, 1870, 1920) and stores it in the config (zml/ops.zig:1929), but nothing reads it. scatter() passes the original update values to stablehlo.scatter (zml/ops.zig:1795, 1812) without transposing them to the operand's axis order. zml/ops.zig:2003 has the TODO "transpose updates if needed".

> **Risk assessment.** Silent wrong results for a tag-based API whose point is that axis order doesn't matter (KV cache updates use scatter).

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_scatter_order --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `scatter_order`: *fails* (matches `p999_absolute_error`)
   `p999_absolute_error: 3`

Verification: confirmed as reported. **Expected behavior, from:** scatterSlices is documented in tag terms: axes are identified by their tags (zml/tensor.zig:3122-3126). The expected value was re-derived by hand from the reproducer: updates is x2 with axes (n,d,h,a); the expected result has z[a=i,h=0,n=j] += x2[n=j,d=0,h=0,a=i], giving -4, -1, -4, -3 at (a0n0, a1n0, a0n1, a1n1), which matches the expected bytes.

- zml/ops.zig:1865: updates_transpose.appendAssumeCapacity(update.axis(t));
- zml/ops.zig:1920: updates_transpose.appendAssumeCapacity(@intCast(i));
- zml/ops.zig:1929: .updates_transpose = updates_transpose, (no reader anywhere in zml/ops.zig)

- zml/ops.zig:1812: &updates_values, (passed to dialects.stablehlo.scatter without transpose)
- zml/ops.zig:2003: // TODO: transpose updates if needed

Suggested fix. Transpose updates to the operand order before lowering.

tools/zmlsmith/regressions/found/scatter_order.zig (62 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 810089.
test "scatter_update_order" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor, x1: zml.Tensor, x2: zml.Tensor) [1]zml.Tensor {
            const x3 = x0.scatterSlices(.{ .a = x1, .n = x1 }, x2, .{ .update_fn = zml.Tensor.ScatterOpts.increment, .indices_are_unique = false });
            _ = &x3;
            return .{x3};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .a = 3, .h = 1, .n = 7 }, .f64));
    const x1: zml.Tensor = .fromShape(zml.Shape.init(.{ .d = 1 }, .u32));
    const x2: zml.Tensor = .fromShape(zml.Shape.init(.{ .n = 2, .d = 1, .h = 1, .a = 2 }, .f64));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x0, x1, x2 }, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .a = 3, .h = 1, .n = 7 }, .f64), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var b1: zml.Buffer = try .fromBytes(io, platform, x1.shape(), .replicated, &[_]u8{
        0, 0, 0, 0,
    });
    defer b1.deinit();
    var b2: zml.Buffer = try .fromBytes(io, platform, x2.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 16, 192, 0, 0, 0, 0, 0, 240, 191, 0, 0, 0, 0, 0, 16, 192,
        0, 0, 0, 0, 0, 0, 8,  192,
    });
    defer b2.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ b0, b1, b2 });
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 16, 192, 0, 0, 0, 0, 0, 0, 16,  192, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0,  0,   0, 0, 0, 0, 0, 0, 0,   0,   0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0,  0,   0, 0, 0, 0, 0, 0, 240, 191, 0, 0, 0, 0, 0, 0, 8, 192,
            0, 0, 0, 0, 0, 0, 0,  0,   0, 0, 0, 0, 0, 0, 0,   0,   0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0,  0,   0, 0, 0, 0, 0, 0, 0,   0,   0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0,  0,   0, 0, 0, 0, 0, 0, 0,   0,   0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0,  0,   0, 0, 0, 0, 0, 0, 0,   0,   0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .a = 3, .h = 1, .n = 7 }, .f64), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .exact_match);
    }
}
```

H08 Chained padded reduce_window gives wrong values

HIGH upstream XLA wrong values · XLA CPU · risk 12 (impact 4 × exposure 3)

API	zml.ops.reduceWindow with padding, then cumulativeSum
Location	`XLA CPU, pass not identified. ZML side is correct: zml/tensor.zig:1780-1795 (cumulativeSum).`
Platform	CPU
KnownBug	`xla_reduce_window_chain`

What happens. The cumulative sum of [0, 0, 1, 0, 0] (the output of a window-1 reduce_window with padding 2 on each side) comes out [1, 1, 1, 1, 1] instead of [0, 0, 1, 1, 1]. The claim that each op alone is correct is not verified beyond the comment in the reproducer.

Root cause. Not established. Both ZML reduce_windows are valid: the first is an identity window (window 1, stride 1, padding (2,2)); cumulativeSum is a reduce_window with window = n and padding (n-1, 0) (zml/tensor.zig:1784-1787, init .scalar(0) at :1792). The wrong result is consistent with an XLA simplifier rewriting the identity window into padding and folding that padding into the second window, but the pass was not identified.

Risk assessment. Silent wrong values for a padded reduce_window chain that XLA rewrites this way; cumulativeSum is one such chain. Other chain shapes are untested.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_xla_rw_cumsum --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `xla_rw_cumsum`: *fails* (matches `got \{ 1, 1, 1, 1, 1 \}`)
   `got { 1, 1, 1, 1, 1 }`

Verification: confirmed, report corrected. **Expected behavior, from:** StableHLO reduce_window padding semantics (pad value = init value; docs/spec.md '### reduce_window' section). cumulativeSum definition in zml/tensor.zig:1780-1795 (expected output [0,0,1,1,1] follows from the prefix sum of the padded window output).

- zml/tensor.zig:1785: 'window_dimensions[a] = self.dim(a);'
- zml/tensor.zig:1787: 'padding[a] = .{ self.dim(a) - 1, 0 };'
- zml/tensor.zig:1792: '.{.scalar(0, self.dtype())},' (init value 0)

Suggested fix. Report upstream with xla_rw_cumsum. The ZML-side workaround (a barrier between chained windows) is not verified and not implemented here.

`tools/zmlsmith/regressions/found/xla_rw_cumsum.zig` (52 lines)

```
const std = @import("std");
const zml = @import("zml");

// Upstream XLA CPU (probably): a padded reduce_window (here an identity window with
// padding 2 on each side, so [0, 0, 1, 0, 0]) followed by Tensor.cumulativeSum, itself a
// padded reduce_window, gives a wrong prefix sum: element 0 is 1 instead of 0 and 2 of 5
// elements differ. Each op alone is correct. In the original seed the wrong elements
// were garbage (3834619964). Found by zmlsmith (seed 110613), reduced from 43 nodes.
test "xla_rw_cumsum" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = zml.ops.reduceWindow(1, .{x0}, .{zml.Tensor.constant(zml.DataType.zero(.u32))}, .{ .window_dimensions = &[_]i64{1}, .window_strides =
&[_]i64{1}, .base_dilations = &[_]i64{1}, .window_dilations = &[_]i64{1}, .padding = &[_][2]i64{.{ 2, 2 }} }, &struct {
                fn f(a: zml.ops.ReduceArgs) [1]zml.Tensor {
                    return .{a.left.add(a.right)};
                }
            }.f)[0];
            _ = &x1;
            const x2 = x1.cumulativeSum(-1);
            _ = &x2;
            return .{x2};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .h = 1 }, .u32));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .h = 5 }, .u32), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        1, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .h = 5 }, .u32), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        std.debug.print("got {any}\n", .{got.constItems(u32)});
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

H09 Nucleus sampling drops the token that crosses top_p

HIGH ZML wrong values · Sampling · risk 12 (impact 3 × exposure 4)

API nn.sampleTokensDynamic
Location `zml/nn.zig:2123`
Platform CPU

What happens. With {0.5, 0.45, 0.05} and `top_p = 0.9` every draw is token 0: moderate top_p values collapse to greedy decoding.

Root cause. Tokens are kept when their inclusive cumulative probability is <= top_p (zml/nn.zig:2123). The crossing token is removed. The first candidate is always kept by the first_token override (zml/nn.zig:2124-2126).

Risk assessment. Changes the nucleus set for any user top_p value where the first two candidates' mass exceeds top_p: the crossing token is dropped. For peaked distributions this reduces the set to the top token; for flatter distributions it drops one token more than HF or vLLM. It does not force greedy decoding in general (a flat distribution keeps several tokens). Generation diversity is affected silently.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_sample_top_p --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `sample_top_p`: *fails* (matches `probs \{ 0\.5, 0\.45, 0\.05 \} top_k 3 top_p 0\.9 min_p 0: counts \{ 4096, 0, 0 \}`)
   `probs { 0.5, 0.45, 0.05 } top_k 3 top_p 0.9 min_p 0: counts { 4096, 0, 0 }`

Verification: confirmed as reported. **Expected behavior, from:** huggingface/transformers src/transformers/generation/logits_process.py, class TopPLogitsWarper.__call__ (fetched from main): sorted_logits sorted ascending, cumulative_probs = softmax(sorted).cumsum(); sorted_indices_to_remove = cumulative_probs <= (1 - self.top_p); first token kept only via min_tokens_to_keep. Equivalent to keeping token i iff its exclusive descending cumulative mass < top_p (keeps the token that crosses top_p). top_p=1 removes nothing. Reference-set check for {0.5,0.45,0.05}, top_p 0.9: exclusive cumsum [0, 0.5, 0.95] -> keep tokens 0 and 1; HF keeps {0,1}. Emulated f32 inclusive cumsum [0.5, 0.94999999, 1.0]; index 1 is 0.95 > 0.9 so ZML drops it, which is exact. The in-repo unit test at zml/nn.zig:2155-2158 also pins the inclusive rule (see notes).
Root cause checked by patching: candidates use the exclusive cumulative sum, `probs_sum.sub(probs).cmp(.LT, top_p)` (zml/nn.zig:2123). `found_sample_top_p` and `found_sample_top_p_one` pass (3 of 3 runs each); unpatched both fail 5 of 5. The in-tree unit test at zml/nn.zig:2155-2158 encodes the old rule and was not run.

- zml/nn.zig:2123: const candidate = probs_sum.cmp(.LE, top_p).logical(.AND, probs.cmp(.GE, min_p));
- zml/nn.zig:2155: .{ .{ .top_k = 4, .top_p = 0.701 }, [_]f32{ @log(4.0), @log(3.0), ___, ___ } },
- zml/nn.zig:2156: .{ .{ .top_k = 4, .top_p = 0.901 }, [_]f32{ @log(4.0), @log(3.0), @log(2.0), ___ } },
- zml/nn.zig:2158: .{ .{ .top_k = 3, .top_p = 0.701 }, [_]f32{ @log(4.0), ___, ___, ___ } },

Suggested fix. Keep a token when `cumsum - p < top_p` (exclusive prefix), always keeping the first.

`tools/zmlsmith/regressions/found/sample_top_p.zig` (55 lines)

```
const std = @import("std");
const zml = @import("zml");

// nn.sampleTokensDynamic keeps tokens whose inclusive cumulative probability is <= top_p
// (zml/nn.zig:2123 at 74c590b0), which drops the token that crosses the threshold.
// Nucleus sampling keeps the smallest set whose probabilities reach top_p: with
// probabilities {0.5, 0.45, 0.05} and top_p = 0.9 it samples tokens 0 and 1, ZML always
// samples token 0 (4096 of 4096 draws), so moderate top_p values collapse to greedy.
// Found by zmlsmith sampling property tests.

const rows = 4096;

const Fns = struct {
    fn sample(logits: zml.Tensor, opts: zml.nn.DynamicSamplingStrategy, rng: zml.Tensor.Rng) zml.Tensor {
        const t, _ = zml.nn.sampleTokensDynamic(logits, opts, rng);
        return t;
    }
};

/// Samples `rows` times from the same probabilities and counts each token.
fn counts(comptime voc: usize, probs: [voc]f32, top_k: u32, top_p: f32, min_p: f32) ![voc]u32 {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const logits: zml.Tensor = .init(.{ .s = rows, .voc = voc }, .f32);
    const opts: zml.nn.DynamicSamplingStrategy = .init(.f32, 0);
    var exe = try platform.compileFn(allocator, io, Fns.sample, .{ logits, opts, zml.Tensor.Rng.init() }, .{});
    defer exe.deinit();
    var data: [rows * voc]f32 = undefined;
    for (0..rows) |r| for (probs, 0..) |p, v| {
        data[r * voc + v] = @log(p);
    };
    var bl = try zml.Buffer.fromBytes(io, platform, logits.shape(), .replicated, std.mem.sliceAsBytes(&data));
    defer bl.deinit();
    var bo = try zml.nn.DynamicSamplingStrategy.makeBuffers(io, platform, .f32, .{ .top_k = top_k, .top_p = top_p, .min_p = min_p });
    defer zml.nn.DynamicSamplingStrategy.deinitBuffers(&bo);
    var br = try zml.Tensor.Rng.initBuffer(io, platform, .replicated, 1234);
    defer zml.Tensor.Rng.deinitBuffer(&br);
    var res = try zml.testing.autoCall(allocator, io, &exe, Fns.sample, .{ bl, bo, br });
    defer res.deinit();
    const s = try res.toSliceAlloc(allocator, io);
    defer s.free(allocator);
    var c: [voc]u32 = @splat(0);
    for (s.constItems(i32)) |t| c[@intCast(t)] += 1;
    std.debug.print("probs {any} top_k {d} top_p {d} min_p {d}: counts {any}\n", .{ probs, top_k, top_p, min_p, c });
    return c;
}

test "top_p keeps the token that reaches it" {
    // 0.5 < 0.9: nucleus sampling also keeps the second token (0.5 + 0.45 >= 0.9).
    const c = try counts(3, .{ 0.5, 0.45, 0.05 }, 3, 0.9, 0);
    try std.testing.expect(c[1] > 0);
```

```
    try std.testing.expectEqual(@as(u32, 0), c[2]);
}
```

H10 SentencePiece encode of an empty text panics

HIGH ZML crash · Tokenizer · risk 12 (impact 3 × exposure 4) · **untrusted input**

API tokenizer sentencepiece Encoder.encode
Location `zml/tokenizer/sentencepiece/sentencepiece.zig:76-81 (Encoder.ids; ffi.ZigSlice.to call at :77-80), reached from Encoder.encode at :70-74`
Platform CPU

What happens. Encoding a text that gives no tokens, such as "", panics with "attempt to use null value".

Root cause. `Encoder.ids()` calls `ffi.ZigSlice.to` (ffi/zig_slice.zig:12-14), which slices a `[*c]` pointer. `std_vector_int_data` returns null when the vector never allocated storage. `Encoder.init` creates a fresh vector (sentencepiece.zig:55-58) and `encode` only reserves `input.len / 3` (:71), which is 0 for short inputs, so encoding a text that yields no tokens on a fresh encoder slices a null pointer and panics. A vector that held elements keeps its buffer after `std_vector_int_clear`, so the failure needs a fresh encoder (or one whose storage was never allocated), not any empty result.

> **Risk assessment.** User input reaches it directly: an empty prompt (or one that normalizes to nothing) crashes a server instead of returning no tokens.

Reproduction.
1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_spm_encode_empty --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `spm_encode_empty`: *fails* (matches `attempt to use null value`)
   ```
   1/1 spm_encode_empty.test.spm_encode_empty...thread 3165560 panic: attempt to use null value
   ```

Verification: confirmed, report corrected. **Expected behavior, from:** Zig pointer semantics: slicing a C pointer that is null panics in safe builds. The expected result for an empty input is an empty slice, the natural contract of encode (returns the ids for the text).
- zml/tokenizer/sentencepiece/sentencepiece.zig:71: c.std_vector_int_reserve(self.vec, input.len / StringToTokenRatio);
- zml/tokenizer/sentencepiece/sentencepiece.zig:76-81: pub fn ids(self: *const Encoder) []const u32 { return ffi.ZigSlice.to(u32, .{ .ptr = c.std_vector_int_data(self.vec), .len = c.std_vector_int_size(self.vec), }); }
- ffi/zig_slice.zig:12-14: pub fn to(comptime T: type, slice: c.zig_slice) []T { return @as([*c]T, @ptrCast(@alignCast(slice.ptr)))[0..slice.len]; }
- zml/tokenizer/tokenizer.zig:59: inline else => |*encoder_| try allocator.dupe(u32, try encoder_.encode(text)), (Tokenizer.Encoder.encodeAlloc path)

Suggested fix. Return an empty slice when the vector size is 0.

`tools/zmlsmith/regressions/found/spm_encode_empty.zig` (19 lines)

```
const std = @import("std");
const zml = @import("zml");

// sentencepiece Encoder.ids makes a slice from std_vector_int_data, which is null for an
// empty vector (zml/tokenizer/sentencepiece/sentencepiece.zig:75-80 at 74c590b0): encoding
// a text that gives no tokens, such as "", panics with "attempt to use null value" in
// ffi.ZigSlice.to. Found by zmlsmith parse-fuzz (spm target).
/// A minimal sentencepiece ModelProto: <unk>, <s>, </s>, then _ a b _a ab.
const model = [_]u8{ 10, 14, 10, 5, 60, 117, 110, 107, 62, 21, 0, 0, 0, 0, 24, 2, 10, 12, 10, 3, 60, 115, 62, 21, 0, 0, 0, 0, 24, 3, 10, 13, 10, 4, 60, 47, 115,
62, 21, 0, 0, 0, 0, 24, 3, 10, 12, 10, 3, 226, 150, 129, 21, 0, 0, 128, 191, 24, 1, 10, 10, 10, 1, 97, 21, 0, 0, 0, 192, 24, 1, 10, 10, 10, 1, 98, 21, 0, 0, 0,
192, 24, 1, 10, 13, 10, 4, 226, 150, 129, 97, 21, 0, 0, 192, 191, 24, 1, 10, 11, 10, 2, 97, 98, 21, 0, 0, 128, 191, 24, 1 };

test "spm_encode_empty" {
    var tok = try zml.tokenizer.Tokenizer.fromBytes(std.testing.allocator, &model);
    defer tok.deinit();
    var enc = try tok.encoder();
    defer enc.deinit();
    const ids = try enc.encodeAlloc(std.testing.allocator, "");
    std.testing.allocator.free(ids);
}
```

H11 Paged attention doesn't compile with fewer queries than page_size

HIGH ZML compile · Attention · risk 12 (impact 3 × exposure 4)

API attention.paged_attention.pagedAttention (.stablehlo)
Location `zml/attention/paged_attention.zig:694 (both branches of zml.ops.if are built), :700 (page-size q slice)`
Platform CPU (stablehlo backend)

What happens. With a few decode tokens the program doesn't compile: "slice size 4 greater than dimension size 2".

Root cause. The prefill branch is always built and slices a page of queries.

> **Risk assessment.** Blocks decoding small batches with the CPU paged backend entirely.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_paged_small --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `paged_small`: *fails* (matches `Failed to create MLIR operation`)
   `thread 3155647 panic: Failed to create MLIR operation`

Verification: confirmed as reported. **Expected behavior, from:** Structural: zml.ops.if builds both branches (onTrue prefill slice of page_size rows, onFalse decode). Shape requirement is MLIR slice verification: slice size cannot exceed the dimension (q has 2 rows, slice 4).

- zml/attention/paged_attention.zig:694: .cmp(seq_num_queries_, .GT, .scalar(1, .i32)),
- zml/attention/paged_attention.zig:700: const active_q = self.q.slice(self.q.axis(.b), .dyn(state.slot_id, q_chunk));
- tools/zmlsmith/regressions/found/paged_small.zig: try check(2, 2, ...) with page size 4 (const ps = 4)

Suggested fix. Build the prefill branch only when the query count allows it, or pad q.

`tools/zmlsmith/regressions/found/paged_small.zig` (140 lines, first 70 shown)

```
const std = @import("std");
const zml = @import("zml");
const paged = zml.attention.paged_attention;

// The stablehlo (CPU) backend of attention.paged_attention.pagedAttention, against a host
// reference of causal paged attention: query j of a sequence with n queries and seq_len L
// sits at position L - n + j and attends keys 0..L - n + j, read from the pages listed in
// block_table. Found by auditing zmlsmith coverage of zml/attention. Same harness as
// paged_attn.zig, split out because this case panics at compile time.

const ps = 4; // page_size
const num_pages = 8;
const max_pages = 4;
const hd = 4;

const Case = struct {
    q_start: []const i32, // query_start_len, batch + 1 entries
    seq_lens: []const i32,
    block_table: []const [max_pages]i32,
};

fn reference(c: Case, q: []const f32, kc: []const f32, vc: []const f32, out: []f32) void {
    @memset(out, 0);
    const scale = 1 / @sqrt(@as(f32, hd));
    for (c.seq_lens, 0..) |len, s| {
        const qs: usize = @intCast(c.q_start[s]);
        const qe: usize = @intCast(c.q_start[s + 1]);
        const n = qe - qs;
        for (0..n) |j| {
            const pos: usize = @as(usize, @intCast(len)) - n + j;
            var w: [max_pages * ps]f32 = undefined;
            var m: f32 = -std.math.inf(f32);
            for (0..pos + 1) |p| {
                const page: usize = @intCast(c.block_table[s][p / ps]);
                var d: f32 = 0;
                for (0..hd) |x| d += q[(qs + j) * hd + x] * kc[(page * ps + p % ps) * hd + x];
                w[p] = d * scale;
                m = @max(m, w[p]);
            }
            var sum: f32 = 0;
            for (0..pos + 1) |p| {
                w[p] = @exp(w[p] - m);
                sum += w[p];
            }
            for (0..hd) |x| {
                var acc: f32 = 0;
                for (0..pos + 1) |p| {
                    const page: usize = @intCast(c.block_table[s][p / ps]);
                    acc += w[p] / sum * vc[(page * ps + p % ps) * hd + x];
                }
                out[(qs + j) * hd + x] = acc;
            }
        }
    }
}

fn check(comptime batch: u32, comptime tokens: u32, c: Case) !void {
    const platform = zml.testing.env();
    const io = std.testing.io;
    const allocator = std.testing.allocator;

    const parameters: paged.Parameters = .init(.fromBackend(.{
        .backend = .stablehlo,
        .is_prefill = true,
        .batch_size = batch,
        .seq_len = max_pages * ps,
```

```
        .max_num_pages = max_pages,
        .max_token_count = tokens,
        .num_heads = 1,
        .num_kv_heads = 1,
```

H12 Malformed safetensors headers panic instead of failing (length, schema, dims)

HIGH ZML crash · Safetensors · risk 9 (impact 3 × exposure 3) · **untrusted input**

API safetensors.parseSafetensors
Location `zml/safetensors.zig:409 (length add), :410 (allocation), :438 (shape .get(.?), :445-447 (data_offsets, dtype), :451 (dims); zml/shape.zig:495-496 (Shape.count)`
Platform CPU

What happens. (1) An 8-byte file of 0xff panics with "integer overflow" (header length + 8). (2) A tensor entry without "shape" panics on an unchecked access. (3) A negative dim, or dims whose product overflows i64, register, then `TensorRegistry.totalBytes` / `byteSize` panic in `Shape.count`.

Root cause. The u64 header length is added and used as an allocation size unchecked; the JSON is accessed with `.object`, `.get(..).?`, `.integer` and `@intCast`; shape integers are copied as is.

> **Risk assessment.** Denial of service from an untrusted file: a bad checkpoint takes the whole process down instead of returning an error the caller can report. parse-fuzz found 13 distinct crash signatures at 9 lines in 3,000 inputs.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_st_header_len --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_st_header --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_st_dims --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `st_header_len`: *fails* (matches `panic: integer overflow`)
   `1/1 st_header_len.test.st_header_len...thread 3163615 panic: integer overflow`
   `st_header`: *fails* (matches `attempt to use null value`)
   `1/1 st_header.test.st_header...thread 3165685 panic: attempt to use null value`
   `st_dims`: *fails* (matches `integer overflow|Can't count elements in shape with negative dimension`)
   `1/1 st_dims.test.st_dims...thread 3163953 panic: integer overflow`

Verification: confirmed, report corrected. **Expected behavior, from:** A malformed header must produce an error (for example error.InvalidSafetensors) rather than a panic. The same file already returns errors for other malformed input (for example error.TooManyDimensions at safetensors.zig:442).

- zml/safetensors.zig:408-410: const json_header_length: u64 = try reader.interface.takeInt(u64, .little); const data_start_offset = BYTES_HEADER + json_header_length; const json_data = try arena.allocator().alloc(u8, @intCast(json_header_length));
- zml/safetensors.zig:438: const shape_field = value.object.get("shape").?.array;
- zml/safetensors.zig:451: dims.appendAssumeCapacity(d.integer);
- zml/shape.zig:495-496: stdx.debug.assert(d >= 0, "Can't count elements in shape with negative dimension: {f}", .{self}); res *= d;

Suggested fix. Validate the header length against the file size, check the JSON schema and return `error.InvalidSafetensors`, reject negative dims and overflowing products.

`tools/zmlsmith/regressions/found/st_header_len.zig` (23 lines)

```
const std = @import("std");
const zml = @import("zml");

// parseSafetensors adds the untrusted u64 header length to BYTES_HEADER without an
// overflow check and allocates that many bytes, so an 8 byte file of 0xff panics with
// "integer overflow" (and a large but smaller length tries a huge allocation) instead of
// returning an error (zml/safetensors.zig:408-410 at 74c590b0). Found by zmlsmith
// parse-fuzz, reduced from 247 to 8 bytes.
test "st_header_len" {
    const io = std.testing.io;
    var tmp = std.testing.tmpDir(.{});
    defer tmp.cleanup();
    try tmp.dir.writeFile(io, .{ .sub_path = "m.safetensors", .data = &([_]u8{0xff} ** 8) });

    const file = try tmp.dir.openFile(io, "m.safetensors", .{});
    defer file.close(io);
    var registry: zml.safetensors.TensorRegistry = .init(std.testing.allocator);
    defer registry.deinit();
    if (zml.safetensors.parseSafetensors(std.testing.allocator, io, &registry, file)) |_| {
        return error.TestUnexpectedResult;
    } else |_| {}
}
```

tools/zmlsmith/regressions/found/st_header.zig (28 lines)

```
const std = @import("std");
const zml = @import("zml");

// parseSafetensors accesses the JSON header without checking its schema
// (`headers.object`, `.get("shape").?`, `.integer`, `@intCast`), so a malformed header
// panics instead of returning an error (zml/safetensors.zig:421-451 at 74c590b0). Here
// the tensor entry has no "shape". Found by reading the code while auditing zmlsmith.
test "st_header" {
    const io = std.testing.io;
    var tmp = std.testing.tmpDir(.{});
    defer tmp.cleanup();

    const header = "{\"w\":{\"dtype\":\"F32\",\"data_offsets\":[0,4]}}";
    var bytes: [8 + header.len + 4]u8 = undefined;
    std.mem.writeInt(u64, bytes[0..8], header.len, .little);
    @memcpy(bytes[8..][0..header.len], header);
    @memset(bytes[8 + header.len ..], 0);
    try tmp.dir.writeFile(io, .{ .sub_path = "m.safetensors", .data = &bytes });

    const file = try tmp.dir.openFile(io, "m.safetensors", .{});
    defer file.close(io);
    var registry: zml.safetensors.TensorRegistry = .init(std.testing.allocator);
    defer registry.deinit();
    if (zml.safetensors.parseSafetensors(std.testing.allocator, io, &registry, file)) |_| {
        return error.TestUnexpectedResult;
    } else |_| {}
}
```

tools/zmlsmith/regressions/found/st_dims.zig (34 lines)

```
const std = @import("std");
const zml = @import("zml");

// parseSafetensors copies the "shape" integers into the tensor Shape unchecked
// (zml/safetensors.zig:449-451 at 74c590b0): a negative dim, or dims whose product
// overflows i64, register fine, then TensorRegistry.totalBytes / Tensor.byteSize panic in
// Shape.count ("Can't count elements in shape with negative dimension", or "integer
// overflow" at zml/shape.zig:496). Found by zmlsmith parse-fuzz.
fn load(io: std.Io, dir: std.Io.Dir, header: []const u8) !void {
    var bytes: std.ArrayList(u8) = .empty;
    defer bytes.deinit(std.testing.allocator);
    var len: [8]u8 = undefined;
    std.mem.writeInt(u64, &len, header.len, .little);
    try bytes.appendSlice(std.testing.allocator, &len);
    try bytes.appendSlice(std.testing.allocator, header);
    try bytes.appendNTimes(std.testing.allocator, 0, 16);
    try dir.writeFile(io, .{ .sub_path = "m.safetensors", .data = bytes.items });

    const file = try dir.openFile(io, "m.safetensors", .{});
    defer file.close(io);
    var registry: zml.safetensors.TensorRegistry = .init(std.testing.allocator);
    defer registry.deinit();
    zml.safetensors.parseSafetensors(std.testing.allocator, io, &registry, file) catch return;
    _ = registry.totalBytes();
}

test "st_dims" {
    const io = std.testing.io;
    var tmp = std.testing.tmpDir(.{});
    defer tmp.cleanup();
    try load(io, tmp.dir, "{\"w\":{\"dtype\":\"F32\",\"shape\":[9223372036854775807,2],\"data_offsets\":[0,16]}}");
    try load(io, tmp.dir, "{\"w\":{\"dtype\":\"BF16\",\"shape\":[-3],\"data_offsets\":[0,6]}}");
}
```

H13 HTTP redirects are followed with no limit

HIGH ZML hang · VFS · risk 9 (impact 3 × exposure 3) · **untrusted input**

API vfs HTTP open (fetchSize)
Location `vfs/http.zig:324-348`
Platform CPU

What happens. A server that redirects a file to itself (here an empty Location) makes opening it loop without bound: the reproducer's server answers 200 redirects, then fails, and the open follows every one. There is no redirect limit and no timeout.

Root cause. Redirects are followed in a hand-written loop (`redirect_behavior = .not_allowed`, then `continue`) without a counter.

> **Risk assessment.** A misconfigured or hostile endpoint hangs model loading with no timeout or error.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_vfs_redirect_loop --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

`vfs_redirect_loop`: *fails* (matches `redirects followed: \d{2,}`)

```
redirects followed: 200
```

Verification: confirmed, report corrected. **Expected behavior, from:** A finite redirect cap is the requirement; without one the open never returns while the server keeps redirecting. The threshold of 20 in the reproducer is the fuzzer's choice, not a documented ZML or std value. Any cap below 200 passes it.

- vfs/http.zig:324-326: while (true) { var req = try self.client.request(.HEAD, uri, .{ .redirect_behavior = .not_allowed,
- vfs/http.zig:346-347: uri = uri.resolveInPlace(location.len, &aux_buffer) catch unreachable; continue;

Suggested fix. Cap redirects (e.g. 10) and return `error.TooManyRedirects`.

`tools/zmlsmith/regressions/found/vfs_redirect_loop.zig` (60 lines)

```
const std = @import("std");
const zml = @import("zml");

// vfs HTTP fetchSize follows redirects in its own loop with no limit (`redirect_behavior
// = .not_allowed`, then `continue`, vfs/http.zig:324-348 at 74c590b0): a server that
// redirects a file to itself, here with an empty Location, makes opening it hang
// forever. The server below redirects 200 times, then fails: the open follows every
// redirect. Found by zmlsmith parse-fuzz (vfs target).
var heads: std.atomic.Value(u32) = .init(0);
var done: std.atomic.Value(bool) = .init(false);

fn serve(server: *std.Io.net.Server) void {
    const io = std.testing.io;
    while (true) {
        const conn = server.accept(io) catch return;
        defer conn.close(io);
        if (done.load(.acquire)) return;
        var rbuf: [4096]u8 = undefined;
        var r = conn.reader(io, &rbuf);
        while (true) {
            const line = r.interface.takeDelimiterExclusive('\n') catch break;
            r.interface.toss(1);
            if (line.len <= 1) break;
        }
        const n = heads.fetchAdd(1, .monotonic);
        var wbuf: [256]u8 = undefined;
        var w = conn.writer(io, &wbuf);
        w.interface.writeAll(if (n < 200) "HTTP/1.1 302 Found\r\nConnection: close\r\nLocation: \r\nContent-Length: 0\r\n\r\n" else "HTTP/1.1 500
Stop\r\nConnection: close\r\nContent-Length: 0\r\n\r\n") catch {};
        w.interface.flush() catch {};
        if (n >= 200) return;
    }
}

test "vfs_redirect_loop" {
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const addr = try std.Io.net.IpAddress.parseIp4("127.0.0.1", 0);
    var server = try addr.listen(io, .{ .reuse_address = true });
    defer server.deinit(io);
    const t = try std.Thread.spawn(.{}, serve, .{&server});
    defer {
        done.store(true, .release);
        if (server.socket.address.connect(io, .{ .mode = .stream })) |c| c.close(io) else |_| {}
        t.join();
    }

    var client: std.http.Client = .{ .allocator = allocator, .io = io };
    defer client.deinit();
    var http: zml.io.VFS.HTTP = try .init(allocator, io, &client, .http);
    defer http.deinit();
    var vfs: zml.io.VFS = try .init(allocator, io);
    defer vfs.deinit();
    try vfs.register("http", http.io());
    var url_buf: [64]u8 = undefined;
    const url = try std.fmt.bufPrint(&url_buf, "http://127.0.0.1:{d}/f.bin", .{server.socket.address.getPort()});
    if (std.Io.Dir.openFile(.cwd(), vfs.io(), url, .{ .mode = .read_only })) |file| file.close(vfs.io()) else |_| {}
    std.debug.print("redirects followed: {d}\n", .{heads.load(.monotonic) - 1});
    try std.testing.expect(heads.load(.monotonic) < 20);
}
```

H14 Answers without Content-Length panic (HTTP, S3, GCS and HF backends)

HIGH ZML crash · VFS · risk 9 (impact 3 × exposure 3) · **untrusted input**

API	vfs HTTP / S3 / GCS open (fetchSize), HF tree listing (fetchTreeFromAPI)
Location	`vfs/http.zig:342; vfs/s3.zig:949; vfs/gcs.zig:1053; vfs/hf.zig:491`
Platform	CPU

What happens. A server answering HEAD with 200 and no Content-Length (chunked, or unknown size) panics with "attempt to use null value" when the file is opened. The HF backend does the same with a chunked tree listing, on the first open of any file of the repository.

Root cause. Each backend reads `res.head.content_length.?`.

Risk assessment. Chunked answers are legal HTTP and common behind proxies and object-store gateways: the process crashes instead of reporting an unsupported server.

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed). These reproducers start found/hf_fixture.zig, a local stand-in for huggingface.co (a python3 https proxy with a test certificate trusted only by the test's client), so python3 must be on the host.
2. Run bazel test //tools/zmlsmith/regressions:found_vfs_no_length --test_output=errors, then bazel test //tools/zmlsmith/regressions:found_s3_no_length --test_output=errors, then bazel test //tools/zmlsmith/regressions:found_gcs_no_length --test_output=errors, then bazel test //tools/zmlsmith/regressions:found_hf_chunked --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   vfs_no_length: *fails* (matches attempt to use null value)
   ```
   1/1 vfs_no_length.test.vfs_no_length...thread 3168949 panic: attempt to use null value
   ```
   s3_no_length: *fails* (matches attempt to use null value)
   ```
   1/1 s3_no_length.test.s3_no_length...thread 3161173 panic: attempt to use null value
   ```
   gcs_no_length: *fails* (matches attempt to use null value)
   ```
   1/1 gcs_no_length.test.gcs_no_length...thread 3151368 panic: attempt to use null value
   ```
   hf_chunked: *fails* (matches attempt to use null value)
   ```
   1/1 hf_chunked.test.hf_chunked...thread 3156714 panic: attempt to use null value
   ```

Verification: confirmed as reported. **Expected behavior, from:** RFC 9112 section 6.3: a response without Content-Length and without chunked framing is delimited by connection close, and a HEAD response has no body. A missing length is legal, so the expected result is an error or a ranged fallback, not a panic.

- vfs/http.zig:342: .success => return res.head.content_length.?,
- vfs/s3.zig:949: .success => res.head.content_length.?,
- vfs/gcs.zig:1053: .success => res.head.content_length.?,
- vfs/hf.zig:491: const body = try res.reader(&.{}).readAlloc(self.allocator, res.head.content_length.?);
- vfs/hf.zig:742-743: HF openFile calls getOrFetchTree (tree listing) before any size lookup

Suggested fix. Return an error (or fall back to a ranged GET) when the length is missing.

tools/zmlsmith/regressions/found/vfs_no_length.zig (67 lines)

```
const std = @import("std");
const zml = @import("zml");

// vfs HTTP fetchSize returns `res.head.content_length.?`: a server answering HEAD with
// 200 and no Content-Length (chunked or unknown size) panics with "attempt to use null
// value" when the file is opened (vfs/http.zig:342 at 74c590b0). Found while adding vfs
// coverage to zmlsmith.

/// Serves `head` to HEAD requests and `get` to the others, `n` connections.
const Server = struct {
    server: std.Io.net.Server,
    head: []const u8,
    get: []const u8,
    n: usize,

    fn run(s: *Server) void {
        const io = std.testing.io;
        for (0..s.n) |_| {
            const conn = s.server.accept(io) catch return;
            defer conn.close(io);
            var rbuf: [4096]u8 = undefined;
            var r = conn.reader(io, &rbuf);
            const first = r.interface.takeDelimiterExclusive('\n') catch return;
            const is_head = std.mem.startsWith(u8, first, "HEAD");
            r.interface.toss(1);
            while (true) {
                const line = r.interface.takeDelimiterExclusive('\n') catch return;
                r.interface.toss(1);
                if (line.len <= 1) break;
            }
            var wbuf: [4096]u8 = undefined;
            var w = conn.writer(io, &wbuf);
            w.interface.writeAll(if (is_head) s.head else s.get) catch return;
            w.interface.flush() catch return;
        }
    }
};

fn readThroughVfs(head: []const u8, get: []const u8, n: usize) !void {
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const addr = try std.Io.net.IpAddress.parseIp4("127.0.0.1", 0);
    var s: Server = .{ .server = try addr.listen(io, .{ .reuse_address = true }), .head = head, .get = get, .n = n };
    defer s.server.deinit(io);
    const t = try std.Thread.spawn(.{}, Server.run, .{&s});
    defer t.join();

    var client: std.http.Client = .{ .allocator = allocator, .io = io };
    defer client.deinit();
```

```
    var http: zml.io.VFS.HTTP = try .init(allocator, io, &client, .http);
    defer http.deinit();
    var vfs: zml.io.VFS = try .init(allocator, io);
    defer vfs.deinit();
    try vfs.register("http", http.io());
    const vio = vfs.io();
    var url_buf: [64]u8 = undefined;
    const url = try std.fmt.bufPrint(&url_buf, "http://127.0.0.1:{d}/f.bin", .{s.server.socket.address.getPort()});
    var file = std.Io.Dir.openFile(.cwd(), vio, url, .{ .mode = .read_only }) catch return;
    defer file.close(vio);
    var buf: [16]u8 = undefined;
    _ = file.readPositional(vio, &.{&buf}, 0) catch return;
}

test "vfs_no_length" {
    try readThroughVfs("HTTP/1.1 200 OK\r\nConnection: close\r\nTransfer-Encoding: chunked\r\n\r\n", "HTTP/1.1 200 OK\r\nConnection: close\r\nContent-Length:
0\r\n\r\n", 1);
}
```

tools/zmlsmith/regressions/found/s3_no_length.zig (67 lines)

```
const std = @import("std");
const zml = @import("zml");

// vfs S3 fetchSize returns `res.head.content_length.?` (vfs/s3.zig:949 at 74c590b0), like
// the HTTP backend: an endpoint answering HEAD with 200 and no Content-Length panics with
// "attempt to use null value" when the object is opened. Found while adding vfs coverage
// to zmlsmith, against a local endpoint (AWS_ENDPOINT_URL style).

/// Serves `head` to HEAD requests and `get` to the others, `n` connections.
const Server = struct {
    server: std.Io.net.Server,
    head: []const u8,
    get: []const u8,
    n: usize,

    fn run(s: *Server) void {
        const io = std.testing.io;
        for (0..s.n) |_| {
            const conn = s.server.accept(io) catch return;
            defer conn.close(io);
            var rbuf: [4096]u8 = undefined;
            var r = conn.reader(io, &rbuf);
            const first = r.interface.takeDelimiterExclusive('\n') catch return;
            const is_head = std.mem.startsWith(u8, first, "HEAD");
            r.interface.toss(1);
            while (true) {
                const line = r.interface.takeDelimiterExclusive('\n') catch return;
                r.interface.toss(1);
                if (line.len <= 1) break;
            }
            var wbuf: [4096]u8 = undefined;
            var w = conn.writer(io, &wbuf);
            w.interface.writeAll(if (is_head) s.head else s.get) catch return;
            w.interface.flush() catch return;
        }
    }
};

fn readThroughS3(head: []const u8, get: []const u8, n: usize) !void {
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const addr = try std.Io.net.IpAddress.parseIp4("127.0.0.1", 0);
    var s: Server = .{ .server = try addr.listen(io, .{ .reuse_address = true }), .head = head, .get = get, .n = n };
    defer s.server.deinit(io);
    const t = try std.Thread.spawn(.{}, Server.run, .{&s});
    defer t.join();

    var client: std.http.Client = .{ .allocator = allocator, .io = io };
    defer client.deinit();
    var endpoint_buf: [64]u8 = undefined;
    const endpoint = try std.fmt.bufPrint(&endpoint_buf, "http://127.0.0.1:{d}", .{s.server.socket.address.getPort()});
    var s3: zml.io.VFS.S3 = try .init(allocator, io, &client, .{ .access_key = "a", .secret_key = "b", .endpoint_url = endpoint, .region = "us-east-1" }, .
{ .read_pool = .{ .chunk_size = 16, .num_workers = 1, .queue_capacity = 4, .max_retries = 0, .retry_initial_delay = .fromMilliseconds(1), .retry_max_delay
= .fromMilliseconds(1) } });
    defer s3.deinit();
    var vfs: zml.io.VFS = try .init(allocator, io);
    defer vfs.deinit();
    try vfs.register("s3", s3.io());
    const vio = vfs.io();
    var file = std.Io.Dir.openFile(.cwd(), vio, "s3://bucket/f.bin", .{ .mode = .read_only }) catch return;
    defer file.close(vio);
    var buf: [16]u8 = undefined;
    _ = file.readPositional(vio, &.{&buf}, 0) catch return;
}

test "s3_no_length" {
    try readThroughS3("HTTP/1.1 200 OK\r\nConnection: close\r\nTransfer-Encoding: chunked\r\n\r\n", "HTTP/1.1 200 OK\r\nConnection: close\r\nContent-Length:
0\r\n\r\n", 1);
}
```

tools/zmlsmith/regressions/found/gcs_no_length.zig (68 lines)

```
const std = @import("std");
const zml = @import("zml");

// vfs GCS fetchSize returns `res.head.content_length.?` (vfs/gcs.zig:1053 at 74c590b0),
// like the HTTP and S3 backends: an endpoint answering HEAD with 200 and no
// Content-Length panics with "attempt to use null value" when the object is opened.
// Found while adding vfs coverage to zmlsmith, against a local endpoint
// (InitOpts.endpoint_url).

/// Serves `head` to HEAD requests and `get` to the others, `n` connections.
```

```
const Server = struct {
    server: std.Io.net.Server,
    head: []const u8,
    get: []const u8,
    n: usize,

    fn run(s: *Server) void {
        const io = std.testing.io;
        for (0..s.n) |_| {
            const conn = s.server.accept(io) catch return;
            defer conn.close(io);
            var rbuf: [4096]u8 = undefined;
            var r = conn.reader(io, &rbuf);
            const first = r.interface.takeDelimiterExclusive('\n') catch return;
            const is_head = std.mem.startsWith(u8, first, "HEAD");
            r.interface.toss(1);
            while (true) {
                const line = r.interface.takeDelimiterExclusive('\n') catch return;
                r.interface.toss(1);
                if (line.len <= 1) break;
            }
            var wbuf: [4096]u8 = undefined;
            var w = conn.writer(io, &wbuf);
            w.interface.writeAll(if (is_head) s.head else s.get) catch return;
            w.interface.flush() catch return;
        }
    }
};

fn readThroughGcs(head: []const u8, get: []const u8, n: usize) !void {
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const addr = try std.Io.net.IpAddress.parseIp4("127.0.0.1", 0);
    var s: Server = .{ .server = try addr.listen(io, .{ .reuse_address = true }), .head = head, .get = get, .n = n };
    defer s.server.deinit(io);
    const t = try std.Thread.spawn(.{}, Server.run, .{&s});
    defer t.join();

    var client: std.http.Client = .{ .allocator = allocator, .io = io };
    defer client.deinit();
    var endpoint_buf: [64]u8 = undefined;
    const endpoint = try std.fmt.bufPrint(&endpoint_buf, "http://127.0.0.1:{d}", .{s.server.socket.address.getPort()});
    var gcs: zml.io.VFS.GCS = try .init(allocator, io, &client, .{ .endpoint_url = endpoint, .read_pool = .{ .chunk_size = 16, .num_workers = 1, .queue_capacity
= 4, .max_retries = 0, .retry_initial_delay = .fromMilliseconds(1), .retry_max_delay = .fromMilliseconds(1) } });
    defer gcs.deinit();
    var vfs: zml.io.VFS = try .init(allocator, io);
    defer vfs.deinit();
    try vfs.register("gs", gcs.io());
    const vio = vfs.io();
    var file = std.Io.Dir.openFile(.cwd(), vio, "gs://bucket/f.bin", .{ .mode = .read_only }) catch return;
    defer file.close(vio);
    var buf: [16]u8 = undefined;
    _ = file.readPositional(vio, &.{&buf}, 0) catch return;
}

test "gcs_no_length" {
    try readThroughGcs("HTTP/1.1 200 OK\r\nConnection: close\r\nTransfer-Encoding: chunked\r\n\r\n", "HTTP/1.1 200 OK\r\nConnection: close\r\nContent-Length:
0\r\n\r\n", 1);
}
```

tools/zmlsmith/regressions/found/hf_chunked.zig (19 lines)

```
const std = @import("std");
const fx = @import("hf_fixture.zig");

// vfs HF fetchTreeFromAPI reads the tree listing with `res.head.content_length.?`
// (vfs/hf.zig:491 at 74c590b0): an API answer without Content-Length (chunked, as a
// proxy or a gzip-less streaming answer sends it) panics with "attempt to use null value"
// on the first open instead of failing it. Found by zmlsmith's HF stand-in.
test "hf_chunked" {
    const routes =
        \\[["/api/models/", "raw", "HTTP/1.1 200 OK\r\nTransfer-Encoding: chunked\r\nConnection: close\r\n\r\n2\r\n[]\r\n0\r\n\r\n"]]
    ;
    var f: fx.Fixture = undefined;
    try f.init(routes);
    defer f.deinit();
    const vio = f.vfs.io();
    var file = std.Io.Dir.openFile(.cwd(), vio, "hf://org/model/config.json", .{ .mode = .read_only }) catch return;
    file.close(vio);
}
```

H15 HF repository listings ignore pagination

HIGH ZML missing files · VFS · risk 9 (impact 3 × exposure 3)

API vfs HF (fetchTreeFromAPI)
Location `vfs/hf.zig:160, :461-508`
Platform CPU

What happens. With a tree listing split in two pages, a file on the second page fails to open with FileNotFound, and directory iteration silently stops at the first page.

Root cause. One tree request (`?recursive=true&limit=1000`) is made and the `Link: <...>; rel="next"` header, with which the Hub paginates long listings (huggingface_hub follows it in list_repo_tree), is ignored.

Risk assessment. Large repositories (many shards, datasets, repos with many revisions of files) are only partly visible: loads fail on files past the first page, and code that discovers files by listing a directory gets an incomplete set with no error. The stand-in reproduces the Hub's pagination header; the exact page size the Hub applies to recursive listings was not measured.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed). These reproducers start `found/hf_fixture.zig`, a local stand-in for huggingface.co (a python3 https proxy with a test certificate trusted only by the test's client), so python3 must be on the host.
2. Run `bazel test //tools/zmlsmith/regressions:found_hf_paginate --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `hf_paginate`: *fails* (matches `FAIL \(FileNotFound\)`)
   ```
   1/1 hf_paginate.test.hf_paginate...FAIL (FileNotFound)
   ```

Verification: confirmed as reported. **Expected behavior, from:** Hub listing endpoints paginate with a Link header (rel="next"), which huggingface_hub's list_repo_tree follows. Not verified offline: huggingface_hub is not installed and no copy exists in the sandbox. The page size the Hub uses for recursive tree listings was not measured, as the report says.

- vfs/hf.zig:160: const TREE_URL_TEMPLATE = "https://huggingface.co/api/models/{[repo]s}/{[model]s}/tree/{[rev]s}/{[path]s}?expand=false&recursive=true&limit=1000";
- vfs/hf.zig:474-484: var req = try self.client.request(.GET, uri, ...); ... var res = try req.receiveHead(&redirect_buffer);
- vfs/hf.zig:491-505: readAlloc of one body, then one parse of []API.Tree and insertTreeNode for each item; no Link header read

Suggested fix. Follow `Link` rel="next" until it is absent, appending each page to the tree.

`tools/zmlsmith/regressions/found/hf_paginate.zig` (28 lines)

```
const std = @import("std");
const fx = @import("hf_fixture.zig");

// vfs HF lists a repository with one tree API request (`?recursive=true&limit=1000`,
// vfs/hf.zig:160, :461-508 at 74c590b0) and ignores the `Link: <...>; rel="next"` header
// with which the Hub paginates long listings (huggingface_hub follows it in
// list_repo_tree): files past the first page don't exist for ZML, so opening them fails
// with FileNotFound. Found by zmlsmith's HF stand-in.
test "hf_paginate" {
    const allocator = std.testing.allocator;
    const page2 = try fx.treeJson(allocator, &.{.{ "model-00002.safetensors", 4 }});
    defer allocator.free(page2);
    const page1 = try fx.treeJson(allocator, &.{.{ "model-00001.safetensors", 4 }});
    defer allocator.free(page1);
    const r2 = try std.fmt.allocPrint(allocator, "HTTP/1.1 200 OK\r\nContent-Length: {d}\r\nConnection: close\r\n\r\n{s}", .{ page2.len, page2 });
    defer allocator.free(r2);
    const r1 = try std.fmt.allocPrint(allocator, "HTTP/1.1 200 OK\r\nContent-Length: {d}\r\nLink: <https://huggingface.co/api/models/org/model/tree/main?
recursive=true&cursor=p2>; rel=\"next\"\r\nConnection: close\r\n\r\n{s}", .{ page1.len, page1 });
    defer allocator.free(r1);
    const routes = try std.json.Stringify.valueAlloc(allocator, .{ .{ "cursor=p2", "raw", r2 }, .{ "/api/models/", "raw", r1 }, .{ "/resolve/", "file",
"abcd" } }, .{});
    defer allocator.free(routes);
    var f: fx.Fixture = undefined;
    try f.init(routes);
    defer f.deinit();
    const vio = f.vfs.io();
    var file = try std.Io.Dir.openFile(.cwd(), vio, "hf://org/model/model-00002.safetensors", .{ .mode = .read_only });
    file.close(vio);
}
```

H16 Malformed Content-Range panics (HTTP and parallel_read)

HIGH ZML crash · VFS · risk 9 (impact 3 × exposure 3) · **untrusted input**

API vfs HTTP / parallel_read parseContentRange
Location `vfs/http.zig:416-425; vfs/parallel_read.zig:161`
Platform CPU

What happens. `Content-Range: 0-15 bytes/16` panics with "start index 5 is larger than end index 1" instead of being rejected.

Root cause. The header is sliced between the first ' ', '-' and '/' without checking their order. parallel_read, used by the HF, S3 and GCS backends, has the same code.

Risk assessment. Server-controlled crash on every backend that reads in parallel.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).

2. Run `bazel test //tools/zmlsmith/regressions:found_vfs_range --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_s3_range --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `vfs_range`: *fails* (matches `start index 5 is larger than end index 1`)
   `1/1 vfs_range.test.vfs_range...thread 3170297 panic: start index 5 is larger than end index 1`
   `s3_range`: *fails* (matches `start index 5 is larger than end index 1`)
   `1/1 s3_range.test.s3_range...thread 3161424 panic: start index 5 is larger than end index 1`

Verification: confirmed as reported. **Expected behavior, from:** parseContentRange returns ?ContentRange and returns null on unparsable input (vfs/http.zig:416-425, vfs/parallel_read.zig:161-171). A malformed header therefore has a designed rejection path, and a panic is not the contract.

- vfs/http.zig:417-419: const space = std.mem.indexOfScalar(u8, value, ' ') orelse return null; const dash = ...; const slash = ...;
- vfs/http.zig:422: .start = std.fmt.parseInt(u64, value[space + 1 .. dash], 10) catch return null,
- vfs/parallel_read.zig:161-168: pub fn parseContentRange(value: []const u8) ?ContentRange { ... value[space + 1 .. dash] ...
- vfs/http.zig:393: break :blk parseContentRange(header.value);

Suggested fix. Parse with `std.mem.indexOfScalarPos` from the previous delimiter and return an error on bad order.

tools/zmlsmith/regressions/found/vfs_range.zig (68 lines)

```
const std = @import("std");
const zml = @import("zml");

// vfs HTTP (and parallel_read, used by HF, S3 and GCS) parseContentRange slices the
// header between the first ' ', '-' and '/' without checking their order: a malformed
// "Content-Range: 0-15 bytes/16" from a server panics with "start index is larger than
// end index" instead of returning null (vfs/http.zig:416-425, vfs/parallel_read.zig:161
// at 74c590b0). Found while adding vfs coverage to zmlsmith.

/// Serves `head` to HEAD requests and `get` to the others, `n` connections.
const Server = struct {
    server: std.Io.net.Server,
    head: []const u8,
    get: []const u8,
    n: usize,

    fn run(s: *Server) void {
        const io = std.testing.io;
        for (0..s.n) |_| {
            const conn = s.server.accept(io) catch return;
            defer conn.close(io);
            var rbuf: [4096]u8 = undefined;
            var r = conn.reader(io, &rbuf);
            const first = r.interface.takeDelimiterExclusive('\n') catch return;
            const is_head = std.mem.startsWith(u8, first, "HEAD");
            r.interface.toss(1);
            while (true) {
                const line = r.interface.takeDelimiterExclusive('\n') catch return;
                r.interface.toss(1);
                if (line.len <= 1) break;
            }
            var wbuf: [4096]u8 = undefined;
            var w = conn.writer(io, &wbuf);
            w.interface.writeAll(if (is_head) s.head else s.get) catch return;
            w.interface.flush() catch return;
        }
    }
};

fn readThroughVfs(head: []const u8, get: []const u8, n: usize) !void {
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const addr = try std.Io.net.IpAddress.parseIp4("127.0.0.1", 0);
    var s: Server = .{ .server = try addr.listen(io, .{ .reuse_address = true }), .head = head, .get = get, .n = n };
    defer s.server.deinit(io);
    const t = try std.Thread.spawn(.{}, Server.run, .{&s});
    defer t.join();

    var client: std.http.Client = .{ .allocator = allocator, .io = io };
    defer client.deinit();
    var http: zml.io.VFS.HTTP = try .init(allocator, io, &client, .http);
    defer http.deinit();
    var vfs: zml.io.VFS = try .init(allocator, io);
    defer vfs.deinit();
    try vfs.register("http", http.io());
    const vio = vfs.io();
    var url_buf: [64]u8 = undefined;
    const url = try std.fmt.bufPrint(&url_buf, "http://127.0.0.1:{d}/f.bin", .{s.server.socket.address.getPort()});
    var file = std.Io.Dir.openFile(.cwd(), vio, url, .{ .mode = .read_only }) catch return;
    defer file.close(vio);
    var buf: [16]u8 = undefined;
    _ = file.readPositional(vio, &.{&buf}, 0) catch return;
}

test "vfs_range" {
    try readThroughVfs("HTTP/1.1 200 OK\r\nConnection: close\r\nContent-Length: 16\r\n\r\n", "HTTP/1.1 206 Partial Content\r\nConnection: close\r\nContent-Range:
0-15 bytes/16\r\nContent-Length: 16\r\n\r\n0123456789abcdef", 2);
}
```

tools/zmlsmith/regressions/found/s3_range.zig (68 lines)

```
const std = @import("std");
const zml = @import("zml");

// vfs parallel_read.parseContentRange, used by the S3, GCS and HF backends, has the same
// unchecked slicing as the HTTP one (vfs/parallel_read.zig:161-171 at 74c590b0): an S3
// endpoint answering a ranged GET with "Content-Range: 0-15 bytes/16" panics with "start
// index is larger than end index" in a read pool worker. Found while adding vfs coverage
// to zmlsmith, against a local endpoint.

/// Serves `head` to HEAD requests and `get` to the others, `n` connections.
const Server = struct {
    server: std.Io.net.Server,
    head: []const u8,
    get: []const u8,
    n: usize,

    fn run(s: *Server) void {
        const io = std.testing.io;
        for (0..s.n) |_| {
            const conn = s.server.accept(io) catch return;
            defer conn.close(io);
            var rbuf: [4096]u8 = undefined;
            var r = conn.reader(io, &rbuf);
            const first = r.interface.takeDelimiterExclusive('\n') catch return;
            const is_head = std.mem.startsWith(u8, first, "HEAD");
            r.interface.toss(1);
            while (true) {
                const line = r.interface.takeDelimiterExclusive('\n') catch return;
                r.interface.toss(1);
                if (line.len <= 1) break;
            }
            var wbuf: [4096]u8 = undefined;
            var w = conn.writer(io, &wbuf);
            w.interface.writeAll(if (is_head) s.head else s.get) catch return;
            w.interface.flush() catch return;
        }
    }
};

fn readThroughS3(head: []const u8, get: []const u8, n: usize) !void {
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const addr = try std.Io.net.IpAddress.parseIp4("127.0.0.1", 0);
    var s: Server = .{ .server = try addr.listen(io, .{ .reuse_address = true }), .head = head, .get = get, .n = n };
    defer s.server.deinit(io);
    const t = try std.Thread.spawn(.{}, Server.run, .{&s});
    defer t.join();

    var client: std.http.Client = .{ .allocator = allocator, .io = io };
    defer client.deinit();
    var endpoint_buf: [64]u8 = undefined;
    const endpoint = try std.fmt.bufPrint(&endpoint_buf, "http://127.0.0.1:{d}", .{s.server.socket.address.getPort()});
    var s3: zml.io.VFS.S3 = try .init(allocator, io, &client, .{ .access_key = "a", .secret_key = "b", .endpoint_url = endpoint, .region = "us-east-1" }, .
{ .read_pool = .{ .chunk_size = 16, .num_workers = 1, .queue_capacity = 4, .max_retries = 0, .retry_initial_delay = .fromMilliseconds(1), .retry_max_delay
= .fromMilliseconds(1) } });
    defer s3.deinit();
    var vfs: zml.io.VFS = try .init(allocator, io);
    defer vfs.deinit();
    try vfs.register("s3", s3.io());
    const vio = vfs.io();
    var file = std.Io.Dir.openFile(.cwd(), vio, "s3://bucket/f.bin", .{ .mode = .read_only }) catch return;
    defer file.close(vio);
    var buf: [16]u8 = undefined;
    _ = file.readPositional(vio, &.{&buf}, 0) catch return;
}

test "s3_range" {
    try readThroughS3("HTTP/1.1 200 OK\r\nConnection: close\r\nContent-Length: 16\r\n\r\n", "HTTP/1.1 206 Partial Content\r\nConnection: close\r\nContent-Range:
0-15 bytes/16\r\nContent-Length: 16\r\n\r\n0123456789abcdef", 2);
}
```

H17 3xx without a valid Location panics

HIGH ZML crash · VFS · risk 9 (impact 3 × exposure 3) · **untrusted input**

API vfs HTTP open (fetchSize)
Location `vfs/http.zig:344, :346`
Platform CPU

What happens. A 3xx answer to HEAD without a Location panics ("attempt to use null value"); a Location such as `http://` panics with "attempt to unwrap error: InvalidHostName".

Root cause. `res.head.location.?` and `catch unreachable` on URI resolution.

> **Risk assessment.** Server-controlled crash while opening a file.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_vfs_no_location --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_vfs_bad_location --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

`vfs_no_location`: *fails* (matches `attempt to use null value`)
`1/1 vfs_no_location.test.vfs_no_location...thread 3169306 panic: attempt to use null value`
`vfs_bad_location`: *fails* (matches `attempt to unwrap error: InvalidHostName`)
`1/1 vfs_bad_location.test.vfs_bad_location...thread 3169589 panic: attempt to unwrap error: InvalidHostName`

Verification: confirmed as reported. **Expected behavior, from:** A 3xx without a usable Location, or an unparsable one, is a malformed server response. fetchSize already returns error values for bad statuses (vfs/http.zig:336-341), so an error is the expected result.

- vfs/http.zig:344: const location = res.head.location.?;
- vfs/http.zig:346: uri = uri.resolveInPlace(location.len, &aux_buffer) catch unreachable; continue;

Suggested fix. Return an error for a missing or unparsable Location.

`tools/zmlsmith/regressions/found/vfs_no_location.zig` (66 lines)

```
const std = @import("std");
const zml = @import("zml");

// vfs HTTP fetchSize follows redirects with `res.head.location.?`: a 3xx answer to HEAD
// without a Location header panics with "attempt to use null value" instead of failing
// the open (vfs/http.zig:344 at 74c590b0). Found while adding vfs coverage to zmlsmith.

/// Serves `head` to HEAD requests and `get` to the others, `n` connections.
const Server = struct {
    server: std.Io.net.Server,
    head: []const u8,
    get: []const u8,
    n: usize,

    fn run(s: *Server) void {
        const io = std.testing.io;
        for (0..s.n) |_| {
            const conn = s.server.accept(io) catch return;
            defer conn.close(io);
            var rbuf: [4096]u8 = undefined;
            var r = conn.reader(io, &rbuf);
            const first = r.interface.takeDelimiterExclusive('\n') catch return;
            const is_head = std.mem.startsWith(u8, first, "HEAD");
            r.interface.toss(1);
            while (true) {
                const line = r.interface.takeDelimiterExclusive('\n') catch return;
                r.interface.toss(1);
                if (line.len <= 1) break;
            }
            var wbuf: [4096]u8 = undefined;
            var w = conn.writer(io, &wbuf);
            w.interface.writeAll(if (is_head) s.head else s.get) catch return;
            w.interface.flush() catch return;
        }
    }
};

fn readThroughVfs(head: []const u8, get: []const u8, n: usize) !void {
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const addr = try std.Io.net.IpAddress.parseIp4("127.0.0.1", 0);
    var s: Server = .{ .server = try addr.listen(io, .{ .reuse_address = true }), .head = head, .get = get, .n = n };
    defer s.server.deinit(io);
    const t = try std.Thread.spawn(.{}, Server.run, .{&s});
    defer t.join();

    var client: std.http.Client = .{ .allocator = allocator, .io = io };
    defer client.deinit();
    var http: zml.io.VFS.HTTP = try .init(allocator, io, &client, .http);
    defer http.deinit();
    var vfs: zml.io.VFS = try .init(allocator, io);
    defer vfs.deinit();
    try vfs.register("http", http.io());
    const vio = vfs.io();
    var url_buf: [64]u8 = undefined;
    const url = try std.fmt.bufPrint(&url_buf, "http://127.0.0.1:{d}/f.bin", .{s.server.socket.address.getPort()});
    var file = std.Io.Dir.openFile(.cwd(), vio, url, .{ .mode = .read_only }) catch return;
    defer file.close(vio);
    var buf: [16]u8 = undefined;
    _ = file.readPositional(vio, &.{&buf}, 0) catch return;
}

test "vfs_no_location" {
    try readThroughVfs("HTTP/1.1 302 Found\r\nConnection: close\r\nContent-Length: 0\r\n\r\n", "HTTP/1.1 200 OK\r\nConnection: close\r\nContent-Length:
0\r\n\r\n", 1);
}
```

`tools/zmlsmith/regressions/found/vfs_bad_location.zig` (67 lines)

```
const std = @import("std");
const zml = @import("zml");

// vfs HTTP fetchSize resolves a redirect Location with `catch unreachable`: a Location the
// URI parser rejects, e.g. "http://" with no host, panics ("attempt to unwrap error:
// InvalidHostName") instead of failing the open (vfs/http.zig:346 at 74c590b0). Found by
// zmlsmith parse-fuzz (vfs target).

/// Serves `head` to HEAD requests and `get` to the others, `n` connections.
const Server = struct {
    server: std.Io.net.Server,
    head: []const u8,
    get: []const u8,
```

```
    n: usize,

    fn run(s: *Server) void {
        const io = std.testing.io;
        for (0..s.n) |_| {
            const conn = s.server.accept(io) catch return;
            defer conn.close(io);
            var rbuf: [4096]u8 = undefined;
            var r = conn.reader(io, &rbuf);
            const first = r.interface.takeDelimiterExclusive('\n') catch return;
            const is_head = std.mem.startsWith(u8, first, "HEAD");
            r.interface.toss(1);
            while (true) {
                const line = r.interface.takeDelimiterExclusive('\n') catch return;
                r.interface.toss(1);
                if (line.len <= 1) break;
            }
            var wbuf: [4096]u8 = undefined;
            var w = conn.writer(io, &wbuf);
            w.interface.writeAll(if (is_head) s.head else s.get) catch return;
            w.interface.flush() catch return;
        }
    }
};

fn readThroughVfs(head: []const u8, get: []const u8, n: usize) !void {
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const addr = try std.Io.net.IpAddress.parseIp4("127.0.0.1", 0);
    var s: Server = .{ .server = try addr.listen(io, .{ .reuse_address = true }), .head = head, .get = get, .n = n };
    defer s.server.deinit(io);
    const t = try std.Thread.spawn(.{}, Server.run, .{&s});
    defer t.join();

    var client: std.http.Client = .{ .allocator = allocator, .io = io };
    defer client.deinit();
    var http: zml.io.VFS.HTTP = try .init(allocator, io, &client, .http);
    defer http.deinit();
    var vfs: zml.io.VFS = try .init(allocator, io);
    defer vfs.deinit();
    try vfs.register("http", http.io());
    const vio = vfs.io();
    var url_buf: [64]u8 = undefined;
    const url = try std.fmt.bufPrint(&url_buf, "http://127.0.0.1:{d}/f.bin", .{s.server.socket.address.getPort()});
    var file = std.Io.Dir.openFile(.cwd(), vio, url, .{ .mode = .read_only }) catch return;
    defer file.close(vio);
    var buf: [16]u8 = undefined;
    _ = file.readPositional(vio, &.{&buf}, 0) catch return;
}

test "vfs_bad_location" {
    try readThroughVfs("HTTP/1.1 302 Found\r\nConnection: close\r\nLocation: http://\r\nContent-Length: 0\r\n\r\n", "HTTP/1.1 200 OK\r\nConnection:
close\r\nContent-Length: 0\r\n\r\n", 1);
}
```

H18 resizeBilinear / Bicubic drop ResizeOpts.precision

HIGH ZML wrong values · nn · risk 9 (impact 3 × exposure 3)

API	nn.resizeBilinear, nn.resizeBicubic
Location	`zml/nn.zig:1418, :1485`
Platform	CPU
KnownBug	`resize_drops_precision`

What happens. With `.precision = .f32` on bf16, the result is bit-identical to the default-precision one.

Root cause. resizeBilinear (zml/nn.zig:1418-1420) and resizeBicubic (zml/nn.zig:1485-1487) build a fresh ResizeOpts per axis containing only original_len, so precision is dropped and the per-axis resizeLinear1d / resizeCubic1d run in the image dtype.

> **Risk assessment.** The documented workaround for the previous finding doesn't work for the 2-D resizes.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_resize_prec --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `resize_prec`: *fails* (matches `found/resize_prec\.zig:28:`)
   `/home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/5270/execroot/_main/tools/zmlsmith/regressions/found/resize_prec.zig:28:5: 0x5e1950bf5c4a in test.resize_prec (t`

Verification: confirmed, report corrected. **Expected behavior, from:** Code reading: resizeBilinear(opts) is documented to take ResizeOpts and ResizeOpts.precision says it controls internal precision (zml/nn.zig:1407-1410). The expected behaviour is that precision = .f32 changes the result of the 2-D resize as it does for resizeLinear1d.

- zml/nn.zig:1418: const child_opt: ResizeOpts = .{

- zml/nn.zig:1419: .original_len = if (opt.original_len) |o| o.slice(0, .single(ax)) else null,
- zml/nn.zig:1485: const child_opt: ResizeOpts = .{

Suggested fix. Forward the whole options struct.

tools/zmlsmith/regressions/found/resize_prec.zig (30 lines)

```
const std = @import("std");
const zml = @import("zml");
// resizeBilinear with .precision = .f32 on a bf16 image must match resizeLinear1d run with the
// same options. The per-axis call in nn.resizeBilinear (zml/nn.zig:1418-1420 at 74c590b0)
// keeps only original_len, so the bilinear result runs in bf16 and differs from the f32 one.
test "resize_prec" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const Local = struct {
        fn forward(x: zml.Tensor) [2]zml.Tensor {
            const opts: zml.nn.ResizeOpts = .{ .precision = .f32 };
            return .{ zml.nn.resizeBilinear(x, .{ .a = 7 }, opts), zml.nn.resizeLinear1d(x, 0, 7, opts) };
        }
    };
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .a = 3 }, .bf16));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x}, platform, .{});
    defer exe.deinit();
    const in = [_]zml.floats.BFloat16{ .fromF32(1.0), .fromF32(1.0078125), .fromF32(250.0) };
    var b: zml.Buffer = try .fromBytes(io, platform, x.shape(), .replicated, std.mem.sliceAsBytes(&in));
    defer b.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b});
    defer for (&res) |*r| r.deinit();
    const bilinear = try res[0].toSliceAlloc(allocator, io);
    defer bilinear.free(allocator);
    const linear = try res[1].toSliceAlloc(allocator, io);
    defer linear.free(allocator);
    try std.testing.expectEqualSlices(u8, linear.constData(), bilinear.constData());
}
```

H19 bf16 Rng.uniform falls outside [min, max)

HIGH ZML wrong values · Random · risk 9 (impact 3 × exposure 3)

API	Tensor.Rng.uniform (bf16)
Location	`zml/tensor.zig:769 (rng_bit_count), :781 (shift)`
Platform	CPU
KnownBug	`uniform_bf16`

What happens. Values fall outside the requested range.

Root cause. In Rng.uniform, `rng_bit_count = if (mantissa_bit_count < 8) 8 else bit_count` (zml/tensor.zig:769) was meant for 8-bit float types. bf16 has a 7-bit mantissa (zml/dtype.zig:380), so the condition is true and rng_bit_count becomes 8 instead of 16. The mantissa shift at zml/tensor.zig:781 is then 8-7 = 1 instead of 9, so bits 8..15 of the random u16 stay in the exponent field after the OR with 0x3F80 and the value is out of [1,2) and therefore outside [min,max). Simulation with random 16-bit words: 5001 of 10000 values out of range with shift 1, 0 with shift 9.

> **Risk assessment.** Silent wrong random tensors in bf16, a common dtype.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_uniform_bf16 --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `uniform_bf16`: *fails* (matches `found/uniform_bf16\.zig:42:`)
   `/home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/5172/execroot/_main/tools/zmlsmith/regressions/found/uniform_bf16.zig:42:9: 0x5fe36c3d07e6 in test.uniform_bf16`

Verification: confirmed, report corrected. **Expected behavior, from:** The doc at zml/tensor.zig:756-757 (values uniformly sampled from an interval) and the min/max arguments. Python simulation of the code path in verify/ (bit-level emulation, shift 1 vs 9).
Root cause checked by patching: `rng_bit_count = bit_count` (zml/tensor.zig:769). `found_uniform_bf16` passes (3 of 3 runs).

- zml/tensor.zig:769: const rng_bit_count = if (mantissa_bit_count < 8) 8 else bit_count;
- zml/tensor.zig:781: var float_bits = bits.shiftRightLogical(scalar(rng_bit_count - mantissa_bit_count, uint_dtype));
- zml/dtype.zig:380: .bf16 => 7,

Suggested fix. Set rng_bit_count = bit_count for all dtypes (for 8-bit float types bit_count is already 8, so the condition is not needed). Do not change the shift to 'the mantissa width': the shift is correct once rng_bit_count is 16.

tools/zmlsmith/regressions/found/uniform_bf16.zig (45 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500623.
test "uniform_bf16" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = blk: {
                const r: zml.Tensor.Rng = .{ ._state = x0 };
                _, const x = r.uniform(zml.Shape.init(.{ .h = 2 }, .bf16), .{ .min = -4, .max = 4 });
                break :blk x.cmp(.GE, zml.Tensor.scalar(@as(f64, -4), x.dtype())).logical(.AND, x.cmp(.LT, zml.Tensor.scalar(@as(f64, 4), x.dtype())));
            };
            _ = &x1;
            return .{x1};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{2}, .u64));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .h = 2 }, .bool), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            1, 1,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .h = 2 }, .bool), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

H20 linspace doesn't end at end

HIGH ZML wrong values · Tensor · risk 9 (impact 3 × exposure 3)

API	Tensor.linspace
Location	zml/tensor.zig:2511 (signature), :2525-2531 (scale/shift)
Platform	CPU
KnownBug	linspace

What happens. The last value isn't end.

Root cause. linspace (zml/tensor.zig:2511-2533) never uses args.end. It scales the iota by args.steps (zml/tensor.zig:2526) and adds args.start, so the values are start + i*steps. Observed: for start -5, end -2, steps 5 the output is -5, 0, 5, 10, 15; the last value is 15, error 17 against -2. The step should be (end - start)/(steps - 1), and steps == 1 should return start.

> **Risk assessment.** Silent off-by-one in a numpy-compatible API (positional encodings, grids).

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run bazel test //tools/zmlsmith/regressions:found_linspace --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   linspace: *fails* (matches p999_absolute_error: 17)
   p999_absolute_error: 17

Verification: confirmed, report corrected. **Expected behavior, from:** Doc comment zml/tensor.zig:2510: 'Returns a Tensor containing args.steps values evenly spaced from args.start to args.end, inclusive.' The reproducer expected values (-5, -4.25, -3.5, -2.75, -2) are numpy.linspace(-5,-2,5); verified by hand.

- zml/tensor.zig:2510: /// Returns a Tensor containing 'args.steps' values evenly spaced from 'args.start' to 'args.end', inclusive.
- zml/tensor.zig:2526: res = res.scale(args.steps);
- zml/tensor.zig:2530: res = res.addConstant(args.start);

Suggested fix. Divide by steps - 1 (and handle steps == 1).

tools/zmlsmith/regressions/found/linspace.zig (35 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500094.
test "linspace" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward() [1]zml.Tensor {
            const x0 = zml.Tensor.linspace(.{ .start = -5, .end = -2, .steps = 5 }, .f64);
            _ = &x0;
            return .{x0};
        }
    };

    var exe = try zml.module.compile(allocator, io, Local.forward, .{}, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{5}, .f64), exe.output_shapes[0]);

    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, {});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 20, 192, 0, 0, 0, 0, 0, 0, 17, 192, 0, 0, 0, 0, 0, 0, 12, 192,
            0, 0, 0, 0, 0, 0, 6,  192, 0, 0, 0, 0, 0, 0, 0,  192,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{5}, .f64), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .exact_match);
    }
}
```

H21 Slice .len is ignored

HIGH ZML shape · Tensor · risk 9 (impact 3 × exposure 3)

API	Tensor.slice (Slice.absolute)
Location	`zml/tensor.zig:1978-1989 (Slice.absolute)`
Platform	CPU
KnownBug	`slice_len`

What happens. A slice with `.len = 1` keeps the whole remaining axis.

Root cause. Slice.absolute tests `s.end == to_the_end` (zml/tensor.zig:1982) before it considers `s.len` (zml/tensor.zig:1986). A slice that sets `.len` and leaves `.end` unset therefore resolves to end = d and the length is never read.

Risk assessment. Wrong shape, which usually fails later, but can broadcast silently.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_slice_len --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `slice_len`: *fails* (matches `Expected \{a=1,e=2,bf16\}, got \{a=2,e=2,bf16\}`)
   `Expected {a=1,e=2,bf16}, got {a=2,e=2,bf16}FAIL (TestExpectedEqual)`

Verification: confirmed as reported. **Expected behavior, from:** Slice doc comment at zml/tensor.zig:1962 and :1967: "to keep the axis use .{ .start = off, .len = 1 }". The reproducer's expected shape {a=1,e=2} follows from that documented form.

- zml/tensor.zig:1982: } else if (s.end == to_the_end)
- zml/tensor.zig:1983: d
- zml/tensor.zig:1986: } else if (s.len != to_the_end)
- zml/tensor.zig:1967: /// Select a single element and drop the corresponding axis (to keep the axis use .{ .start = off, .len =1 })

Suggested fix. Compute `end = start + len` when len is set.

`tools/zmlsmith/regressions/found/slice_len.zig` (39 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500280.
test "slice_len" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = x0.slice(.a, .{ .start = 2, .len = 1, .step = 2 });
            _ = &x1;
```

```
            return .{x1};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .a = 5, .e = 2 }, .bf16));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .a = 1, .e = 2 }, .bf16), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .a = 1, .e = 2 }, .bf16), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .exact_match);
    }
}
```

H22 Paged attention: wrong outputs for batches with an empty sequence or a partial prefill chunk

HIGH ZML wrong values · Attention · risk 8 (impact 4 × exposure 2)

API attention.paged_attention.pagedAttention (.stablehlo)
Location `zml/attention/paged_attention.zig:626 (cond, has_queries), :694 (prefill if), :700 (q slice clamped), :698-727 (attentionOnePage)`
Platform CPU (stablehlo backend)

What happens. (1) A sequence with no queries in the middle of the batch stops the loop, and every later sequence is left at 0. (2) A prefill whose query count isn't a multiple of page_size, as the last sequence, slices a full page of queries past the end of q: the dynamic-slice start is clamped and the shifted rows are attended with the wrong causal positions and written over earlier outputs.

Root cause. (1) cond at :626 requires the current seq_id to have queries, so the while loop stops at the first zero-query sequence and later sequences are never written (their outputs stay 0). Mechanism verified; whether callers may pass zero-query entries is not specified in the source (no contract doc on pagedAttention, zml/attention/paged_attention.zig:277). (2) For a prefill whose query count is not a multiple of page_size, the last chunk of the last sequence slices page_size rows at :700 via dyn(slot_id, q_chunk); the start index is clamped by the dynamic slice, so the rows read and the rows written by updateState's dynamicUpdateSlice (:753) are shifted onto earlier rows of the same sequence, with positions computed from q_offset. For non-last sequences the extra rows belong to the next sequence and are overwritten later, so the wrong results only show for the last sequence.

Risk assessment. Wrong attention outputs in the CPU stablehlo path for the two batch shapes above. Only the stablehlo backend was exercised. Whether the shapes arise in the serving loop depends on the caller; the claim 'both shapes occur in a serving loop' is not established by the code.

Reproduction.
1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_paged_attn --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `paged_attn`: *fails* (matches `token 1 lane 0: got 0, want [0-9.]+`)
   `token 1 lane 0: got 0, want 5.918403`

Verification: confirmed, report corrected. **Expected behavior, from:** Expected values come from a host reference in the reproducer (causal paged attention, query j of a sequence with n queries at position seq_len - n + j, seq_lens including the current queries). The convention matches the comment at zml/attention/paged_attention.zig:655 ('seq_lens includes the current queries'). This is the varlen convention used by FlashAttention-style kernels; the reference itself is the fuzzer's implementation, not an external library.
- zml/attention/paged_attention.zig:626: pub fn cond(self: While, state: State) zml.Tensor {
- zml/attention/paged_attention.zig:630: const has_queries = query_start.cmp(.LT, query_end).asScalar();
- zml/attention/paged_attention.zig:694: .cmp(seq_num_queries_, .GT, .scalar(1, .i32)),
- zml/attention/paged_attention.zig:700: const active_q = self.q.slice(self.q.axis(.b), .dyn(state.slot_id, q_chunk));
- zml/attention/paged_attention.zig:655: // seq_lens includes the current queries; subtracting their count gives the context length.

Suggested fix. Iterate over all sequences and skip empty ones; mask, or slice and pad, the last chunk instead of clamping.

tools/zmlsmith/regressions/found/paged_attn.zig (157 lines, first 70 shown)

```
const std = @import("std");
const zml = @import("zml");
const paged = zml.attention.paged_attention;

// The stablehlo (CPU) backend of attention.paged_attention.pagedAttention, against a host
// reference of causal paged attention: query j of a sequence with n queries and seq_len L
// sits at position L - n + j and attends keys 0..L - n + j, read from the pages listed in
// block_table. Found by auditing zmlsmith coverage of zml/attention.

const ps = 4; // page_size
const num_pages = 8;
const max_pages = 4;
const hd = 4;

const Case = struct {
    q_start: []const i32, // query_start_len, batch + 1 entries
    seq_lens: []const i32,
    block_table: []const [max_pages]i32,
};

fn reference(c: Case, q: []const f32, kc: []const f32, vc: []const f32, out: []f32) void {
    @memset(out, 0);
    const scale = 1 / @sqrt(@as(f32, hd));
    for (c.seq_lens, 0..) |len, s| {
        const qs: usize = @intCast(c.q_start[s]);
        const qe: usize = @intCast(c.q_start[s + 1]);
        const n = qe - qs;
        for (0..n) |j| {
            const pos: usize = @as(usize, @intCast(len)) - n + j;
            var w: [max_pages * ps]f32 = undefined;
            var m: f32 = -std.math.inf(f32);
            for (0..pos + 1) |p| {
                const page: usize = @intCast(c.block_table[s][p / ps]);
                var d: f32 = 0;
                for (0..hd) |x| d += q[(qs + j) * hd + x] * kc[(page * ps + p % ps) * hd + x];
                w[p] = d * scale;
                m = @max(m, w[p]);
            }
            var sum: f32 = 0;
            for (0..pos + 1) |p| {
                w[p] = @exp(w[p] - m);
                sum += w[p];
            }
            for (0..hd) |x| {
                var acc: f32 = 0;
                for (0..pos + 1) |p| {
                    const page: usize = @intCast(c.block_table[s][p / ps]);
                    acc += w[p] / sum * vc[(page * ps + p % ps) * hd + x];
                }
                out[(qs + j) * hd + x] = acc;
            }
        }
    }
}

fn check(comptime batch: u32, comptime tokens: u32, c: Case) !void {
    const platform = zml.testing.env();
    const io = std.testing.io;
    const allocator = std.testing.allocator;

    const parameters: paged.Parameters = .init(.fromBackend(.{
        .backend = .stablehlo,
        .is_prefill = true,
        .batch_size = batch,
        .seq_len = max_pages * ps,
        .max_num_pages = max_pages,
        .max_token_count = tokens,
        .num_heads = 1,
        .num_kv_heads = 1,
        .head_dim = hd,
```

H23 Homemade tokenizer encode returns a slice that can't be freed

HIGH ZML memory · Tokenizer · risk 8 (impact 4 × exposure 2)

API tokenizer homemade Tokenizer.encode
Location `zml/tokenizer/homemade.zig:288-289`
Platform CPU

What happens. When the allocator can't resize in place, the caller frees a slice shorter than the allocation: an invalid free.

Root cause. `_ = allocator.resize(buf, n)` ignores the result, then returns `buf[0..n]`.

> **Risk assessment.** Heap corruption with general-purpose allocators. Exposure is low because the homemade loaders don't compile (see below), so only direct users are affected.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).

2. Run `bazel test //tools/zmlsmith/regressions:found_tok_encode_free --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `tok_encode_free`: *fails* (matches `Invalid free`)
   `1/1 tok_encode_free.test.tok_encode_free...thread 3164880 panic: Invalid free`

Verification: confirmed as reported. **Expected behavior, from:** Zig std Allocator contract: `free` must receive the length of the original allocation, and `resize` may be refused by returning false (lib/std/mem/Allocator.zig, resize doc at :304-313; free at :440-443). The code ignores the resize result (homemade.zig:288) and then returns a shorter slice (:289).
- zml/tokenizer/homemade.zig:287-289: allocator.free(mergeable); _ = allocator.resize(tok_buff_alloc, num_tokens); return tok_buff_alloc[0..num_tokens];
- zml/tokenizer/homemade.zig:149: const tok_buff_alloc = try allocator.alloc(u32, @max(options.pad_to, input.len + 2));

Suggested fix. Use `allocator.realloc`, or `remap` with a copy fallback.

`tools/zmlsmith/regressions/found/tok_encode_free.zig` (22 lines)

```zig
const std = @import("std");
const homemade = @import("zml").tokenizer.homemade;

// homemade.Tokenizer.encode shrinks its result with `_ = allocator.resize(buf, n)` and
// ignores the result, then returns buf[0..n] (zml/tokenizer/homemade.zig:288-289 at
// 74c590b0). When the allocator can't resize in place the caller holds a slice shorter
// than the allocation, and freeing it, as encode's callers do, is an invalid free
// (detected by the testing allocator; undefined behavior otherwise). Found by zmlsmith
// tokenizer property tests.
test "tok_encode_free" {
    const allocator = std.testing.allocator;
    var t = try homemade.Tokenizer.init(allocator, 5, 4, null, .{ .bos = 0, .eos = 1, .unk = 2 }, true);
    defer t.deinit();
    try t.addToken(0, "<s>");
    try t.addToken(0, "</s>");
    try t.addToken(0, "<unk>");
    try t.addToken(0, "a");
    try t.addToken(1, "aa");
    const ids = try t.encode(allocator, "aaaaaaaa", .{ .add_bos = false });
    allocator.free(ids);
}
```

H24 FNUZ formats use the IEEE exponent bias

HIGH ZML wrong values · Numerics · risk 8 (impact 4 × exposure 2)

API zml.floats Float8E4M3FNUZ, Float8E5M2FNUZ, B11FNUZ
Location `zml/floats.zig:23`
Platform CPU

What happens. E4M3FNUZ and E5M2FNUZ values decode and encode at 2x their true value (exponent bias 7 and 15 instead of 8 and 16). B11FNUZ values are off by 16x (bias 7 instead of 11). fromF32(-0.0) gives 0x80 for the FNUZ formats, which is their NaN code, so -0 turns into NaN. The e8m0 0xff decode (inf instead of NaN, zml/floats.zig:445-452) is a separate issue and not an FNUZ issue; it should be split out.

Root cause. exp_bias is maxInt(unsigned exponent_bits - 1) (zml/floats.zig:23): 7 for a 4-bit exponent and 15 for a 5-bit exponent. The FNUZ biases are 8 (E4M3FNUZ) and 16 (E5M2FNUZ); B11FNUZ uses bias 11.

Risk assessment. Silent wrong values for every FNUZ value read or written on the host (AMD fp8 checkpoints). Reads are off by 2x (E4M3FNUZ, E5M2FNUZ) or 16x (B11FNUZ).

Reproduction.
1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_host_floats --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `host_floats`: *fails* (matches `exponent bias\.\.\.expected 0\.0078125, found 0\.015625`)
   `1/4 host_floats.test.floats: FNUZ and B11FNUZ exponent bias...expected 0.0078125, found 0.015625`

Verification: confirmed, report corrected. **Expected behavior, from:** OCP 8-bit float bias conventions as used by host_floats.zig (reference comment 'Reference values from the OCP 8-bit float spec and ml_dtypes'): the smallest normal of E4M3FNUZ is code 0x08 (exponent 1, mantissa 0), which is 2^(1-8) = 2^-7 at bias 8, matching the reproducer's std.math.pow(f32, 2, -7); E5M2FNUZ code 0x04 is 2^(1-16) = 2^-15 at bias 16; B11FNUZ code 0x08 is 2^(1-11) = 2^-10 at bias 11.
- zml/floats.zig:23: 'const exp_bias: i16 = std.math.maxInt(std.meta.Int(.unsigned, exponent_bits - 1));'
- zml/floats.zig:243-244: 'pub const nan: Float8E4M3FNUZ = .{ .sign = 1, .exponent = 0, .mantissa = 0 };'

- tools/zmlsmith/regressions/found/host_floats.zig:8-10: 'expectEqual(std.math.pow(f32, 2, -7), Float8E4M3FNUZ.toF32(0x08))' and 'expectEqual(std.math.pow(f32, 2, -10), Float8E4M3B11FNUZ.toF32(0x08))'

Suggested fix. Give each format its own bias (8 / 16 / 11) instead of the generic maxInt formula, and map -0 to +0 in fromF32 for the FNUZ formats (the only FNUZ NaN is 0x80).

tools/zmlsmith/regressions/found/host_floats.zig (30 lines)

```
const std = @import("std");
const zml = @import("zml");
const floats = zml.floats;

// Reference values from the OCP 8-bit float spec and ml_dtypes.

test "floats: FNUZ and B11FNUZ exponent bias" {
    // Smallest normal: exponent field 1, mantissa 0, is 2^(1 - bias).
    try std.testing.expectEqual(std.math.pow(f32, 2, -7), floats.Float8E4M3FNUZ.toF32(@bitCast(@as(u8, 0x08))));
    try std.testing.expectEqual(std.math.pow(f32, 2, -15), floats.Float8E5M2FNUZ.toF32(@bitCast(@as(u8, 0x04))));
    try std.testing.expectEqual(std.math.pow(f32, 2, -10), floats.Float8E4M3B11FNUZ.toF32(@bitCast(@as(u8, 0x08))));
}

test "floats: f8 subnormals" {
    // E4M3FN code 0x01 is the smallest subnormal, 2^-9.
    try std.testing.expectEqual(std.math.pow(f32, 2, -9), floats.Float8E4M3FN.toF32(@bitCast(@as(u8, 0x01))));
    // E5M2 code 0x01 is 2^-16.
    try std.testing.expectEqual(std.math.pow(f32, 2, -16), floats.Float8E5M2.toF32(@bitCast(@as(u8, 0x01))));
}

test "floats: e8m0 0xff is NaN" {
    try std.testing.expect(std.math.isNan(floats.Float8E8M0.toF32(@bitCast(@as(u8, 0xff)))));
}

test "floats: -0.0 doesn't encode to the FNUZ NaN" {
    // FNUZ types have no negative zero: 0x80 is their only NaN.
    try std.testing.expect(@as(u8, @bitCast(floats.Float8E4M3FNUZ.fromF32(-0.0))) != 0x80);
    try std.testing.expect(@as(u8, @bitCast(floats.Float8E5M2FNUZ.fromF32(-0.0))) != 0x80);
}
```

H25 scatterSlices with rank-0 indices ignores update_fn

HIGH ZML wrong values · Tensor · risk 8 (impact 4 × exposure 2)

API	Tensor.scatterSlices
Location	zml/ops.zig:1790
Platform	CPU
KnownBug	scatter_scalar_indices

What happens. .increment overwrites instead of adding. Other shapes on the same path panic inside dynamicUpdateSlice (rank, axis order, dims).

Root cause. Rank-0 indices are rewritten to dynamicUpdateSlice, which has no combiner.

Risk assessment. Silent wrong accumulation (e.g. a counter or histogram update with one index).

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_scatter_scalar --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   scatter_scalar: *fails* (matches p999_absolute_error)
   p999_absolute_error: 1

Verification: confirmed as reported. **Expected behavior, from:** ScatterOpts.update_fn defaults to increment and is an explicit option (zml/tensor.zig:3103). The reproducer passes update_fn = increment explicitly and expects x0 + x2 at index k.

- zml/ops.zig:1790-1791: if (@TypeOf(inputs) == struct { Tensor } and indices_shape.rank() == 0) { return .{self.dynamicUpdateSlice(index_tensors, update)}; }
- zml/tensor.zig:3103: update_fn: *const fn (ops.ScatterArgs) struct { Tensor } = increment,

Suggested fix. Only take the shortcut for the overwrite update_fn.

tools/zmlsmith/regressions/found/scatter_scalar.zig (49 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500297.
test "scatter_scalar_indices" {
    const platform = zml.testing.env();
```

```
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor, x1: zml.Tensor, x2: zml.Tensor) [1]zml.Tensor {
            const x3 = x0.scatterSlices(.{ .k = x1 }, x2, .{ .update_fn = zml.Tensor.ScatterOpts.increment, .indices_are_unique = false });
            _ = &x3;
            return .{x3};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .k = 1 }, .f64));
    const x1: zml.Tensor = .fromShape(zml.Shape.init(.{}, .i64));
    const x2: zml.Tensor = .fromShape(zml.Shape.init(.{}, .f64));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x0, x1, x2 }, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .k = 1 }, .f64), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 240, 63,
    });
    defer b0.deinit();
    var b1: zml.Buffer = try .fromBytes(io, platform, x1.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b1.deinit();
    var b2: zml.Buffer = try .fromBytes(io, platform, x2.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b2.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ b0, b1, b2 });
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 240, 63,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .k = 1 }, .f64), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .exact_match);
    }
}
```

H26 ops.while with the same tensor in two state slots

HIGH ZML wrong values · Control flow · risk 8 (impact 4 × exposure 2)

API zml.ops.while
Location `zml/ops.zig:521 (cond block argument registration), zml/ops.zig:544 (body block argument registration)`
Platform CPU
KnownBug `while_aliased_state`

What happens. `.{x, x}` with a+1, b*2 over 3 steps gives a = 5 instead of 4.

Root cause. Block arguments are registered by tensor id, so both slots read the last one.

> **Risk assessment.** Silent wrong loop results whenever two state fields start from the same tensor (common for zero-initialized accumulators).

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_while_aliased --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `while_aliased`: *fails* (matches `expected 4, found 5`)
   `expected 4, found 5`

Verification: confirmed as reported. **Expected behavior, from:** Loop state fields are independent tensors. The reproducer's expectation a = 1 + 3 = 4 is the direct reading of body(a+1) over 3 iterations.

- zml/ops.zig:521: scope.registerTensorAsBlockArgument(input.id, i);
- zml/ops.zig:544: scope.registerTensorAsBlockArgument(input.id, i);
- zml/ops.zig:521/544 register by input.id; both state slots hold the same tensor id, so the last registration (slot 2, b) shadows slot 1 and `st.a` reads b's block argument. Hand trace: b doubles, a = b+1 gives a = 2, 3, 5 after three steps (observed 5, expected 4).

Suggested fix. Register block arguments by slot index, not tensor id.

`tools/zmlsmith/regressions/found/while_aliased.zig` (40 lines)

```
const std = @import("std");
const zml = @import("zml");

// The same tensor in two loop state slots: ops.while registers block arguments by tensor
// id, so both slots read the last registered argument.
```

```
test "while_aliased_state" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Loop = struct {
        pub const State = struct { i: zml.Tensor, a: zml.Tensor, b: zml.Tensor };
        pub fn cond(_: @This(), st: State) zml.Tensor {
            return st.i.cmp(.LT, zml.Tensor.scalar(3, .i32));
        }
        pub fn body(_: @This(), st: State) State {
            return .{ .i = st.i.addConstant(1), .a = st.a.addConstant(1), .b = st.b.scale(2) };
        }
    };
    const Local = struct {
        fn forward(x: zml.Tensor) [2]zml.Tensor {
            const r = zml.ops.@"while"(Loop, .{}, .{ .i = zml.Tensor.scalar(0, .i32), .a = x, .b = x });
            return .{ r.a, r.b };
        }
    };

    const x: zml.Tensor = .fromShape(zml.Shape.init(.{}, .f32));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x}, platform, .{});
    defer exe.deinit();

    var bx: zml.Buffer = try .scalar(io, platform, 1.0, .f32);
    defer bx.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{bx});
    defer for (&res) |*r| r.deinit();

    // a: 1 + 3 = 4, b: 1 * 2^3 = 8.
    try std.testing.expectEqual(@as(f32, 4), try res[0].getValue(f32, io));
    try std.testing.expectEqual(@as(f32, 8), try res[1].getValue(f32, io));
}
```

H27 broadcast ignores axes when dims already match

HIGH ZML wrong values · Tensor · risk 8 (impact 4 × exposure 2)

API	Tensor.broadcast
Location	`zml/tensor.zig:2628`
Platform	CPU
KnownBug	`broadcast_equal_dims_perm`

What happens. `broadcast(x.shape(), &.{1, 0})` on a square tensor should transpose it and returns it unchanged.

Root cause. Early return of self when dims match, before looking at `axes_`.

> **Risk assessment.** Silent wrong values that only show on square shapes, so tests on rectangular shapes pass.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_broad_permute --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

   `broad_permute`: *fails* (matches `found/broad_permute\.zig:27:`)

   ```
   /home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/4545/execroot/_main/
   tools/zmlsmith/regressions/found/broad_permute.zig:27:5: 0x5c234946b6d4 in test.broad_permut
   ```

Verification: confirmed as reported. **Expected behavior, from:** Tensor.broadcast takes axes_ as the mapping of self axes to output axes (zml/tensor.zig:2621 asserts it), so [1,0] on a 2x2 tensor is a transpose. Expected {1,3,2,4} is the transpose of {1,2,3,4} in row-major order.

- zml/tensor.zig:2628: if (std.mem.eql(i64, self.dims(), output_shape.dims())) {
- zml/tensor.zig:2628-2631: if (std.mem.eql(...dims...)) { // No broadcast needed ... return _result(res_shape, self.value()); } (returns self without consulting axes_)

Suggested fix. Only return early when axes are the identity.

`tools/zmlsmith/regressions/found/broad_permute.zig` (29 lines)

```
const std = @import("std");
const zml = @import("zml");

// Tensor.broadcast returns self when the dims already match, ignoring `axes_`: mapping
// axis 0 to 1 and 1 to 0 on a square tensor should transpose it.
test "broad_permute" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x: zml.Tensor) zml.Tensor {
            return x.broadcast(x.shape(), &.{ 1, 0 });
```

```
        }
    };
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ 2, 2 }, .f32));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x}, platform, .{});
    defer exe.deinit();

    const in = [_]f32{ 1, 2, 3, 4 };
    var b: zml.Buffer = try .fromBytes(io, platform, x.shape(), .replicated, std.mem.sliceAsBytes(&in));
    defer b.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b});
    defer res.deinit();
    const got = try res.toSliceAlloc(allocator, io);
    defer got.free(allocator);
    try std.testing.expectEqualSlices(f32, &.{ 1, 3, 2, 4 }, got.items(f32));
}
```

H28 Convolution with negative padding and permuted layouts gives wrong values

HIGH upstream XLA wrong values · XLA CPU · risk 8 (impact 4 × exposure 2)

API conv2d with padding < 0 and permuted *_dimension options
Location `XLA CPU (conv lowering or layout assignment, not identified). ZML side: zml/tensor.zig:1040-1066 computes the output shape and passes the padding through unchanged.`
Platform CPU
KnownBug `xla_conv_neg_pad_layout`

What happens. 8 of 120 random cases differ from a naive grouped convolution. With padding >= 0, or negative padding in the default layout, all are right; ZML passes the padding through unchanged.

Root cause. Upstream XLA CPU. The StableHLO is valid: negative convolution padding is a pad with negative edges, and the convolution is defined through padded_lhs = pad(lhs, ...) (docs/spec.md '### convolution', around line 2275; pad allows negative edge padding, docs/spec.md:4703-4709). The XLA mechanism is not identified; layout permutation plus negative padding is the only trigger found.

> **Risk assessment.** Silent, but the combination is rare.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_xla_conv_neg --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `xla_conv_neg: fails (matches round \d+: in .* pad \{ .*-1.* layouts)`
   `round 8: in { 1, 4, 4, 3 } kernel { 4, 2, 2, 2 } strides { 1, 1 } pad { -1, 1, 0, -1 } groups 2 layouts { 0, 2, 3, 1 } { 0, 1, 2, 3 } { 0, 1, 2, 3 }`

Verification: confirmed, report corrected. **Expected behavior, from:** StableHLO convolution is defined via pad(lhs, padding) (docs/spec.md:2275); pad allows negative edge padding (docs/spec.md:4703-4709). The naive reference in the reproducer (xla_conv_neg.zig lines 11-30) implements that definition: out-of-range taps are skipped as zero, and the output size uses d + lo + hi.

- docs/spec.md:4703: 'each dimension respectively. The amount of padding can be negative, where the'
- docs/spec.md:2275: '* `padded_lhs = pad(lhs, padding_value, lhs_padding[:, 0], lhs_padding[:, 1], lhs_base_dilations - 1)`.'
- zml/tensor.zig:1046-1054: output shape computed from padded_input_shape (negative pad values included), padding passed unchanged to stablehlo.convolution

Suggested fix. Report upstream; in ZML, lower negative padding as a slice before the conv.

`tools/zmlsmith/regressions/found/xla_conv_neg.zig` (140 lines, first 70 shown)

```
const std = @import("std");
const zml = @import("zml");

// Upstream XLA CPU: a convolution with negative padding and non default dimension numbers
// (any of the input, kernel or output layouts permuted) gives wrong values, against a
// naive grouped convolution with StableHLO semantics; 8 of 120 random cases differ.
// The same cases with padding >= 0 (conv_layouts.zig), or negative padding in the
// default layout, are right. ZML passes the padding through unchanged
// (zml/tensor.zig:1040-1065). Found by zmlsmith (conv family, seed 250785).

/// out[b][o][y][x] = sum over c in o's group, ky, kx of in[b][g*cg + c][y*s + ky - pl][x*s + kx - pl'].
fn naive(in: []const f32, k: []const f32, out: []f32, d: [4]usize, kd: [4]usize, od: [4]usize, s: [2]i64, pad: [4]i64, groups: usize) void {
    const cg = d[1] / groups;
    const og = kd[0] / groups;
    for (0..od[0]) |b| for (0..od[1]) |o| for (0..od[2]) |y| for (0..od[3]) |x| {
        var acc: f32 = 0;
        const g = o / og;
        for (0..cg) |c| for (0..kd[2]) |ky| for (0..kd[3]) |kx| {
            const iy = @as(i64, @intCast(y)) * s[0] + @as(i64, @intCast(ky)) - pad[0];
            const ix = @as(i64, @intCast(x)) * s[1] + @as(i64, @intCast(kx)) - pad[2];
```

```
            if (iy < 0 or ix < 0 or iy >= d[2] or ix >= d[3]) continue;
            const ii = ((b * d[1] + g * cg + c) * d[2] + @as(usize, @intCast(iy))) * d[3] + @as(usize, @intCast(ix));
            const ki = ((o * kd[1] + c) * kd[2] + ky) * kd[3] + kx;
            acc += in[ii] * k[ki];
        };
        out[((b * od[1] + o) * od[2] + y) * od[3] + x] = acc;
    };
}

/// Physical layouts: perm[r] is the physical axis of logical role r ([b|o, c|i, y, x]).
const S = struct {
    var st: [2]i64 = undefined;
    var pd: [4]i64 = undefined;
    var groups: i64 = 1;
    var il: [4]i64 = .{ 0, 1, 2, 3 };
    var kl: [4]i64 = .{ 0, 1, 2, 3 };
    var ol: [4]i64 = .{ 0, 1, 2, 3 };
    fn f(x: zml.Tensor, k: zml.Tensor) zml.Tensor {
        return x.conv2d(k, .{
            .window_strides = &st,
            .padding = &pd,
            .feature_group_count = groups,
            .input_batch_dimension = il[0],
            .input_feature_dimension = il[1],
            .input_spatial_dimensions = il[2..4],
            .kernel_output_feature_dimension = kl[0],
            .kernel_input_feature_dimension = kl[1],
            .kernel_spatial_dimensions = kl[2..4],
            .output_batch_dimension = ol[0],
            .output_feature_dimension = ol[1],
            .output_spatial_dimensions = ol[2..4],
        });
    }
};

/// Logical row-major [4] dims and data to the physical layout `perm`.
fn physical(perm: [4]i64, d: [4]usize, logical: []const f32, out: []f32) [4]usize {
    var pd: [4]usize = undefined;
    for (0..4) |r| pd[@intCast(perm[r])] = d[r];
    for (0..d[0]) |a| for (0..d[1]) |b| for (0..d[2]) |c| for (0..d[3]) |e| {
        const li = ((a * d[1] + b) * d[2] + c) * d[3] + e;
        var pi: [4]usize = undefined;
        const idx = [4]usize{ a, b, c, e };
        for (0..4) |r| pi[@intCast(perm[r])] = idx[r];
        out[((pi[0] * pd[1] + pi[1]) * pd[2] + pi[2]) * pd[3] + pi[3]] = logical[li];
    };
    return pd;
}

test "xla_conv_neg" {
```

H29 SPMD reduce_window with base dilation and padding returns shifted values

HIGH upstream XLA wrong values · Sharding · risk 8 (impact 4 × exposure 2)

API	reduceWindow on the mesh
Location	`XLA SPMD partitioner`
Platform	CPU, 4-device mesh
KnownBug	`sharded_reduce_window`

What happens. Wrong values once a consumer's sharding propagates to a replicated reduce_window input.

Root cause. SPMD partitioning of a dilated, padded window.

> **Risk assessment.** Silent wrong values in sharded programs only.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_shard_rwindow --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

   `shard_rwindow`: *fails* (matches `found/shard_rwindow\.zig:71:`)

   ```
   /home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/4847/execroot/_main/
   tools/zmlsmith/regressions/found/shard_rwindow.zig:71:9: 0x5c1163b108ea in test.sharded_redu
   ```

Verification: confirmed as reported. **Expected behavior, from:** StableHLO reduce_window with base dilation and padding is valid; output dim 0 = (5-1)*2+1 + 1 + 2 = 12, matching n = 12 in the reproducer (shard_rwindow.zig lines 12 and 30). The consumer (cmp against the sharded x3) makes the partitioner propagate a sharding into the replicated window input.

- tools/zmlsmith/regressions/found/shard_rwindow.zig:12: 'base_dilations = &[_]i64{ 2, 1 }, ...padding = &[_][2]i64{ .{ 1, 2 }, .{ 0, 0 } }'
- tools/zmlsmith/regressions/found/shard_rwindow.zig:30: 'x3 = ... Shape.init(.{ .n = 12, .b = 1 }, .u64).withPartitioning(.{ .n = .data })'

Suggested fix. Report upstream.

`tools/zmlsmith/regressions/found/shard_rwindow.zig` (74 lines, first 70 shown)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500986.
test "sharded_reduce_window" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor, x3: zml.Tensor) [2]zml.Tensor {
            const x1 = zml.ops.reduceWindow(1, .{x0}, .{zml.Tensor.constant(zml.DataType.zero(.u64))}, .{ .window_dimensions = &[_]i64{ 1, 1 }, .window_strides =
&[_]i64{ 1, 1 }, .base_dilations = &[_]i64{ 2, 1 }, .window_dilations = &[_]i64{ 1, 1 }, .padding = &[_][2]i64{ .{ 1, 2 }, .{ 0, 0 } } }, &struct {
                fn f(a: zml.ops.ReduceArgs) [1]zml.Tensor {
                    return .{a.left.add(a.right)};
                }
            }.f)[0];
            _ = &x1;
            const x2 = x1.reshape(@as([]const i64, &.{ 12, 1 }));
            _ = &x2;
            const x4 = x1.cmp(.GE, x3);
            _ = &x4;
            const x5 = x4.triangular(.{ .b, .n }, 0);
            _ = &x5;
            return .{ x5, x2 };
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_500986", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .n = 5, .b = 1 }, .u64));
    const x3: zml.Tensor = .fromShape(zml.Shape.init(.{ .n = 12, .b = 1 }, .u64).withPartitioning(.{ .n = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x0, x3 }, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .n = 12, .b = 1 }, .bool), exe.output_shapes[0]);
    try zml.testing.expectEqualShapes(zml.Shape.init(.{ 12, 1 }, .u64), exe.output_shapes[1]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
        1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var b3: zml.Buffer = try .fromBytes(io, platform, x3.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b3.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ b0, b3 });
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .n = 12, .b = 1 }, .bool), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ 12, 1 }, .u64), @as([]const u8, &expected_data));
        const got = try res[1].toSliceAlloc(allocator, io);
        defer got.free(allocator);
```

H30 resizeBicubic permutes axes for axis >= 2

HIGH ZML wrong values · nn · risk 6 (impact 3 × exposure 2)

API	nn.resizeBicubic
Location	`zml/nn.zig:1561`
Platform	CPU
KnownBug	`cubic_axis_order`

What happens. When dims coincide the result is silently transposed, otherwise the final assert fires.

Root cause. The bicubic result is laid out with the new axis first (batch dim of the pos/values dot) and the axis is moved back with `swapAxes(0, axis)` (zml/nn.zig:1560-1561). swapAxes exchanges dims 0 and axis and leaves the other dims in the wrong order, so for axis >= 2 the non-resized dims are permuted. The axis 1 case is correct, which is why it only shows for axis >= 2. The correct step is a moveAxis from 0 to axis (inverse permutation).

> **Risk assessment.** Silent transposition on square images.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).

2. Run `bazel test //tools/zmlsmith/regressions:found_cubic_order_val --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_cubic_order --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `cubic_order_val`: *fails* (matches `p999_absolute_error: [0-9.]+`)
   `p999_absolute_error: 7.5`
   `cubic_order`: *fails* (matches `reached unreachable code`)
   `1/1 cubic_order.test.cubic_axis_order...thread 3150635 panic: reached unreachable code`

Verification: confirmed as reported. **Expected behavior, from:** Code reading and shape arithmetic: for dims (a0,a1,a2) resized on axis 2, the output before the swap is [N, a0, a1]; swapAxes(0,2) gives [a1, a0, N] instead of [a0, a1, N]. Expected values in cubic_order_val are the per-axis formula: a python model of the axis-3 formula with XLA window clamp reproduces the expected bytes exactly (max error 0.0), so the expectation is the intended result.

- zml/nn.zig:1560: if (axis != 0) {
- zml/nn.zig:1561: res = res.swapAxes(0, axis);
- zml/nn.zig:1567: std.debug.assert(std.mem.eql(i64, res_shape.dims(), res.dims()));

Suggested fix. Use the inverse permutation.

`tools/zmlsmith/regressions/found/cubic_order_val.zig` (130 lines, first 70 shown)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500660.
test "cubic_axis_order_values" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = zml.nn.resizeBicubic(x0, .{ .n = @as(u63, 4) }, .{});
            _ = &x1;
            return .{x1};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .g = 5, .a = 5, .n = 7 }, .f64));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .g = 5, .a = 5, .n = 4 }, .f64), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 240, 63,  0, 0, 0, 0, 0, 0, 208, 63,  0, 0, 0, 0, 0, 0, 240, 191,
        0, 0, 0, 0, 0, 0, 0, 16,  64,  0, 0, 0, 0, 0, 0, 240, 191, 0, 0, 0, 0, 0, 0, 12,  192,
        0, 0, 0, 0, 0, 0, 0, 10,  192, 0, 0, 0, 0, 0, 0, 244, 63,  0, 0, 0, 0, 0, 0, 4,   192,
        0, 0, 0, 0, 0, 0, 0, 16,  64,  0, 0, 0, 0, 0, 0, 244, 63,  0, 0, 0, 0, 0, 0, 6,   192,
        0, 0, 0, 0, 0, 0, 0, 4,   192, 0, 0, 0, 0, 0, 0, 240, 63,  0, 0, 0, 0, 0, 0, 0,   64,
        0, 0, 0, 0, 0, 0, 0, 6,   64,  0, 0, 0, 0, 0, 0, 224, 191, 0, 0, 0, 0, 0, 0, 232, 191,
        0, 0, 0, 0, 0, 0, 0, 10,  64,  0, 0, 0, 0, 0, 0, 2,   192, 0, 0, 0, 0, 0, 0, 232, 63,
        0, 0, 0, 0, 0, 0, 0, 14,  64,  0, 0, 0, 0, 0, 0, 2,   192, 0, 0, 0, 0, 0, 0, 12,  64,
        0, 0, 0, 0, 0, 0, 0, 224, 63,  0, 0, 0, 0, 0, 0, 240, 191, 0, 0, 0, 0, 0, 0, 12,  192,
        0, 0, 0, 0, 0, 0, 0, 12,  192, 0, 0, 0, 0, 0, 0, 232, 191, 0, 0, 0, 0, 0, 0, 0,   64,
        0, 0, 0, 0, 0, 0, 0, 224, 63,  0, 0, 0, 0, 0, 0, 16,  64,  0, 0, 0, 0, 0, 0, 14,  192,
        0, 0, 0, 0, 0, 0, 0, 232, 63,  0, 0, 0, 0, 0, 0, 244, 63,  0, 0, 0, 0, 0, 0, 2,   192,
        0, 0, 0, 0, 0, 0, 0, 240, 191, 0, 0, 0, 0, 0, 0, 4,   64,  0, 0, 0, 0, 0, 0, 4,   64,
        0, 0, 0, 0, 0, 0, 0, 232, 191, 0, 0, 0, 0, 0, 0, 0,   0,   0, 0, 0, 0, 0, 0, 12,  192,
        0, 0, 0, 0, 0, 0, 0, 10,  64,  0, 0, 0, 0, 0, 0, 0,   192, 0, 0, 0, 0, 0, 0, 14,  64,
        0, 0, 0, 0, 0, 0, 0, 240, 191, 0, 0, 0, 0, 0, 0, 232, 63,  0, 0, 0, 0, 0, 0, 208, 191,
        0, 0, 0, 0, 0, 0, 0, 252, 63,  0, 0, 0, 0, 0, 0, 16,  64,  0, 0, 0, 0, 0, 0, 16,  64,
        0, 0, 0, 0, 0, 0, 0, 2,   64,  0, 0, 0, 0, 0, 0, 16,  192, 0, 0, 0, 0, 0, 0, 2,   192,
        0, 0, 0, 0, 0, 0, 0, 252, 63,  0, 0, 0, 0, 0, 0, 244, 191, 0, 0, 0, 0, 0, 0, 240, 191,
        0, 0, 0, 0, 0, 0, 0, 252, 191, 0, 0, 0, 0, 0, 0, 2,   192, 0, 0, 0, 0, 0, 0, 0,   0,
        0, 0, 0, 0, 0, 0, 0, 248, 191, 0, 0, 0, 0, 0, 0, 232, 63,  0, 0, 0, 0, 0, 0, 2,   192,
        0, 0, 0, 0, 0, 0, 0, 2,   64,  0, 0, 0, 0, 0, 0, 16,  64,  0, 0, 0, 0, 0, 0, 252, 191,
        0, 0, 0, 0, 0, 0, 0, 240, 191, 0, 0, 0, 0, 0, 0, 6,   192, 0, 0, 0, 0, 0, 0, 208, 191,
        0, 0, 0, 0, 0, 0, 0, 10,  64,  0, 0, 0, 0, 0, 0, 244, 191, 0, 0, 0, 0, 0, 0, 0,   0,
        0, 0, 0, 0, 0, 0, 0, 14,  192, 0, 0, 0, 0, 0, 0, 16,  64,  0, 0, 0, 0, 0, 0, 224, 191,
        0, 0, 0, 0, 0, 0, 0, 252, 63,  0, 0, 0, 0, 0, 0, 12,  192, 0, 0, 0, 0, 0, 0, 2,   192,
        0, 0, 0, 0, 0, 0, 0, 224, 191, 0, 0, 0, 0, 0, 0, 14,  64,  0, 0, 0, 0, 0, 0, 252, 63,
        0, 0, 0, 0, 0, 0, 0, 6,   64,  0, 0, 0, 0, 0, 0, 12,  64,  0, 0, 0, 0, 0, 0, 4,   64,
        0, 0, 0, 0, 0, 0, 0, 8,   192, 0, 0, 0, 0, 0, 0, 4,   192, 0, 0, 0, 0, 0, 0, 16,  64,
        0, 0, 0, 0, 0, 0, 0, 244, 63,  0, 0, 0, 0, 0, 0, 240, 191, 0, 0, 0, 0, 0, 0, 0,   0,
        0, 0, 0, 0, 0, 0, 0, 2,   64,  0, 0, 0, 0, 0, 0, 240, 63,  0, 0, 0, 0, 0, 0, 248, 191,
        0, 0, 0, 0, 0, 0, 0, 244, 191, 0, 0, 0, 0, 0, 0, 248, 191, 0, 0, 0, 0, 0, 0, 240, 63,
        0, 0, 0, 0, 0, 0, 0, 16,  192, 0, 0, 0, 0, 0, 0, 4,   64,  0, 0, 0, 0, 0, 0, 6,   64,
        0, 0, 0, 0, 0, 0, 0, 0,   192, 0, 0, 0, 0, 0, 0, 224, 191, 0, 0, 0, 0, 0, 0, 0,   64,
        0, 0, 0, 0, 0, 0, 0, 12,  192, 0, 0, 0, 0, 0, 0, 6,   64,  0, 0, 0, 0, 0, 0, 2,   192,
        0, 0, 0, 0, 0, 0, 0, 252, 63,  0, 0, 0, 0, 0, 0, 16,  64,  0, 0, 0, 0, 0, 0, 232, 191,
        0, 0, 0, 0, 0, 0, 0, 4,   192, 0, 0, 0, 0, 0, 0, 248, 63,  0, 0, 0, 0, 0, 0, 248, 191,
        0, 0, 0, 0, 0, 0, 0, 16,  64,  0, 0, 0, 0, 0, 0, 208, 191, 0, 0, 0, 0, 0, 0, 224, 191,
        0, 0, 0, 0, 0, 0, 0, 12,  64,  0, 0, 0, 0, 0, 0, 240, 63,  0, 0, 0, 0, 0, 0, 0,   64,
        0, 0, 0, 0, 0, 0, 0, 0,   64,  0, 0, 0, 0, 0, 0, 240, 63,  0, 0, 0, 0, 0, 0, 12,  192,
        0, 0, 0, 0, 0, 0, 0, 0,   192, 0, 0, 0, 0, 0, 0, 248, 191, 0, 0, 0, 0, 0, 0, 12,  192,
        0, 0, 0, 0, 0, 0, 0, 12,  192, 0, 0, 0, 0, 0, 0, 12,  64,  0, 0, 0, 0, 0, 0, 12,  64,
        0, 0, 0, 0, 0, 0, 0, 224, 63,  0, 0, 0, 0, 0, 0, 16,  192, 0, 0, 0, 0, 0, 0, 14,  64,
        0, 0, 0, 0, 0, 0, 0, 232, 191, 0, 0, 0, 0, 0, 0, 0,   0,   0, 0, 0, 0, 0, 0, 14,  192,
        0, 0, 0, 0, 0, 0, 0, 10,  64,  0, 0, 0, 0, 0, 0, 252, 63,  0, 0, 0, 0, 0, 0, 252, 191,
        0, 0, 0, 0, 0, 0, 0, 244, 63,  0, 0, 0, 0, 0, 0, 10,  192, 0, 0, 0, 0, 0, 0, 232, 191,
```

tools/zmlsmith/regressions/found/cubic_order.zig (43 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500294.
test "cubic_axis_order" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = zml.nn.resizeBicubic(x0, .{ .m = @as(u63, 2) }, .{});
            _ = &x1;
            return .{x1};
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_500294", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .h = 1, .c = 4, .e = 1, .m = 4 }, .f32).withPartitioning(.{ .c = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .h = 1, .c = 4, .e = 1, .m = 2 }, .f32), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .h = 1, .c = 4, .e = 1, .m = 2 }, .f32), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .{ .absolute_tolerance = 3.0517578125e-5, .relative_tolerance =
2.384185791015625e-7, .minimum_close_fraction = 1 });
    }
}
```

H31 maximum(clamp(lo, x, hi), lo) is simplified to the clamp

HIGH upstream XLA wrong values · XLA · risk 6 (impact 3 × exposure 2)

API	max/min of a clamp with its own bound
Location	`XLA algebraic simplifier, xla/hlo/transforms/simplifiers/algebraic_simplifier.cc:4766-4770 (external; no ZML change involved)`
Platform	CPU
KnownBug	`xla_clamp_simplify`

What happens. With x = hi = -100 and lo = -4 the result is -100 instead of -4.

Root cause. The simplifier assumes lo <= hi; StableHLO defines clamp as min(max(x, lo), hi) without that precondition.

Risk assessment. Silent; only when bounds can cross.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_xla_clamp_max --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

   `xla_clamp_max`: *fails* (matches `found/xla_clamp_max\.zig:35:`)

   ```
   /home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/5150/execroot/_main/
   tools/zmlsmith/regressions/found/xla_clamp_max.zig:35:5: 0x652f4d42798f in test.xla_clamp_ma
   ```

Verification: confirmed, report corrected. **Expected behavior, from:** StableHLO clamp is defined as minimum(maximum(operand, min), max) (docs/spec.md:1742-1748). Its constraints (C1)-(C4) do not require min <= max. XLA's rule rewrites maximum(clamp(lo, x, hi), lo) into the clamp, which is only valid when clamp(...) >= lo; with lo > hi that fails.

- docs/spec.md:1746-1748: 'More formally, `result[result_index] = minimum(maximum(operand[result_index], min_element), max_element)`'
- xla/hlo/transforms/simplifiers/algebraic_simplifier.cc:4768-4770: 'if (max_operand == clamp_lower_bound && ReplaceInstructionIfCompatible(maximum, clamp))'

Suggested fix. Report upstream.

tools/zmlsmith/regressions/found/xla_clamp_max.zig (37 lines)

```
const std = @import("std");
const zml = @import("zml");

// Upstream XLA: maximum(clamp(lo, x, hi), lo) is simplified to the clamp, which assumes
// clamp(...) >= lo. StableHLO defines clamp as min(max(x, lo), hi) without requiring
// lo <= hi: with x = hi = -100 and lo = -4 the clamp is -100 and the maximum must be -4.
// Found by zmlsmith from seed 1600886.
test "xla_clamp_max" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x: zml.Tensor, lo: zml.Tensor) [2]zml.Tensor {
            const c = x.clamp(lo, x);
            return .{ c, c.maximum(lo) };
        }
    };
    const s: zml.Shape = .init(.{2}, .i64);
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ zml.Tensor.fromShape(s), zml.Tensor.fromShape(s) }, platform, .{});
    defer exe.deinit();

    var xb: zml.Buffer = try .fromBytes(io, platform, s, .replicated, std.mem.sliceAsBytes(&[_]i64{ -100, 5 }));
    defer xb.deinit();
    var lb: zml.Buffer = try .fromBytes(io, platform, s, .replicated, std.mem.sliceAsBytes(&[_]i64{ -4, -4 }));
    defer lb.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ xb, lb });
    defer for (&res) |*r| r.deinit();

    const c = try res[0].toSliceAlloc(allocator, io);
    defer c.free(allocator);
    try std.testing.expectEqualSlices(i64, &.{ -100, 5 }, c.items(i64));
    const m = try res[1].toSliceAlloc(allocator, io);
    defer m.free(allocator);
    try std.testing.expectEqualSlices(i64, &.{ -4, 5 }, m.items(i64));
}
```

H32 f4e2m1 to integer saturates +-6.0

HIGH upstream XLA wrong values · XLA · risk 6 (impact 3 × exposure 2)

API	Tensor.convert from f4e2m1 to integers
Location	XLA
Platform	CPU
KnownBug	xla_f4_to_int

What happens. +-6.0, the largest code, becomes the integer type's max/min (2147483647 for i32, 255 for u8). Every other code, and conversions to floats, are right.

Root cause. Upstream conversion of the top f4 code.

> **Risk assessment.** Silent; affects dequantization paths that go through integers.

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_xla_f4_to_int --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   xla_f4_to_int: *fails* (matches found/xla_f4_to_int\.zig:30:)
   ```
   /home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/5189/execroot/_main/
   tools/zmlsmith/regressions/found/xla_f4_to_int.zig:30:5: 0x5ef88e1c3081 in test.xla_f4_to_in
   ```

Verification: confirmed as reported. **Expected behavior, from:** StableHLO convert float-to-integer truncates the fractional part; the result is TBD only when the truncated value is not representable in the destination type (docs/spec.md:2197-2199). 6.0 is representable in i32, so 6 is the required result. Converting the finite code 0x7 to i32 as 2147483647 is therefore a bug in XLA.

- docs/spec.md:2197-2199: 'For conversions involving **floating-point-to-integer**, the fractional part is truncated. If the truncated value cannot be represented in the destination type, the behavior is TBD'
- tools/zmlsmith/regressions/found/xla_f4_to_int.zig: 'try std.testing.expectEqualSlices(i32, &.{ 6, 4, -6, -4 }, got.items(i32));'

Suggested fix. Report upstream; in ZML, convert through f32.

tools/zmlsmith/regressions/found/xla_f4_to_int.zig (32 lines)

```
const std = @import("std");
const zml = @import("zml");

// Upstream XLA: converting f4e2m1 to an integer type saturates +-6.0, the largest code, to
// the integer type's max/min. Every other code converts correctly, and so do conversions
// to floats. Found by zmlsmith from seed 900061.
test "xla_f4_to_int" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
```

```
    const io = std.testing.io;

    const Local = struct {
        fn forward(x: zml.Tensor) zml.Tensor {
            return x.convert(.i32);
        }
    };

    // Codes 0x7 (6.0), 0x6 (4.0), 0xf (-6.0), 0xe (-4.0), low nibble first.
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .k = 4 }, .f4e2m1));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x}, platform, .{});
    defer exe.deinit();

    var b: zml.Buffer = try .fromBytes(io, platform, x.shape(), .replicated, &[_]u8{ 0x67, 0xef });
    defer b.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b});
    defer res.deinit();

    const got = try res.toSliceAlloc(allocator, io);
    defer got.free(allocator);
    try std.testing.expectEqualSlices(i32, &.{ 6, 4, -6, -4 }, got.items(i32));
}
```

6 Medium findings

M01 TensorStore.View.hasKey matches by prefix

MEDIUM ZML wrong values · IO · risk 12 (impact 4 × exposure 3)

API io.TensorStore.View.hasKey
Location zml/io.zig:157-163
Platform CPU

What happens. With only `lm_head.weight_scale_inv` in the checkpoint, `hasKey("lm_head.weight")` is true, and so is `hasKey("model.layers.1")` with only `model.layers.10.*`.

Root cause. `std.mem.startsWith(k, key)` instead of an exact match or a match on a `.` boundary.

Risk assessment. Callers test exact tensor names: examples/llm/models/qwen3_5/model.zig:158 (store.hasKey("lm_head.weight")) and examples/llm/models/qwen3_5_moe/model.zig:306 (store.root().hasKey("lm_head.weight")). A checkpoint that has lm_head.weight_scale_inv but no lm_head.weight selects the lm_head prefix and then fails when it loads lm_head.weight. The model.layers.1 case has no caller in the repository.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_store_has_key --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

```
store_has_key: fails (matches found/store_has_key\.zig:41:)
/home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/4991/execroot/_main/
tools/zmlsmith/regressions/found/store_has_key.zig:41:5: 0x64521c0b4c7a in test.store_has_ke
```

Verification: confirmed, report corrected. **Expected behavior, from:** The API takes a tensor subkey and the callers pass exact tensor names, so exact or dot-boundary matching is the implied contract. Prefix matching is not documented anywhere in zml/io.zig. The reproducer's expectation (no false positive) follows from that name-based use.

- zml/io.zig:157-163: pub fn hasKey(self: *const View, subkey: []const u8) bool { ... const key = makeKey(&buffer, "{s}{s}", ...); return for (self.store.registry.tensors.keys()) |k| { if (std.mem.startsWith(u8, k, key)) break true; } else false; }
- examples/llm/models/qwen3_5/model.zig:158: if (store.hasKey("lm_head.weight")) break :b "lm_head";
- examples/llm/models/qwen3_5_moe/model.zig:306: const lm_head_prefix = if (store.root().hasKey("lm_head.weight")) "lm_head" else "model.language_model.embed_tokens";

Suggested fix. Match exact keys, or prefixes followed by `.`.

tools/zmlsmith/regressions/found/store_has_key.zig (44 lines)

```
const std = @import("std");
const zml = @import("zml");

// io.TensorStore.View.hasKey matches keys by prefix (`std.mem.startsWith(k, key)`,
// zml/io.zig:157-163 at 74c590b0): with only "lm_head.weight_scale_inv" in the checkpoint,
// hasKey("lm_head.weight") is true, and so is hasKey("model.layers.1") with only
// "model.layers.10.*". Callers use it to test for a tensor before creating it
// (examples/llm/models/qwen3_5/model.zig:158). Found while adding TensorStore coverage
// to zmlsmith.
const keys = [_][]const u8{ "model.layers.10.w", "model.layers.2.w", "model.norm", "lm_head.weight_scale_inv" };

/// Writes `keys` as f32[2] tensors holding 4 * index + {0, 1} and returns the registry.
fn checkpoint(allocator: std.mem.Allocator, io: std.Io, dir: std.Io.Dir) !zml.safetensors.TensorRegistry {
    var header: std.Io.Writer.Allocating = .init(allocator);
    defer header.deinit();
    try header.writer.writeAll("{");
    for (keys, 0..) |k, i| try header.writer.print("{s}\"{s}\":{{\"dtype\":\"F32\",\"shape\":[2],\"data_offsets\":[{d},{d}]}}", .{ if (i > 0) "," else "", k, 8 *
i, 8 * i + 8 });
    try header.writer.writeAll("}");
    var file: std.Io.Writer.Allocating = .init(allocator);
    defer file.deinit();
    try file.writer.writeInt(u64, header.written().len, .little);
    try file.writer.writeAll(header.written());
    for (0..keys.len) |i| for (0..2) |j| try file.writer.writeAll(std.mem.asBytes(&@as(f32, @floatFromInt(4 * i + j))));
    try dir.writeFile(io, .{ .sub_path = "m.safetensors", .data = file.written() });
    var path_buf: [1024]u8 = undefined;
    const n = try dir.realPath(io, &path_buf);
    const path = try std.fmt.allocPrint(allocator, "{s}/m.safetensors", .{path_buf[0..n]});
    defer allocator.free(path);
    return .fromPath(allocator, io, path);
}

test "store_has_key" {
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    var tmp = std.testing.tmpDir(.{});
    defer tmp.cleanup();
    var registry = try checkpoint(allocator, io, tmp.dir);
```

```
    defer registry.deinit();
    var store: zml.io.TensorStore = .fromRegistry(allocator, &registry);
    defer store.deinit();
    try std.testing.expect(!store.view().hasKey("lm_head.weight"));
    try std.testing.expect(!store.view().hasKey("model.layers.1"));
}
```

M02 maximum / minimum and max / min reductions don't propagate NaN on CPU

MEDIUM ZML wrong values · Numerics · risk 12 (impact 4 × exposure 3)

API	Tensor.maximum, minimum, max, min, clamp
Location	`zml/Compiler.zig:841-889 (XLA option override block; no xla_cpu_* flag is set anywhere in zml)`
Platform	CPU
KnownBug	`cpu_fast_min_max`

What happens. maximum(1, NaN) is 1, max([NaN, 1, NaN, -2]) is 1, f64 clamp(NaN, -inf, inf) is -inf. Everything built on them inherits it (maxPool, the max in softmax, some relu lanes).

Root cause. XLA CPU lowers maximum and max reductions through its fast min/max path when xla_cpu_enable_fast_min_max is true, and that flag defaults to true (xla/debug_options_flags.cc:252-253, with the flag help at :1267-1271 saying it may not propagate NaNs). ZML does not override it, so the default applies. StableHLO defines maximum and minimum for floats as IEEE-754 maximum and minimum (docs/spec.md:4368, :4413), so the XLA default deviates from the spec ZML targets.

> **Risk assessment.** NaN inputs to maximum, minimum, max/min reductions (and ops built on them: maxPool, softmax max, some relu lanes) can be replaced by finite values, hiding diverging models. This is an explicit XLA default and performance trade-off, not a crash or memory issue.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_nan_minmax --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

   `nan_minmax`: *fails* (matches `found/nan_minmax\.zig:32:`)

   ```
   /home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/4671/execroot/_main/
   tools/zmlsmith/regressions/found/nan_minmax.zig:32:5: 0x5751f53af11e in test.nan_minmax (too
   ```

Verification: reproduced; the cause is a documented XLA CPU default (fast min/max) that deviates from the StableHLO spec. **Expected behavior, from:** docs/spec.md:4368 (maximum: 'For floats: maximum from IEEE-754'), :4413 (minimum). XLA default: xla/debug_options_flags.cc:252-253 ('By default, copy TF's Eigen style min_max behavior with nans.' then set_xla_cpu_enable_fast_min_max(true)); flag help :1267-1271. GPU counterpart: debug_options_flags.cc:271-273 sets xla_gpu_enable_fast_min_max=false.

- xla/debug_options_flags.cc:252-253: '// By default, copy TF's Eigen style min_max behavior with nans.' / 'opts.set_xla_cpu_enable_fast_min_max(true);'
- xla/debug_options_flags.cc:1270-1271: 'Enable fast floating point min/max lowering that might not propagate NaNs.'
- xla/debug_options_flags.cc:273: 'opts.set_xla_gpu_enable_fast_min_max(false);' (GPU already defaults to NaN-propagating lowering)
- docs/spec.md:4368: '* For floats: `maximum` from IEEE-754.'
- zml/Compiler.zig:841-889: XLA override block sets only gpu/dump flags; grep finds no xla_cpu_enable_fast_min_max in zml/

Suggested fix. Decide whether ZML's CPU target must follow the StableHLO NaN semantics. If yes, set xla_cpu_enable_fast_min_max=false in the CPU compile options; the performance cost is not measured here.

`tools/zmlsmith/regressions/found/nan_minmax.zig` (37 lines)

```
const std = @import("std");
const zml = @import("zml");

// On CPU, maximum and the max reduction don't propagate NaN, as StableHLO specifies:
// maximum(1, NaN) is 1 and max([NaN, 1, NaN, -2]) is 1. XLA CPU enables fast min/max by
// default (xla_cpu_enable_fast_min_max) and ZML doesn't override it; with
// XLA_FLAGS=--xla_cpu_enable_fast_min_max=false both are NaN. Found by zmlsmith (ieee).
test "nan_minmax" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x: zml.Tensor, y: zml.Tensor) [2]zml.Tensor {
            return .{ x.maximum(y), x.max(0) };
        }
    };
    const s: zml.Shape = .init(.{4}, .f32);
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ zml.Tensor.fromShape(s), zml.Tensor.fromShape(s) }, platform, .{});
```

```
    defer exe.deinit();

    const nan = std.math.nan(f32);
    var xb: zml.Buffer = try .fromBytes(io, platform, s, .replicated, std.mem.sliceAsBytes(&[_]f32{ nan, 1, nan, -2 }));
    defer xb.deinit();
    var yb: zml.Buffer = try .fromBytes(io, platform, s, .replicated, std.mem.sliceAsBytes(&[_]f32{ 0, nan, nan, 3 }));
    defer yb.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ xb, yb });
    defer for (&res) |*r| r.deinit();

    const m = try res[0].toSliceAlloc(allocator, io);
    defer m.free(allocator);
    try std.testing.expect(std.math.isNan(m.items(f32)[1]));
    const r = try res[1].toSliceAlloc(allocator, io);
    defer r.free(allocator);
    try std.testing.expect(std.math.isNan(r.items(f32)[0]));
}
```

M03 Resize sample coordinates are computed in f16 / bf16

MEDIUM ZML wrong values · nn · risk 12 (impact 4 × exposure 3)

API nn.resizeLinear1d and the resizes built on it
Location `zml/nn.zig:1456-1460`
Platform CPU
KnownBug `resize_half_coords`

What happens. Resizing a bf16 pattern of 1000 values in 0..6 to 333 gives a worst error of about 9 at output 170 when precision is left at the default. f16 and bf16 coordinates are rounded to the image dtype, so the sample positions are wrong on long axes. With .precision = .f32 the result is exact.

Root cause. ratio (og_len.convert(dtype).scale(...)) and scaled (arange(dtype) * ratio) are computed in the image dtype unless ResizeOpts.precision is set (zml/nn.zig:1456-1459). Coordinates above 2^8 (bf16) or 2^11 (f16) are coarser than one pixel, so the wrong neighbours are read.

> **Risk assessment.** Lower-quality resizes for half-precision images unless the caller sets precision. This is the documented default, so it is a precision-quality issue, not an undocumented behaviour; the report should not call it silent corruption of the API contract.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_resize_coords --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `resize_coords`: *fails* (matches `worst error [0-9.]+ at output [0-9]+`)
   `worst error 9.063063063063169 at output 170`

Verification: confirmed, report corrected. **Expected behavior, from:** zml/nn.zig:1407-1410 doc comment on ResizeOpts.precision: 'If not set, will use the image dtype, unless it's an integer type, in which case f32 will be used.' So the default is documented. The expected values in the reproducer are exact bilinear values (the fuzzer's reference), a reasonable quality target but not a documented contract.

- zml/nn.zig:1409: /// If not set, will use the image dtype, unless it's an integer type, in which case f32 will be used.
- zml/nn.zig:1456: const dtype = opt.precision orelse if (image.dtype().class() == .integer) .f32 else image.dtype();
- zml/nn.zig:1459: const scaled = Tensor.arange(.{ .end = new_len }, dtype).mul(ratio);

Suggested fix. Always compute coordinates in f32.

`tools/zmlsmith/regressions/found/resize_coords.zig` (52 lines)

```
const std = @import("std");
const zml = @import("zml");

// nn.resizeLinear1d computes the sample coordinates (arange * ratio, floor) in the image
// dtype unless ResizeOpts.precision is given (zml/nn.zig:1456-1460 at 74c590b0): for a
// bf16 image, coordinates above 256 are rounded to multiples of 2 or 4, so the
// interpolation reads the wrong pixels with weights outside [0, 1]. Resizing a bf16
// pattern of 1000 values in 0..6 to 333 gives an error of 9.06 at output 170. f16 images
// are affected too, from a few hundred pixels. Found by zmlsmith (resize family, an f16
// axis of 125 resized to 76: 8 of 76 values off, by up to 0.75).
test "resize_coords" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const n = 1000;
    const m = 333;
    const Local = struct {
        fn f(x: zml.Tensor) zml.Tensor {
```

```
            return zml.nn.resizeLinear1d(x, 0, m, .{});
        }
    };
    const x: zml.Tensor = .init(.{n}, .bf16);
    var exe = try platform.compileFn(allocator, io, Local.f, .{x}, .{});
    defer exe.deinit();
    // Values i % 7: exact in bf16, and neighbors differ.
    var data: [n]zml.floats.BFloat16 = undefined;
    for (&data, 0..) |*d, i| d.* = .fromF32(@floatFromInt(i % 7));
    var b = try zml.Buffer.fromBytes(io, platform, x.shape(), .replicated, std.mem.sliceAsBytes(&data));
    defer b.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.f, .{b});
    defer res.deinit();
    const got = try res.toSliceAlloc(allocator, io);
    defer got.free(allocator);
    var worst: f64 = 0;
    var at: usize = 0;
    for (got.constItems(zml.floats.BFloat16), 0..) |g, i| {
        const pos = @as(f64, @floatFromInt(i)) * n / m;
        const l: usize = @intFromFloat(@floor(pos));
        const w = pos - @floor(pos);
        const r = @min(l + 1, n - 1);
        const want = @as(f64, @floatFromInt(l % 7)) * (1 - w) + @as(f64, @floatFromInt(r % 7)) * w;
        const d = @abs(@as(f64, g.toF32()) - want);
        if (d > worst) {
            worst = d;
            at = i;
        }
    }
    std.debug.print("worst error {d} at output {d}\n", .{ worst, at });
    // bf16 rounding of the result alone is below 0.05 for values below 7.
    try std.testing.expect(worst < 0.1);
}
```

M04 zml.floats conversions don't round to nearest even and mishandle NaN

MEDIUM ZML wrong values · Numerics · risk 12 (impact 3 × exposure 4)

API zml.floats fromF32 (DataType.constant, Tensor.scalar)
Location `zml/floats.zig:27-55 (generic fromF32, used by Float8E4M3FN, Float8E5M2, Float8E3M4, Float4E2M1 and others) with truncMantissa at zml/floats.zig:82-85; zml/floats.zig:354-358 (BFloat16.fromF32); zml/dtype.zig:287-289 (DataType.Value.init calls fromF32 for constants).`
Platform CPU

What happens. BFloat16.fromF32(1 + 2^-8) is 1.0078125 (ties up; the even result is 1.0). Float8E4M3FN.fromF32(1.97) is 1.0 instead of 2.0 (a factor-2 error at a binade boundary). BFloat16.fromF32 turns NaN 0x7F800001 into +inf and 0x7FFF8000 into -0, and overflows u32 (a safety panic) for negative NaNs at or above 0xFFFF8000. Float8E5M2.fromF32(NaN) is inf, and Float8E5M2.nan.toF32() decodes as a finite value.

Root cause. BFloat16.fromF32 adds 0x8000 to the f32 bits (zml/floats.zig:357) with no NaN check, so ties round up instead of to even, NaN payloads can round into the exponent (0x7F800001 -> +inf), 0x7FFF8000 carries into the sign (-0), and for 0xFFFF8000 and above the add overflows u32 (a safety panic). The generic path's truncMantissa (zml/floats.zig:82-85) adds half a unit and truncates the result to the mantissa width, so a carry out of the mantissa is dropped instead of incrementing the exponent (1.97 -> 1.0 in f8e4m3fn). NaN is not special-cased in the overflow branch, so e5m2 NaN becomes inf (zml/floats.zig:32-36).

> **Risk assessment.** Every constant and scalar in a non-f32 dtype goes through these: small systematic biases, and f8 constants off by 2x near binade boundaries. The NaN overflow is a panic on a value.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_floats_round --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_bf16_nan_ovf --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `floats_round`: *fails* (matches `expected 1, found 1\.0078125`)
   `1/7 floats_round.test.floats_round_bf16_tie...expected 1, found 1.0078125`
   `bf16_nan_ovf`: *fails* (matches `panic: integer overflow`)
   `1/1 bf16_nan_ovf.test.bf16_nan_ovf...thread 3152144 panic: integer overflow`

Verification: confirmed, report corrected. **Expected behavior, from:** Round-to-nearest-even is the StableHLO reduce_precision rule ('roundToIntegralTiesToEven', docs/spec.md:5047-5049), and the reproducers use it as the expected value; NaN and inf behaviour follow IEEE-754 (NaN in, NaN out). Constant conversions are the ZML side of the same contract (dtype.zig:287-289).

- zml/floats.zig:357: ' int += 0x8000;'
- zml/floats.zig:82-85: 'fn truncMantissa(...) { const rounding_val ...; return @truncate((f32_mantissa + rounding_val) >> (f32_mantissa_bits - mantissa_bits)); }'
- zml/floats.zig:32-36: overflow test 'std.math.isInf(f) or exponent > maxInt(...)' then 'if (vf32.sign == 0) Float.inf else Float.minus_inf'

- zml/dtype.zig:288-289: '.comptime_float, .float => @unionInit(Value, @tagName(tag), @FieldType(Value, @tagName(tag)).fromF32(@floatCast(value)))'

Suggested fix. Use round-to-nearest-even with carry propagation, and special-case NaN / inf.

tools/zmlsmith/regressions/found/floats_round.zig (45 lines)

```
const std = @import("std");
const zml = @import("zml");
const floats = zml.floats;

// zml.floats fromF32 doesn't round to nearest even. BFloat16.fromF32 adds 0x8000 (half
// up); the f8/f4 truncMantissa adds half a unit and truncates the mantissa. These feed
// DataType.constant, so Tensor.scalar(x, .bf16) and f8 constants are affected.
// Found by zmlsmith host checks.
test "floats_round_bf16_tie" {
    // 1 + 2^-8 is halfway between bf16 1.0 and 1.0078125: the even one is 1.0.
    try std.testing.expectEqual(@as(f32, 1.0), floats.BFloat16.fromF32(1.0 + 0x1p-8).toF32());
}

test "floats_round_f4_tie" {
    // 0.25 is halfway between f4e2m1 0 and 0.5: the even one is 0.
    try std.testing.expectEqual(@as(f32, 0.0), floats.Float4E2M1.fromF32(0.25).toF32());
}

test "floats_round_bf16_nan" {
    // A NaN with a small payload rounds into the exponent: 0x7F800001 becomes +inf.
    const nan: f32 = @bitCast(@as(u32, 0x7F800001));
    try std.testing.expect(std.math.isNan(floats.BFloat16.fromF32(nan).toF32()));
}

test "floats_round_bf16_nan_wraps" {
    // 0x7FFF8000 + 0x8000 carries out of the exponent into the sign: -0.
    const nan: f32 = @bitCast(@as(u32, 0x7FFF8000));
    try std.testing.expect(std.math.isNan(floats.BFloat16.fromF32(nan).toF32()));
}

test "floats_round_f8_nan" {
    // The generic fromF32 takes the overflow branch for NaN: e5m2 returns inf.
    try std.testing.expect(floats.Float8E5M2.fromF32(std.math.nan(f32)).isNan());
}

test "floats_round_f8_nan_to_f32" {
    // The generic toF32 only special cases zero and inf: e5m2 NaN decodes as a finite value.
    try std.testing.expect(std.math.isNan(floats.Float8E5M2.nan.toF32()));
}

test "floats_round_f8_carry" {
    // 1.97 rounds to 2.0 in f8e4m3fn: the rounding carries into the exponent.
    try std.testing.expectEqual(@as(f32, 2.0), floats.Float8E4M3FN.fromF32(1.97).toF32());
}
```

tools/zmlsmith/regressions/found/bf16_nan_ovf.zig (11 lines)

```
const std = @import("std");
const floats = @import("zml").floats;

// BFloat16.fromF32 rounds with `int += 0x8000` on the f32 bits without checking for NaN:
// negative NaNs with a high payload overflow u32, a panic in safe builds
// (zml/floats.zig:354 at 74c590b0). Found by reading the code while auditing zmlsmith.
test "bf16_nan_ovf" {
    const nan: f32 = @bitCast(@as(u32, 0xFFFF8000));
    try std.testing.expect(std.math.isNan(floats.BFloat16.fromF32(nan).toF32()));
}
```

M05 Integer Rng.uniform is biased

MEDIUM ZML wrong values · Random · risk 9 (impact 3 × exposure 3)

API	Tensor.Rng.uniform (integers)
Location	`zml/tensor.zig:792`
Platform	CPU
KnownBug	`uniform_int_bias`

What happens. With min = -4, -4 is never produced and 0 appears twice as often.

Root cause. For integer dtypes Rng.uniform samples floats in [min, max) and converts with truncation toward zero (zml/tensor.zig:792, convert). Values in (-1,1) map to 0 (twice the width of other bins), and -4 is produced only when the float is exactly -4 (probability about 2^-23 per draw), so it is effectively never produced. The bins are: -3,-2,-1 and 1,2,3 each 1/8, 0 is 2/8.

> **Risk assessment.** Silent statistical bias (dropout masks, random indices, synthetic data).

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_uniform_int --test_output=errors`.

3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   uniform_int: *fails* (matches found/uniform_int\.zig:31:)
   ```
   /home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/5133/execroot/_main/
   tools/zmlsmith/regressions/found/uniform_int.zig:31:22: 0x5dd59f30c0f4 in test.uniform_int (
   ```

Verification: confirmed, report corrected. **Expected behavior, from:** The doc comment (zml/tensor.zig:756-757) promises 'uniformly sampled floating point numbers from an interval' only; it says nothing about integer output. The reproducer's expected uniform over the 8 integers -4..3 (512 each) is the fuzzer's assumption, not a documented contract. The bias is still a real statistical defect for a function that accepts integer dtypes.

- zml/tensor.zig:756: /// Returns a Tensor of the given shape, filled with uniformly sampled floating point numbers from an interval,
- zml/tensor.zig:792: return .{ rng, floats.convert(shape_.dtype()) };

Suggested fix. Sample integers directly, or floor before converting.

tools/zmlsmith/regressions/found/uniform_int.zig (33 lines)

```
const std = @import("std");
const zml = @import("zml");

// Rng.uniform on an integer shape samples floats in [min, max) and converts them, which
// truncates toward zero: with min = -4, -4 is never produced and 0 gets [-1, 1), twice
// its share. Found by zmlsmith (rng uniform_stats).
test "uniform_int" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const n = 4096;
    const Local = struct {
        fn forward(r: zml.Tensor.Rng) zml.Tensor {
            _, const x = r.uniform(zml.Shape.init(.{n}, .i32), .{ .min = -4, .max = 4 });
            return x;
        }
    };
    var exe = try zml.module.compile(allocator, io, Local.forward, .{zml.Tensor.Rng.init()}, platform, .{});
    defer exe.deinit();
    var state = try zml.Tensor.Rng.initBuffer(io, platform, .replicated, 1234);
    defer state._state.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{state});
    defer res.deinit();
    const got = try res.toSliceAlloc(allocator, io);
    defer got.free(allocator);

    var counts: [8]usize = @splat(0);
    for (got.items(i32)) |v| counts[@intCast(v + 4)] += 1;
    // Each of -4..3 should come out about n / 8 = 512 times.
    for (counts) |c| try std.testing.expect(c > 256 and c < 1024);
}
```

M06 Quantized Linear / scaledDot don't compile on CPU

MEDIUM ZML compile · nn · risk 9 (impact 3 × exposure 3)

API nn.scaledDot, quantized nn.Linear.forward
Location zml/nn.zig:319-327 (scaledDotReference: the custom call `zml$scaled_dot_unmatched` at :320-326); zml/nn.zig:212 (ops.composite("xla.scaled_dot", ..., scaledDotReference, ...)); zml/nn.zig:114-121 (doc: "CPU has no specialized path"); zml/quantization.zig:114-126 (quantizeInput returns null on CPU)
Platform CPU

What happens. "No FFI handler registered for zml$scaled_dot_unmatched on platform Host". Every quantized Linear.forward reaches it on CPU (quantizeInput returns null there).

Root cause. nn.scaledDot always emits the composite xla.scaled_dot (zml/nn.zig:212) with scaledDotReference as its decomposition (zml/nn.zig:319-326), which is a custom call with no CPU handler. Nothing in scaledDot branches on the platform, so the decomposition is what the CPU compiler sees. The doc comment (zml/nn.zig:114-115) says 'dequant + Dot as the fallback', but the code's fallback is that custom call, not a dequantize + dot.

Risk assessment. Quantized nn.Linear cannot run on CPU at all: Linear.forward (zml/nn.zig:46-55) calls quantizeInput, which returns null on CPU for nvfp4 and fp8 (zml/quantization.zig:117-126), then forwardScaled (zml/nn.zig:64-72), which calls scaledDot unconditionally. Non-quantized Linear is unaffected.

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_scaled_dot_cpu --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:

scaled_dot_cpu: *fails* (matches `NotFound|No FFI handler registered for zml\$scaled_dot_unmatched`)
`[pjrt] (warn): [PJRT_Client_Compile] NotFound: No FFI handler registered for zml$scaled_dot_unmatched on a platform Host (canonical host)`

Verification: confirmed as reported. **Expected behavior, from:** The composite's decomposition must be a valid program on every backend; a custom call target that no backend registers is not a valid fallback. The doc's own 'fallback' sentence describes the intended behavior (dequantize + dot).

- zml/nn.zig:212: `return ops.composite("xla.scaled_dot", &.{ lhs, rhs, lhs_scale_operand, rhs_scale_operand }, &.{res_shape}, scaledDotReference, res_shape, .{`
- zml/nn.zig:320-321: `return ops.customCall("zml$scaled_dot_unmatched",`
- zml/nn.zig:121: `/// CPU has no specialized path.`
- zml/nn.zig:69: `const acc = scaledDot(lhs, weight, lhs_scale, scales, output_dtype, self.tag, .{ .rhs_scale_swizzled = q.swizzled_scales });`

Suggested fix. Emit the dequantize + dot fallback on CPU.

tools/zmlsmith/regressions/found/scaled_dot_cpu.zig (29 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// nn.scaledDot is a `xla.scaled_dot` composite whose decomposition is the custom call
// `zml$scaled_dot_unmatched`. Nothing registers it on CPU, so every scaledDot, and every
// quantized nn.Linear.forward, fails to compile there: "No FFI handler registered for
// zml$scaled_dot_unmatched on a platform Host". The doc comment says "CPU has no
// specialized path", which reads as if the dequantize + dot fallback applied.
test "scaled_dot_cpu" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(lhs: zml.Tensor, rhs: zml.Tensor, lhs_scale: zml.Tensor, rhs_scale: zml.Tensor) zml.Tensor {
            return zml.nn.scaledDot(lhs, rhs, lhs_scale, rhs_scale, .f32, .k, .{});
        }
    };

    // MXFP8: f8e4m3fn values, one e8m0 scale per 32 contracted values.
    const lhs: zml.Tensor = .fromShape(zml.Shape.init(.{ .m = 2, .k = 64 }, .f8e4m3fn));
    const rhs: zml.Tensor = .fromShape(zml.Shape.init(.{ .n = 3, .k = 64 }, .f8e4m3fn));
    const lhs_scale: zml.Tensor = .fromShape(zml.Shape.init(.{ .m = 2, .k = 2 }, .f8e8m0));
    const rhs_scale: zml.Tensor = .fromShape(zml.Shape.init(.{ .n = 3, .k = 2 }, .f8e8m0));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ lhs, rhs, lhs_scale, rhs_scale }, platform, .{});
    defer exe.deinit();
    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .m = 2, .n = 3 }, .f32), exe.output_shapes[0]);
}
```

M07 pjrt struct helpers exceed the eval branch quota on long root module names

MEDIUM ZML compile · Build · risk 9 (impact 3 × exposure 3)

API pjrt Struct / structSize
Location `pjrt/pjrt.zig:23-31 (structSize) and pjrt/pjrt.zig:33-45 (Struct); the failing site reported by the build is pjrt/pjrt.zig:42:62 inside Struct`
Platform CPU

What happens. Building a test whose root file is named `pjrt_quota_long_root_module_name.zig` fails with `evaluation exceeded 1000 backwards branches` (std/mem.zig:1487 called from pjrt/pjrt.zig:27 in structSize, and pjrt/pjrt.zig:42 in Struct). The identical file copied to `pq.zig` compiles (and then fails at run time on the unrelated cartesianProduct bug).

Root cause. structSize searches `@typeName(T)` for `.struct_` at comptime (pjrt/pjrt.zig:26-27) and Struct compares every field name (pjrt/pjrt.zig:38-42), with the default branch quota of 1000. The amount of comptime work grows with the length of the type names, which depend on the root module name: the only difference between the failing and the passing build is the file name.

> **Risk assessment.** Builds break depending on a file name: confusing for new users, and it forced zmlsmith to keep reproducer names short.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_pjrt_quota_long_root_module_name --test_output=errors`; also `cp found/pjrt_quota_long_root_module_name.zig found/pq.zig`, add nothing else, and run `bazel test //tools/zmlsmith/regressions:found_pq`: it builds.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `pjrt_quota_long_root_module_name`: *fails to build* (matches `evaluation exceeded \d+ backwards branches`)
   `external/rules_zig++zig+zig_0.16.0_x86_64-linux/lib/std/mem.zig:1487:5: error: evaluation exceeded 1000 backwards branches`

Verification: confirmed, report corrected. **Expected behavior, from:** Zig's default @setEvalBranchQuota is 1000; exceeding it is a compile error, and the fix pattern (@setEvalBranchQuota in a comptime-heavy helper) is standard. Whether the failure is intended to depend on a file name is not a contract; it is an artifact.

- pjrt/pjrt.zig:4: `const c = @import("c");` (the C types come from a generated module, not from the root file)
- pjrt/pjrt.zig:26-28: `const needle = ".struct_"; const idx = std.mem.indexOf(u8, @typeName(T), needle).?;`
- pjrt/pjrt.zig:38-45: per-field loop in Struct calling structSize(T) and std.mem.zeroes

Suggested fix. Add `@setEvalBranchQuota` in both helpers.

`tools/zmlsmith/regressions/found/pjrt_quota_long_root_module_name.zig` (45 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Compiles under a short file name (cartesian_scalar.zig), not under this one: pjrt.zig
// Struct/structSize exceed the default eval branch quota when the root module name is long.
test "pjrt branch quota with a long root module name" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor, x1: zml.Tensor) [1]zml.Tensor {
            const x2 = zml.Tensor.cartesianProduct(3, .{ x0, x1, x0 })[2];
            _ = &x2;
            return .{x2};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{}, .f64));
    const x1: zml.Tensor = .fromShape(zml.Shape.init(.{ .f = 3 }, .f64));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x0, x1 }, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ 1, 3, 1 }, .f64).withTags(.{ ._, .f, ._ }), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var b1: zml.Buffer = try .fromBytes(io, platform, x1.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b1.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ b0, b1 });
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ 1, 3, 1 }, .f64).withTags(.{ ._, .f, ._ }), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .exact_match);
    }
}
```

M08 Partitioning metadata survives ops that change the partitioned dim

MEDIUM ZML crash · Sharding · risk 9 (impact 3 × exposure 3)

API	Tensor shape propagation, Buffer.toSliceAlloc
Location	`zml/tensor.zig:2730-2738` (pad copies self's shape and only changes dims, keeping the partitioning), `zml/tensor.zig:2750` (_result), `zml/buffer.zig:433` (panic site)
Platform	CPU, 4-device mesh
KnownBug	`sharded_reduced_axis`

What happens. After pad of a partitioned axis, the result still claims the partitioning, so the buffer conversion fails with "divisible by the number of devices".

Root cause. Shape dims and partitioning are stored separately (zml/shape.zig:161 `_partitioning`). pad only updates dims (zml/tensor.zig:2738), so the partitioning of the padded axis survives. The padded axis now has 6 elements on a 4-device data axis, and the buffer code panics (zml/buffer.zig:433). Reduction was not exercised by this reproducer; only pad is confirmed.

> **Risk assessment.** Breaks sharded programs that pad or reshape a partitioned axis. The report's remark that this masks the reuseBuffer check and two XLA bugs is about the fuzzer's generator, not ZML, and should be dropped.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_shard_reduced --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

```
shard_reduced: fails (matches Buffer shape should be divisible by the number of devices along the sharded axis)
thread 3159421 panic: Buffer shape should be divisible by the number of devices along the sharded axis.
```

Verification: confirmed, report corrected. **Expected behavior, from:** A partitioned axis must have a size divisible by the device count on that axis (zml/buffer.zig:431-433 message). The padded size 6 is not divisible by 4 on the data axis of the reproducer's mesh.

- zml/tensor.zig:2738: res_shape._dims.set(i, d);
- zml/tensor.zig:2750: return _result(res_shape, pad_op.result(0));
- zml/shape.zig:161: _partitioning: PartitionArray = .empty,
- zml/buffer.zig:433: @panic("Buffer shape should be divisible by the number of devices along the sharded axis.");

Suggested fix. Drop or recompute the partitioning of changed axes.

tools/zmlsmith/regressions/found/shard_reduced.zig (53 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500225.
test "sharded_reduced_axis" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = x0.pad(false, .{ .m = zml.Tensor.Pad{ .low = 1, .high = 1, .interior = 0 } });
            _ = &x1;
            return .{x1};
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_500225", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .c = 5, .e = 6, .m = 4 }, .bool).withPartitioning(.{ .m = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .c = 5, .e = 6, .m = 6 }, .bool), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .c = 5, .e = 6, .m = 6 }, .bool), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

M09 Exe.call passes undefined pointers when arguments are missing

MEDIUM ZML memory · Runtime · risk 8 (impact 4 × exposure 2)

API Exe.call / Exe.Arguments
Location zml/exe.zig:87, :140, :173, :245-253, :306
Platform CPU

What happens. Calling a two-argument executable after setting one argument passes an undefined pointer to PJRT_LoadedExecutable_Execute, which faults. Too many arguments in set/setPartial, extra buffers in bake, and more destinations than results in Results.fill index out of bounds.

Root cause. The per-device argument table is allocated undefined and nothing checks counts; the shape assert in fill is commented out.

Risk assessment. Programming errors become segfaults inside PJRT with no message, which is expensive to debug. Exposure is limited to API misuse, but the API is low level and easy to misuse.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_exe_args_unset --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `exe_args_unset`: *fails* (matches `General protection exception|Segmentation fault`)
   `General protection exception (no address available)`

Verification: confirmed, report corrected. **Expected behavior, from:** Exe.call has no documented contract for unset arguments, and the code does not check. Calling with fewer arguments than the executable takes is API misuse, so a clear error is the reasonable expectation but not a documented one.

- zml/exe.zig:87-88: pub fn init(allocator: std.mem.Allocator, count: usize, num_devices: usize) !FlatBuffers { const raw_buffers = try allocator.alloc(*pjrt.Buffer, num_devices * count);
- zml/exe.zig:140-145: pub fn setPartial(self: *Arguments, v: anytype, offset: usize) void { ... var context: LocalContext = .{ .self = self, .current_index = offset + self.baked_count };
- zml/exe.zig:253: //stdx.debug.assert(ctx.results.expected_shapes[ctx.current_index].eql(buffer.shape()), ...
- zml/exe.zig:306-318: pub fn internalCall(...) { ... .arguments = arguments.flat_buffers.buffers, .num_args = arguments.expected_shapes.len,

Suggested fix. Track which arguments were set and check counts in `call`, `set`, `bake` and `fill`.

`tools/zmlsmith/regressions/found/exe_args_unset.zig` (30 lines)

```
const std = @import("std");
const zml = @import("zml");

// Exe.Arguments allocates its per device buffer table undefined (FlatBuffers.init,
// zml/exe.zig:87 at 74c590b0) and Exe.call never checks that every argument was set:
// calling a two argument executable after setting one passes an undefined pointer to
// PJRT_LoadedExecutable_Execute, which segfaults instead of failing with "argument 1 not
// set" (zml/exe.zig:306-316). Found by zmlsmith robust (call_too_few_args).
test "exe_args_unset" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const Local = struct {
        fn add(x: zml.Tensor, y: zml.Tensor) zml.Tensor {
            return x.add(y);
        }
    };
    const sh: zml.Shape = .init(.{4}, .f32);
    var exe = try zml.module.compile(allocator, io, Local.add, .{ zml.Tensor.fromShape(sh), zml.Tensor.fromShape(sh) }, platform, .{});
    defer exe.deinit();
    var b = try zml.Buffer.fromBytes(io, platform, sh, .replicated, std.mem.sliceAsBytes(&[_]f32{ 1, 2, 3, 4 }));
    defer b.deinit();
    var args = try exe.args(allocator);
    defer args.deinit(allocator);
    var results = try exe.results(allocator);
    defer results.deinit(allocator);
    args.set(.{b});
    exe.callOpts(io, args, &results, .{ .wait = true });
}
```

`M10` top_p = 1 still filters: the token whose f32 cumulative sum rounds above 1 is never sampled

MEDIUM ZML wrong values · Sampling · risk 6 (impact 3 × exposure 2)

API nn.sampleTokensDynamic
Location `zml/nn.zig:2114, 2117, 2123 (probs_sum, top_p, candidate test)`
Platform CPU

What happens. With `top_p = 1`, which should disable nucleus filtering, probabilities {0.3, 0.3, 0.2, 0.1, 0.1} never sample the last token (0 of 4096 draws).

Root cause. With top_p = 1 the candidate test is `probs_sum <= top_p` on the inclusive f32 cumulative sum of the softmax probabilities of the sorted candidates. Only the token at which that sum first rounds above 1.0 is removed (and any after it). In the reproducer the f32 cumsum is 0.3, 0.6, 0.8, 0.9, 1.0000001, so only the last token (prob 0.1) is removed; the earlier token with prob 0.1 is kept (474 draws). The filter is not disabled when top_p = 1.

Risk assessment. Deterministic wrong set for top_p = 1 (the default). The removed mass is the probability of the first candidate whose inclusive f32 cumulative sum exceeds 1, so it is bounded by that token's probability plus the f32 summation error. For large vocabularies with the tail spread over tiny probabilities this is typically very small; for short candidate lists (small vocab or max_top_k) it can drop a token of substantial probability, as in the reproducer (0.1 mass).

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_sample_top_p_one --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `sample_top_p_one`: *fails* (matches `probs \{ 0\.3, 0\.3, 0\.2, 0\.1, 0\.1 \} top_k 5 top_p 1 min_p 0: counts \{ \d+, \d+, \d+, \d+, 0 \}`)
   `probs { 0.3, 0.3, 0.2, 0.1, 0.1 } top_k 5 top_p 1 min_p 0: counts { 1343, 1336, 943, 474, 0 }`

Verification: confirmed, report corrected. **Expected behavior, from:** top_p = 1 must disable nucleus filtering: HF TopPLogitsWarper (see note), vLLM skips top-p when top_p == 1 (not re-checked in this run). f32 cumsum emulated in python with struct-rounding: /tmp scratch verify/f32sim.py; softmax exp assumed correctly rounded (XLA exp may differ by 1 ulp, which could move the crossing point).
Root cause checked by patching: candidates use the exclusive cumulative sum, `probs_sum.sub(probs).cmp(.LT, top_p)` (zml/nn.zig:2123). `found_sample_top_p_one` passes (3 of 3 runs); unpatched it fails 5 of 5.
- zml/nn.zig:2114: const probs_sum = probs.cumulativeSum(.topk);
- zml/nn.zig:2117: const top_p = opts.top_p.convert(x.dtype()).broad(x.shape());
- zml/nn.zig:2123: const candidate = probs_sum.cmp(.LE, top_p).logical(.AND, probs.cmp(.GE, min_p));
- zml/nn.zig:2034: top_p: f32 = 1.0, (default is 1, so the default path hits this)

Suggested fix. Skip the filter when top_p >= 1 (or use the exclusive cumulative sum, which is what HF uses, see C03). Note that the existing unit test at zml/nn.zig:2155-2158 encodes the inclusive rule and will need updating together with the C03 fix.

tools/zmlsmith/regressions/found/sample_top_p_one.zig (52 lines)

```
const std = @import("std");
const zml = @import("zml");

// nn.sampleTokensDynamic with top_p = 1, the default of DynamicSamplingStrategy.Opts,
// still drops tokens: candidates are `cumsum(probs) <= top_p` (zml/nn.zig:2114-2123 at
// 74c590b0) and the f32 cumulative sum rounds above 1 before the last tokens. With
// probabilities {0.3, 0.3, 0.2, 0.1, 0.1} the last token is never sampled in 4096 draws.
// Found by zmlsmith sampling property tests.

const rows = 4096;

const Fns = struct {
    fn sample(logits: zml.Tensor, opts: zml.nn.DynamicSamplingStrategy, rng: zml.Tensor.Rng) zml.Tensor {
        const t, _ = zml.nn.sampleTokensDynamic(logits, opts, rng);
        return t;
    }
};

/// Samples `rows` times from the same probabilities and counts each token.
fn counts(comptime voc: usize, probs: [voc]f32, top_k: u32, top_p: f32, min_p: f32) ![voc]u32 {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const logits: zml.Tensor = .init(.{ .s = rows, .voc = voc }, .f32);
    const opts: zml.nn.DynamicSamplingStrategy = .init(.f32, 0);
    var exe = try platform.compileFn(allocator, io, Fns.sample, .{ logits, opts, zml.Tensor.Rng.init() }, .{});
    defer exe.deinit();
    var data: [rows * voc]f32 = undefined;
    for (0..rows) |r| for (probs, 0..) |p, v| {
        data[r * voc + v] = @log(p);
    };
    var bl = try zml.Buffer.fromBytes(io, platform, logits.shape(), .replicated, std.mem.sliceAsBytes(&data));
    defer bl.deinit();
    var bo = try zml.nn.DynamicSamplingStrategy.makeBuffers(io, platform, .f32, .{ .top_k = top_k, .top_p = top_p, .min_p = min_p });
    defer zml.nn.DynamicSamplingStrategy.deinitBuffers(&bo);
    var br = try zml.Tensor.Rng.initBuffer(io, platform, .replicated, 1234);
    defer zml.Tensor.Rng.deinitBuffer(&br);
    var res = try zml.testing.autoCall(allocator, io, &exe, Fns.sample, .{ bl, bo, br });
    defer res.deinit();
    const s = try res.toSliceAlloc(allocator, io);
    defer s.free(allocator);
    var c: [voc]u32 = @splat(0);
    for (s.constItems(i32)) |t| c[@intCast(t)] += 1;
    std.debug.print("probs {any} top_k {d} top_p {d} min_p {d}: counts {any}\n", .{ probs, top_k, top_p, min_p, c });
    return c;
}

test "top_p = 1 keeps every token, uneven" {
    const c = try counts(5, .{ 0.3, 0.3, 0.2, 0.1, 0.1 }, 5, 1, 0);
    for (c) |x| try std.testing.expect(x > 0);
}
```

M11 Homemade tokenizer: crashes on empty input, unknown UTF-8 and id == len

MEDIUM ZML crash · Tokenizer · risk 6 (impact 3 × exposure 2) · **untrusted input**

API tokenizer homemade Tokenizer.encode / lookupPiece
Location `zml/tokenizer/homemade.zig:196 (num_tokens - 1 underflow), :197-199 and :233 (unknown token advances the offset by 1 byte, the char iterator consumed a full codepoint, see :544), :298 (id > tokens.len)`
Platform CPU

What happens. (1) An input that normalizes to nothing underflows `num_tokens - 1`. (2) An unknown multi-byte character advances the offset by 1 byte, so merges are looked up at the wrong offset ("invalid tokenization" in Debug). (3) `lookupPiece(tokens.len)` indexes out of bounds instead of returning "<oob>".

Root cause. Missing empty check, byte vs codepoint length, `id > tokens.len` instead of `>=`.

> **Risk assessment.** User text reaches it, but the homemade backend has no compiling loader today, so exposure is low.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_tok_empty --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_tok_unk_utf8 --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_tok_homemade --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `tok_empty`: *fails* (matches `integer overflow`)
   `1/1 tok_empty.test.tok_empty...thread 3165281 panic: integer overflow`
   `tok_unk_utf8`: *fails* (matches `invalid tokenization`)
   `thread 3168150 panic: invalid tokenization`
   `tok_homemade`: *fails* (matches `index out of bounds: index 4, len 4`)
   `1/1 tok_homemade.test.tok_homemade...thread 3164050 panic: index out of bounds: index 4, len 4`

Verification: confirmed, report corrected. **Expected behavior, from:** The code's own comments: lookupPiece documents that invalid ids return "<oob>" (homemade.zig:299). An input that yields no tokens must encode to an empty result (tok_empty expects ids.len == 0). An unknown codepoint must map to unk as one token (tok_unk_utf8 expects [0, 5]).

- homemade.zig:155-157: var num_tokens: usize = 0; while (try it.nextCodepointToken(self)) |token| : (num_tokens += 1) {
- homemade.zig:196: for (stable_prefix..num_tokens - 1) |i| {
- homemade.zig:197-199: if (tok_buff[i] == self.special_tokens.unk) { input_off += 1; continue; }
- homemade.zig:233: const next_tok_len = if (tok_buff[i + 1] == self.special_tokens.unk) 1 else next_tok.len;
- homemade.zig:544-545: self.input = self.input[utf8_len..]; return tokenizer.special_tokens.unk;
- homemade.zig:298-299: else if (id > self.tokens.len) "<oob>" // this means we received an invalid id, but we didn't want to panic.
- homemade.zig:300-301: else self.tokens[id];

Suggested fix. Guard the empty case, advance by the codepoint length, use `>=`.

tools/zmlsmith/regressions/found/tok_empty.zig (18 lines)

```
const std = @import("std");
const homemade = @import("zml").tokenizer.homemade;

// homemade.Tokenizer.encode on an input that normalizes to nothing: zero char tokens, and
// `num_tokens - 1` underflows in the merge loop (zml/tokenizer/homemade.zig:196 at
// 74c590b0). Found by reading the code while auditing zmlsmith coverage.
test "tok_empty" {
    var t = try homemade.Tokenizer.init(std.testing.allocator, 4, 4, null, .{ .bos = 0, .eos = 1, .unk = 2 }, true);
    defer t.deinit();
    try t.addToken(0, "<s>");
    try t.addToken(0, "</s>");
    try t.addToken(0, "a");
    try t.addToken(0, "b");
    const ids = try t.encode(std.testing.allocator, "", .{ .add_bos = false });
    defer std.testing.allocator.free(ids);
    try std.testing.expectEqual(@as(usize, 0), ids.len);
}
```

tools/zmlsmith/regressions/found/tok_unk_utf8.zig (24 lines)

```
const std = @import("std");
const homemade = @import("zml").tokenizer.homemade;

// homemade.Tokenizer.encode advances its input offset by 1 byte for an unknown token,
// but the char iterator consumed a whole UTF-8 codepoint for it (homemade.zig:198, :233
// vs :544 at 74c590b0). After an unknown multi-byte character the merge candidates are
// read at the wrong offset: in Debug builds the corruption check panics with "invalid
// tokenization", in release builds merges are looked up on the wrong bytes. Found by
// reading the code while auditing zmlsmith coverage.
test "tok_unk_utf8" {
    var t = try homemade.Tokenizer.init(std.testing.allocator, 6, 4, null, .{ .unk = 0, .bos = 1, .eos = 2 }, true);
    defer t.deinit();
    try t.addToken(0, "<unk>");
    try t.addToken(0, "<s>");
    try t.addToken(0, "</s>");
    try t.addToken(0, "a");
    try t.addToken(0, "b");
    try t.addToken(1, "ab");
    // "é" is two bytes and not in the vocabulary.
    const ids = try t.encode(std.testing.allocator, "\xc3\xa9ab", .{ .add_bos = false });
    defer std.testing.allocator.free(ids);
    try std.testing.expectEqualSlices(u32, &.{ 0, 5 }, ids);
}
```

tools/zmlsmith/regressions/found/tok_homemade.zig (17 lines)

```
const std = @import("std");
const homemade = @import("zml").tokenizer.homemade;

// homemade.Tokenizer.lookupPiece guards invalid ids with `id > tokens.len`, so
// `id == tokens.len` indexes one past the end instead of returning "<oob>"
// (zml/tokenizer/homemade.zig:298 at 74c590b0). Found by reading the code while auditing
// zmlsmith coverage.
test "tok_homemade" {
    var t = try homemade.Tokenizer.init(std.testing.allocator, 4, 4, null, .{ .bos = 0, .eos = 1, .unk = 2 }, true);
    defer t.deinit();
    try t.addToken(0, "<s>");
    try t.addToken(0, "</s>");
    try t.addToken(0, "a");
    try t.addToken(0, "b");
    try std.testing.expectEqualStrings("<oob>", t.lookupPiece(t.tokens.len));
}
```

M12 Homemade streaming decode drops text

MEDIUM ZML wrong values · Tokenizer · risk 6 (impact 3 × exposure 2)

API tokenizer homemade Decoder.feedOne
Location `zml/tokenizer/homemade.zig:479-499 (Decoder.next; window kept in last_token_ids, capacity 4 at :439, slide at :480-483)`
Platform CPU

What happens. Ids {5, 7, 4, 6, 3, 3, 3, 3, 3} stream "abéb aaaa" where batch decoding gives "abéb aaaaa".

Root cause. Same windowed-suffix design as the sentencepiece streamer.

Risk assessment. Same failure as the sentencepiece streamer, on a less used backend.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_tok_hm_stream --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `tok_hm_stream: fails (matches streamed \"ab.b aaaa\", batch \"ab.b aaaaa\")`
   `1/1 tok_hm_stream.test.tok_hm_stream...ids { 5, 7, 4, 6, 3, 3, 3, 3, 3 }: streamed "abéb aaaa", batch "abéb aaaaa"`

Verification: confirmed, report corrected. **Expected behavior, from:** Streaming contract as for H01: the concatenated feedOne outputs plus finalize must equal the batch decode (decodeAlloc, homemade.zig:465-474 via Decoder.decode).

- homemade.zig:439: const TokensIdsBuffer = stdx.BoundedArray(u32, 4);
- homemade.zig:480-484: if (self.last_token_ids.len >= self.last_token_ids.capacity()) { _ = self.last_token_ids.orderedRemove(0); } self.last_token_ids.appendAssumeCapacity(token_id); const new_string = try self.decode(self.last_token_ids.constSlice());
- homemade.zig:493-496: if (std.mem.startsWith(u8, new_string, self.last_string.constSlice()[start..])) { const chunk = new_string[self.last_string.len - start ..];

Suggested fix. Same fix as the sentencepiece streamer.

tools/zmlsmith/regressions/found/tok_hm_stream.zig (40 lines)

```
const std = @import("std");
const zml = @import("zml");
const homemade = zml.tokenizer.homemade;

// Streaming decode with the homemade backend (Decoder.feedOne then finalize) against a
// batch decode of the same ids with a fresh decoder, on random id sequences: like the
// sentencepiece one (spm_stream.zig) it decodes a window of recent ids and emits the
// difference, and repeated tokens get dropped: ids {5, 7, 4, 6, 3, 3, 3, 3, 3} stream
// "abéb aaaa" where the batch decode is "abéb aaaaa" (zml/tokenizer/homemade.zig:479 at
// 74c590b0). Found by zmlsmith tokenizer property tests.
test "tok_hm_stream" {
    const allocator = std.testing.allocator;
    var t = try homemade.Tokenizer.init(allocator, 9, 8, null, .{ .bos = 0, .eos = 1, .unk = 2 }, true);
    defer t.deinit();
    for ([_][]const u8{ "<s>", "</s>", "<unk>", "a", "b", "ab", " ", "\xc3\xa9", "aaa" }) |p| try t.addToken(0, p);
    var tok: zml.tokenizer.Tokenizer = .{ .homemade = &t };
    var prng: std.Random.DefaultPrng = .init(9);
    const rng = prng.random();
    for (0..300) |_| {
        var ids: [12]u32 = undefined;
        const n = rng.intRangeAtMost(usize, 1, ids.len);
        for (ids[0..n]) |*id| id.* = rng.intRangeAtMost(u32, 3, 8);
        var batch_dec = try tok.decoder();
        defer batch_dec.deinit();
        var want = try batch_dec.decodeAlloc(allocator, ids[0..n]);
        defer want.deinit(allocator);
        var dec = try tok.decoder();
        defer dec.deinit();
        var got: std.ArrayList(u8) = .empty;
        defer got.deinit(allocator);
        var buf: [256]u8 = undefined;
```

```
        for (ids[0..n]) |id| try got.appendSlice(allocator, try dec.feedOne(id, &buf));
        try got.appendSlice(allocator, try dec.finalize(&buf));
        if (!std.mem.eql(u8, got.items, want.items)) {
            std.debug.print("ids {any}: streamed \"{s}\", batch \"{s}\"\n", .{ ids[0..n], got.items, want.items });
            return error.TestUnexpectedResult;
        }
    }
}
```

M13 Tokenizer.fromBytes indexes an empty file and misroutes JSON with leading whitespace

MEDIUM ZML crash · Tokenizer · risk 6 (impact 3 × exposure 2) · **untrusted input**

API tokenizer.Tokenizer.fromBytes
Location `zml/tokenizer/tokenizer.zig:143`
Platform CPU

What happens. An empty tokenizer file indexes out of bounds. JSON with leading whitespace or a BOM is sent to the sentencepiece protobuf parser.

Root cause. The backend is picked from `bytes[0]` without a length check or whitespace skip.

> **Risk assessment.** Crash on an empty or truncated download; confusing parse errors for valid JSON.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_tok_from_bytes --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `tok_from_bytes`: *fails* (matches `index out of bounds: index 0, len 0`)
   `1/1 tok_from_bytes.test.tok_from_bytes...thread 3165934 panic: index out of bounds: index 0, len 0`

Verification: confirmed as reported. **Expected behavior, from:** Public API contract: fromBytes returns an error on bad input (it is an error union). An empty file should give an error, not a panic.

- zml/tokenizer/tokenizer.zig:142-146: pub fn fromBytes(allocator, bytes) !Tokenizer { if (bytes[0] == '{') { return .{ .iree = … }; } return .{ .sentencepiece = try .fromBytes(bytes) }; }
- examples/llm/main.zig:189-197: reads tokenizer.json from the model directory and calls `.fromBytes(allocator, bytes)` (untrusted file from the model repo)

Suggested fix. Check the length, skip whitespace and BOM before sniffing.

`tools/zmlsmith/regressions/found/tok_from_bytes.zig` (16 lines)

```
const std = @import("std");
const zml = @import("zml");

// zml.tokenizer.Tokenizer.fromBytes picks the backend from `bytes[0]` without checking
// the length: an empty tokenizer file indexes out of bounds instead of returning an
// error (zml/tokenizer/tokenizer.zig:143 at 74c590b0). JSON with leading whitespace or a
// BOM is also routed to the sentencepiece protobuf parser. Found by reading the code while
// auditing zmlsmith.
test "tok_from_bytes" {
    if (zml.tokenizer.Tokenizer.fromBytes(std.testing.allocator, "")) |t| {
        var tok = t;
        tok.deinit();
        return error.TestUnexpectedResult;
    } else |_| {}
}
```

M14 Homemade Normalizer.fromHfJson overflows its 8-byte whitespace buffer

MEDIUM ZML crash · Tokenizer · risk 6 (impact 3 × exposure 2) · **untrusted input**

API tokenizer homemade Normalizer.fromHfJson
Location `zml/tokenizer/homemade.zig:594 (_whitespace is stdx.BoundedArray(u8, 8)), :802-805 (appendSliceAssumeCapacity at :803)`
Platform CPU

What happens. A Replace " " step with a longer replacement, or several such steps, overflow the buffer instead of returning InvalidNormalizerJson.

Root cause. `appendSliceAssumeCapacity` into a fixed 8-byte `_whitespace` buffer.

Risk assessment. Untrusted JSON, safety panic in Debug, undefined behavior in ReleaseFast; low exposure (no compiling loader).

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_tok_normalizer_ws --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `tok_normalizer_ws`: *fails* (matches `index out of bounds: index 16, len 8`)
   `1/1 tok_normalizer_ws.test.tok_normalizer_ws...thread 3170053 panic: index out of bounds: index 16, len 8`

Verification: confirmed, report corrected. **Expected behavior, from:** stdx BoundedArray contract (stdx/bounded_array.zig:257-261): appendSliceAssumeCapacity asserts capacity; a caller reading untrusted JSON must check it and return error.InvalidNormalizerJson, as the function's error set declares.

- homemade.zig:594: _whitespace: stdx.BoundedArray(u8, 8) = .empty,
- homemade.zig:802-805: if (std.mem.eql(u8, str_pattern, " ")) { normalizer._whitespace.appendSliceAssumeCapacity(objectGet(step, .string, "content") orelse return error.InvalidNormalizerJson,);
- stdx/bounded_array.zig:257-261: pub fn appendSliceAssumeCapacity(self, items) void { const old_len = self.len; self.len += items.len; @memcpy(self.slice()[old_len..][0..items.len], items); }
- homemade.zig:1088: try Normalizer.fromHfJson(normalizer_config) (only caller is the dead homemade.Tokenizer.fromHfJson at :1056)

Suggested fix. Check capacity and return the error.

tools/zmlsmith/regressions/found/tok_normalizer_ws.zig (17 lines)

```
const std = @import("std");
const homemade = @import("zml").tokenizer.homemade;

// homemade.Normalizer.fromHfJson appends the "content" of every Replace " " step to the
// fixed capacity _whitespace buffer with appendSliceAssumeCapacity, so a replacement
// longer than the buffer, or several Replace steps, overflow it instead of returning
// error.InvalidNormalizerJson (zml/tokenizer/homemade.zig:802-804 at 74c590b0). Found by
// reading the code while auditing zmlsmith, reached by zmlsmith parse-fuzz.
test "tok_normalizer_ws" {
    const json =
        \\{"type":"Replace","pattern":{"String":" "},"content":"0123456789abcdef"}
    ;
    const parsed = try std.json.parseFromSlice(std.json.Value, std.testing.allocator, json, .{});
    defer parsed.deinit();
    _ = homemade.Normalizer.fromHfJson(parsed.value.object) catch {};
}
```

M15 RoPE yarn config without factor panics

MEDIUM ZML crash · nn · risk 6 (impact 3 × exposure 2) · **untrusted input**

API RopeOpts.Scaling.jsonParse (yarn)
Location `zml/nn.zig:491 (optional field), :560, :562 (attentionScaling unwraps, only when attention_factor is null), :773 (yarn inverse frequencies, always unwrapped)`
Platform CPU

What happens. The config parses (factor is optional), then attentionScaling and the yarn frequencies unwrap `factor.?` and panic.

Root cause. Yarn.factor is `?f32` (zml/nn.zig:491). jsonParseFromValue accepts a yarn object without factor (zml/nn.zig:537). attentionScaling unwraps factor.? at :560/:562 when attention_factor is absent, and the yarn invFreq branch unwraps s.factor.? at :773 unconditionally, so a yarn config without factor panics.

Risk assessment. Untrusted model config crashes the process instead of failing to parse.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_rope_yarn --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `rope_yarn`: *fails* (matches `attempt to use null value`)
   `1/1 rope_yarn.test.rope_yarn...thread 3175921 panic: attempt to use null value`

Verification: confirmed, report corrected. **Expected behavior, from:** HF transformers src/transformers/modeling_rope_utils.py, _compute_yarn_parameters: `factor = rope_parameters_dict["factor"]` (direct indexing, no default). A yarn config without factor is therefore not valid for HF, so rejecting it at parse time matches the reference.
- zml/nn.zig:491: factor: ?f32 = null,
- zml/nn.zig:537: return .{ .yarn = try std.json.parseFromValueLeaky(Yarn, stdx.noalloc, source, .{ .ignore_unknown_fields = true }) };
- zml/nn.zig:773: const downscaling = 1.0 / s.factor.?;

Suggested fix. Make factor required for yarn in jsonParse, or default it.

tools/zmlsmith/regressions/found/rope_yarn.zig (16 lines)

```
const std = @import("std");
const zml = @import("zml");

// RopeOpts.Scaling.jsonParse accepts a yarn config without `factor` (an optional field),
// then attentionScaling and the yarn inverse frequencies unwrap `factor.?` and panic
// (zml/nn.zig:491,560,562 at 74c590b0). Found by reading the code while auditing zmlsmith.
test "rope_yarn" {
    const json =
        \\{"rope_type": "yarn", "original_max_position_embeddings": 4096}
    ;
    // Rejecting the config at parse time is a valid fix (HF reads `factor` unconditionally).
    const parsed = std.json.parseFromSlice(zml.nn.RopeOpts.Scaling, std.testing.allocator, json, .{}) catch return;
    defer parsed.deinit();
    try std.testing.expect(std.math.isFinite(parsed.value.attentionScaling()));
}
```

M16 Tensor.print aborts for sub-byte, e8m0, e3m4, e4m3 and f4 dtypes

MEDIUM ZML crash · Tensor · risk 6 (impact 3 × exposure 2)

API Tensor.print (zml$print FFI callback)
Location `pjrt/ffi.zig:230-256 (DataType enum: no S1, S2, S4, U1, U2, U4, F8E8M0FNU entries; the C header has them at xla/ffi/api/c_api.h:189-220), zml/platform.zig:928-951 (dataTypeFromFfiDataType; f8e3m4, f8e4m3 and f4e2m1 hit 'else => unreachable' at zml/platform.zig:950).`
Platform CPU

What happens. Printing a tensor of an unlisted sub-byte or e8m0 dtype (for example i4) aborts the process with 'switch on corrupt value' in the XLA runtime thread. Printing f8e3m4, f8e4m3 or f4e2m1 reaches unreachable in dataTypeFromFfiDataType. The reproducer exercises only i4.

Root cause. The callback maps the buffer dtype with an exhaustive enum missing S4/U4/S2/U2/F8E8M0FNU, and a switch sends f8e3m4, f8e4m3 and f4e2m1 to unreachable.

> **Risk assessment.** A debugging aid kills the process exactly when debugging quantized models.

Reproduction.
1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_print_dtypes --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `print_dtypes:` *fails* (matches `panic: switch on corrupt value`)
   `thread 3156286 panic: switch on corrupt value`

Verification: confirmed, report corrected. **Expected behavior, from:** Tensor.print is a debugging aid and should not abort for a valid dtype; the expected behaviour follows the ZML enum (the buffer's dtype must map to a ZML dtype or be printed as raw bytes, as the finding proposes). The reproducer asserts the printed bytes match the input bytes.
- pjrt/ffi.zig:230: 'pub const DataType = enum(c.XLA_FFI_DataType) {'
- zml/platform.zig:950: '   else => unreachable,'
- xla/ffi/api/c_api.h:189-220: XLA_FFI_DataType_S4 = 21, XLA_FFI_DataType_U4 = 22, XLA_FFI_DataType_F8E8M0FNU = 33, etc.
- tools/zmlsmith/regressions/found/print_dtypes.zig: 'try run(.i4, .{4});'

Suggested fix. Complete the enum (non-exhaustive) and print unsupported dtypes as raw bytes.

tools/zmlsmith/regressions/found/print_dtypes.zig (39 lines)

```
const std = @import("std");
const zml = @import("zml");

// Tensor.print lowers to the zml$print FFI callback, which maps the buffer dtype with an
// exhaustive enum and a switch (pjrt/ffi.zig:230-256, zml/platform.zig:927-950 at
// 74c590b0). The enum has no S4/U4/S2/U2/F8E8M0FNU entries ("switch on corrupt value"
// in the XLA runtime thread) and the switch sends f8e3m4, f8e4m3 and f4e2m1 to
```

```zig
// unreachable: printing a tensor of any of these dtypes aborts the process. Found by
// zmlsmith runtime coverage of Tensor.print.

fn run(comptime dt: zml.DataType, comptime dims: anytype) !void {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const Local = struct {
        fn f(x: zml.Tensor) zml.Tensor {
            x.print("x");
            return x;
        }
    };
    const x: zml.Tensor = .init(dims, dt);
    var exe = try platform.compileFn(allocator, io, Local.f, .{x}, .{});
    defer exe.deinit();
    const bytes = try allocator.alloc(u8, x.shape().byteSize());
    defer allocator.free(bytes);
    for (bytes, 0..) |*b, i| b.* = @truncate(i *% 37 % 3);
    var b = try zml.Buffer.fromBytes(io, platform, x.shape(), .replicated, bytes);
    defer b.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.f, .{b});
    defer res.deinit();
    const got = try res.toSliceAlloc(allocator, io);
    defer got.free(allocator);
    try std.testing.expectEqualSlices(u8, bytes, got.constData());
}

test "print_dtypes" {
    try run(.i4, .{4});
}
```

M17 Sub-byte Shape byte strides count whole bytes per element

MEDIUM ZML crash · Host · risk 6 (impact 3 × exposure 2)

API Shape.computeByteStrides / Slice (sub-byte dtypes)
Location `zml/shape.zig:1219-1221 (computeByteStrides uses dtype().sizeOf(), which is @sizeOf of the Zig type, 1 byte for i4); zml/slice.zig:113-123 (subSlice offsets by these strides); zml/slice.zig:324 and :331 (row loops in prettyPrintIndented)`
Platform CPU

What happens. An i4 {2, 4} slice (4 bytes total, 2 bytes per row) gets byte stride {4, 1}. subSlice along axis 0 moves 4 bytes instead of 2. Printing a rank-2 sub-byte slice reads beyond the buffer and panics (the row slice at zml/slice.zig:331 runs past slice.constData()'s length).

Root cause. Shape.computeByteStrides calls computeStrides(self.dtype().sizeOf()) (zml/shape.zig:1220). DataType.sizeOf is `@sizeOf(tag.toZigType())` (zml/dtype.zig:159-163), which is 1 for i4/f4e2m1, while the real row size is bitSizeOf-based (zml/shape.zig:502-504 byteSize uses bitSizeOf). So the stride of a sub-byte dim is counted in whole bytes per element: i4 {2,4} gets {4,1} instead of {2,1}. Slice.subSlice (zml/slice.zig:119) and the row loops (zml/slice.zig:324, :331) then use the wrong offsets.

> **Risk assessment.** Host-side views and prints of packed int4 / fp4 tensors use wrong offsets. Reads through subSlice return the wrong data; the print path panics.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_slice_subbyte --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `slice_subbyte`: *fails* (matches `expected 2, found 4`)
   `1/1 slice_subbyte.test.slice_subbyte...expected 2, found 4`

Verification: confirmed, report corrected. **Expected behavior, from:** Packed sub-byte layout: a row of 4 i4 values is 16 bits, i.e. 2 bytes (Shape.byteSize, zml/shape.zig:502-504, divCeil of count * bitSizeOf / 8 is the layout contract that Slice.init asserts at zml/slice.zig:29).

- zml/shape.zig:1220: `return self.computeStrides(self.dtype().sizeOf());`
- zml/dtype.zig:159-161: `pub fn sizeOf(self: DataType) u16 { return switch (self) { inline else => |tag| @sizeOf(tag.toZigType()),`
- zml/shape.zig:502-504: `return std.math.divCeil(usize, self.count() * self.dtype().bitSizeOf(), 8) catch unreachable;`
- zml/slice.zig:29: `stdx.debug.assert(bytes.len == shape.byteSize(), ...)`

Suggested fix. Compute strides in bits, or reject sub-byte strided views.

`tools/zmlsmith/regressions/found/slice_subbyte.zig` (17 lines)

```zig
const std = @import("std");
const zml = @import("zml");
```

```
// An i4 {2, 4} tensor is 8 nibbles, 4 bytes, and each row is 4 nibbles = 2 bytes, so the
// byte stride of dim 0 must be 2. Shape.computeByteStrides builds strides from
// dtype().sizeOf(), which is 1 byte for i4 (zml/shape.zig:1219-1220 at 74c590b0), so it
// returns 4 and subSlice moves to the wrong offset.
test "slice_subbyte" {
    const sh = zml.Shape.init(.{ 2, 4 }, .i4);
    try std.testing.expectEqual(@as(usize, 4), sh.byteSize());
    try std.testing.expectEqual(@as(i64, 2), sh.computeByteStrides().get(0));

    var bytes: [4]u8 = .{ 0x21, 0x43, 0x65, 0x87 };
    const s: zml.Slice = .init(sh, &bytes);
    try std.testing.expectEqual(@as(usize, 2), s.subSlice(0, 1, 1).offset_bytes);
}
```

M18 slices() with mixed static and dynamic slices produces invalid MLIR

MEDIUM ZML crash · Tensor · risk 6 (impact 3 × exposure 2)

API	Tensor.slices
Location	zml/tensor.zig:2049-2052 (dynamic path taken for all axes), zml/tensor.zig:2061 (size taken from s.len only)
Platform	CPU
KnownBug	slices_mixed_static

What happens. "slice size 9223372036854775807": the doc comment's first example mixes them this way.

Root cause. Tensor.slices takes the dynamic path for every axis as soon as one slice has dyn_start (zml/tensor.zig:2047-2052). The new size is then `s.len` (zml/tensor.zig:2061). For a static `.{ .start, .end }` slice `.len` is to_the_end (Slice fields, zml/tensor.zig:1954), so the dimension becomes maxInt and the static `.end` is ignored. The dynamic path is also the only one that reads `s.start` for the offset (zml/tensor.zig:2065).

> **Risk assessment.** Any call that mixes a dynamic slice with a static `.{ .start, .end }` slice on another axis fails at trace time with an MLIR error. The doc example at zml/tensor.zig:2032 has this shape, but it is not valid Zig as written. The example at zml/tensor.zig:2037 mixes `.dyn` with `.{ .start, .len }`, which does not hit the bug.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_slices_mixed --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `slices_mixed`: *fails* (matches `slice size 9223372036854775807|Failed to create MLIR operation`)
   `1/1 slices_mixed.test.slices_mixed...error: has slice size 9223372036854775807 greater than dimension size 4 in dimension 0 of operand`

Verification: confirmed, report corrected. **Expected behavior, from:** Slice.len/.end semantics (zml/tensor.zig:1951-1957) and the slices() doc example (zml/tensor.zig:2032). Note that the dynamic path reads only s.len, while the static path (staticSlices, zml/tensor.zig:2091) uses absolute(), which respects .end.

- zml/tensor.zig:2061: new_shape._dims.buffer[a] = s.len;
- zml/tensor.zig:2091: const args: Slice = s.absolute(self.dim(a));
- zml/tensor.zig:1954: len: i64 = to_the_end,
- zml/tensor.zig:2032: /// Tensor(.{.a=20,.b=30,.c=40 }).slices(.{ .a, .b }, &.{ .start = 5, .end = 11}, .dyn(x) });

Suggested fix. Compute the static length from start/end.

tools/zmlsmith/regressions/found/slices_mixed.zig (23 lines)

```
const std = @import("std");
const zml = @import("zml");

// Tensor.slices with one dynamic slice takes the dynamic_slice path for every axis and sets
// each new dim to `s.len`, which is `to_the_end` for a static `.{ .start, .end }` slice.
// The doc comment's first example mixes the two this way.
test "slices_mixed" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x: zml.Tensor, off: zml.Tensor) zml.Tensor {
            return x.slices(.{ .a, .b }, &.{ .{ .start = 1, .end = 3 }, .{ .dyn_start = off, .len = 2 } });
        }
    };
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .a = 4, .b = 5 }, .f32));
    const off: zml.Tensor = .fromShape(zml.Shape.init(.{}, .i32));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x, off }, platform, .{});
    defer exe.deinit();
    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .a = 2, .b = 2 }, .f32), exe.output_shapes[0]);
}
```

M19 Sub-byte tensors: empty, odd last dim, rank 0 and rank 8 panic

MEDIUM ZML crash · Sub-byte · risk 6 (impact 3 × exposure 2)

API Compiler.repack, Shape.packedShape
Location zml/Compiler.zig:731 (repack splitAxis with { -1, items_per_byte }; not :746, which is the bitcast return); zml/shape.zig:1327 (self.axis(-1) on rank 0), zml/shape.zig:1330 (replaceRange ... catch unreachable, rank 8 overflow), zml/shape.zig:1630-1635 (packedShape divExact of the last dim)
Platform CPU
KnownBug subbyte_empty, subbyte_last_dim, subbyte_max_rank

What happens. Sub-byte (bits < 8) arguments or results panic on: size-0 tensors (`Can't split { 2, 0 } along axis 1 into { 2, -1, 4 }: TooBig`); a last dim that is not a multiple of 8/bitSizeOf (e.g. i4 with odd last dim, i2 with last dim 1..3) (`exact division produced remainder`); rank 0 (`Tensor {f4e2m1} doesn't have dimension: -1`); rank 8 (`attempt to unwrap error: Overflow`, because the split adds an axis past MAX_RANK).

Root cause. Compiler.repack (zml/Compiler.zig:728-748) reshapes to the last dim split as `.{ -1, items_per_byte }` (Compiler.zig:731). For a zero-element tensor the -1 cannot be inferred (zml/shape.zig:594 path). For rank 0 there is no last axis (shape.zig:1327). For rank 8 the extra axis exceeds MAX_RANK (shape.zig:1330). packedShape (shape.zig:1634-1635) divides the last dim exactly by items_per_byte.

> **Risk assessment.** Crashes compiling valid int2/int4/fp4 graphs that have empty, scalar, or high-rank sub-byte tensors. Odd last dims are a weaker case: Shape.byteSize rounds up (shape.zig:502-504), so a host-side odd shape is valid; the PJRT packed shape is the part that cannot represent it.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_subbyte_empty --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_subbyte_last --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_subbyte_rank0 --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_subbyte_rank8 --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `subbyte_empty`: *fails* (matches `Can't split \{ 2, 0 \} along axis 1 into \{ 2, -1, 4 \}: TooBig`)
   `1/1 subbyte_empty.test.subbyte_empty...thread 3165993 panic: Can't split { 2, 0 } along axis 1 into { 2, -1, 4 }: TooBig`
   `subbyte_last`: *fails* (matches `exact division produced remainder`)
   `1/1 subbyte_last.test.subbyte_last_dim...thread 3164642 panic: exact division produced remainder`
   `subbyte_rank0`: *fails* (matches `doesn't have dimension: -1`)
   `1/1 subbyte_rank0.test.subbyte_rank0...thread 3164822 panic: Tensor {f4e2m1} doesn't have dimension: -1`
   `subbyte_rank8`: *fails* (matches `attempt to unwrap error: Overflow`)
   `1/1 subbyte_rank8.test.subbyte_rank8...thread 3163852 panic: attempt to unwrap error: Overflow`

Verification: confirmed, report corrected. **Expected behavior, from:** Host layout contract: Shape.byteSize (zml/shape.zig:502-504) rounds bits up to bytes, so zero-size, scalar and odd-length sub-byte shapes have well-defined storage. Zero-size and scalar tensors are valid: Slice copy already returns early for total_bytes == 0 (zml/slice.zig:342-345), so the host side handles them.

- zml/Compiler.zig:730-731: `const elem_per_bytes = @divExact(8, og_shape.dtype().bitSizeOf()); / const intermediary_type = ... og_shape.splitAxis(-1, .{ -1, elem_per_bytes }));`
- zml/shape.zig:1634-1635: `const items_per_byte = @divExact(8, bit_size); / return sh.setDim(-1, @divExact(sh.dim(-1), items_per_byte));`
- zml/shape.zig:1330: `new_shape._dims.replaceRange(ax, 1, dims_.constSlice()) catch unreachable;`

Suggested fix. Pack over the flat byte count; reject odd dims with an error.

tools/zmlsmith/regressions/found/subbyte_empty.zig (24 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Compiler.repack splits the last dim of sub-byte arguments and results into
// `.{ -1, items_per_byte }`, which can't be inferred when the tensor has no elements:
// "Can't split { 2, 0 } along axis 1 into { 2, -1, 4 }: TooBig". Found by zmlsmith from
// seed 900133.
test "subbyte_empty" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x: zml.Tensor) zml.Tensor {
            return x;
        }
    };

    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .a = 2, .b = 0 }, .i2));
```

```
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x}, platform, .{});
    defer exe.deinit();
    try zml.testing.expectEqualShapes(x.shape(), exe.output_shapes[0]);
}
```

tools/zmlsmith/regressions/found/subbyte_last.zig (35 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500713.
test "subbyte_last_dim" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            return .{x0};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ 0, 1, 1 }, .i2).withTags(.{ ._, .g, .h }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ 0, 1, 1 }, .i2).withTags(.{ ._, .g, .h }), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{});
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{};
        const expected: zml.Slice = .init(zml.Shape.init(.{ 0, 1, 1 }, .i2).withTags(.{ ._, .g, .h }), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

tools/zmlsmith/regressions/found/subbyte_rank0.zig (39 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500923.
test "subbyte_rank0" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            return .{x0};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{}, .f4e2m1));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{}, .f4e2m1), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{}, .f4e2m1), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

tools/zmlsmith/regressions/found/subbyte_rank8.zig (19 lines)

```
const std = @import("std");
const zml = @import("zml");

// Compiler.repack packs a sub-byte result by reshaping it with its last axis split in
// two (zml/Compiler.zig:746 at 74c590b0): for a rank 8 result, Shape.MAX_RANK, the split
// shape has 9 axes and Shape.splitAxis panics with "attempt to unwrap error: Overflow"
// (zml/shape.zig:1330). Found by zmlsmith (seed 91633).
test "subbyte_rank8" {
    const platform = zml.testing.env();
    const Local = struct {
        fn forward(x: zml.Tensor) zml.Tensor {
            return x;
        }
    };
    const x: zml.Tensor = .init(.{ 1, 1, 1, 1, 1, 1, 1, 2 }, .f4e2m1);
```

```
    var exe = try zml.module.compile(std.testing.allocator, std.testing.io, Local.forward, .{x}, platform, .{});
    exe.deinit();
}
```

M20 rope assumes .hd is the last axis

MEDIUM ZML crash · nn · risk 6 (impact 3 × exposure 2)

API nn.rope
Location zml/nn.zig:604-605 (rope uses .hd for head_dim), :620 (inv_freq), :635-640 (splitRealImg uses n = x.dim(-1)), :632 (mergeRealImgPass)
Platform CPU
KnownBug rope_hd_not_last

What happens. Exact-division panic, "Can't merge", or wrong values when .hd isn't last.

Root cause. splitRealImg takes the split size from the last axis (n = x.dim(-1), zml/nn.zig:636) while rope asserts only that .hd exists (zml/nn.zig:609-612) and uses x.dim(.hd) for the head dim (zml/nn.zig:605). When .hd is not the last axis the exact division and the merge reshape see the wrong axis, so they panic or mix axes.

> **Risk assessment.** Crash or silent wrong values depending on the shape.

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run bazel test //tools/zmlsmith/regressions:found_rope_hd_last --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   rope_hd_last: *fails* (matches exact division produced remainder)
   1/1 rope_hd_last.test.rope_hd_not_last...thread 3160729 panic: exact division produced remainder

Verification: confirmed, report corrected. **Expected behavior, from:** rope doc comment (zml/nn.zig:600-601): 'x: .{ .s, .hd }'. The doc does not require .hd last, but the assertion at :609 only checks presence, so the layout assumption is undocumented.

- zml/nn.zig:636: const n = x.dim(-1);
- zml/nn.zig:605: const head_dim = x.dim(.hd);
- zml/nn.zig:601: /// - x: .{ .s, .hd } where .s is the sequence length and .hd the head dimension

Suggested fix. Transpose .hd last internally, or assert it with a message.

tools/zmlsmith/regressions/found/rope_hd_last.zig (56 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500176.
test "rope_hd_not_last" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = zml.nn.rope(x0, null, .{ .layout = .real_pass_im_pass, .scaling = .{ .llama3 = .{ .factor = 32, .high_freq_factor = 4, .low_freq_factor =
1, .original_max_position_embeddings = 8, .rope_theta = 100 } } });
            _ = &x1;
            return .{x1};
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_500176", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .hd = 12, .s = 3, .h = 3 }, .f16).withPartitioning(.{ .hd = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .hd = 12, .s = 3, .h = 3 }, .f16), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
```

```
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .hd = 12, .s = 3, .h = 3 }, .f16), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .{ .absolute_tolerance = 4.8828125e-2, .relative_tolerance = 1.953125e-3, .minimum_close_fraction =
1 });
    }
}
```

M21 XLA compiler aborts and verifier errors

MEDIUM upstream XLA compile · XLA · risk 6 (impact 3 × exposure 2)

API	various
Location	XLA HLO verifier and SPMD partitioner (xla_spmd_*: spmd_partitioner; gspmd_*: GSPMD path only). ZML side: zml/Sharding.zig:50-54 (partitioner default) and zml/Compiler.zig:777-780.
Platform	CPU, 4-device and 2-level meshes
KnownBug	xla_bitcast_chain, xla_spmd_small_indices, xla_spmd_bitwise_reduce, xla_spmd_interior_pad, gspmd_pool_manual, gspmd_mesh2d_manual

What happens. Chained narrowing bitcasts ("expected u8[2,4], actual u8[2,2,2]"); SPMD gather/scatter with u8/u16 indices ("Unsupported indices type"); strided slice of a partitioned reshape; SPMD bitwise or/and reduction ("Unsupported reduction computation"); SPMD interior pad with a negative edge (CHECK failure, spmd_partitioner.cc:1120); GSPMD strided maxPool1d into manualComputation (CHECK, spmd_partitioner.cc:890); GSPMD on a 2-level mesh with causalAttnMask next to manualComputation (CHECK, hlo_sharding.cc:1530).

Root cause. Upstream XLA. The CHECK failures abort the whole process.

> **Risk assessment.** A CHECK failure aborts the process at compile time. The gspmd_* items need an explicit .partitioner = .gspmd; on CPU, Sharding.fromTarget selects .shardy by default (zml/Sharding.zig:50-54), so those two are opt-in only. The other items use the default partitioner path, which runs XLA's SPMD partitioner.

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run bazel test //tools/zmlsmith/regressions:found_xla_bitcast --test_output=errors, then bazel test //tools/zmlsmith/regressions:found_xla_spmd_idx --test_output=errors, then bazel test //tools/zmlsmith/regressions:found_xla_spmd_slice --test_output=errors, then bazel test //tools/zmlsmith/regressions:found_xla_spmd_or --test_output=errors, then bazel test //tools/zmlsmith/regressions:found_xla_spmd_pad --test_output=errors, then bazel test //tools/zmlsmith/regressions:found_gspmd_pool_manual --test_output=errors, then bazel test //tools/zmlsmith/regressions:found_gspmd_mask_manual --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:

```
xla_bitcast: fails (matches Expected instruction to have shape equal to u8\[2,4\])
[pjrt] (warn): [PJRT_Client_Compile] Internal: during context [hlo verifier]: Expected instruction to have shape equal to u8[2,4],
actual shape is u8[2,2,2]:
xla_spmd_idx: fails (matches Unsupported indices type: U16)
F0000 00:00:1792976314.038920 3169981 gather_scatter_handler.cc:244] Unsupported indices type: U16
xla_spmd_slice: fails (matches Expected instruction to have shape equal to f32\[1,1,2,1\])
[pjrt] (warn): [PJRT_Client_Compile] Internal: during context [hlo verifier]: Expected instruction to have shape equal to f32[1,1,2,1],
actual shape is f32[1,1,1,1]:
xla_spmd_or: fails (matches Unimplemented: Unsupported reduction computation)
[pjrt] (warn): [PJRT_Client_Compile] Unimplemented: Unsupported reduction computation: %region_0.0.clone (reduce.0: s16[], reduce.4:
s16[]) -> s16[] {
xla_spmd_pad: fails (matches get_first_valid_element_offset_on_dilated_shard)
F0000 00:00:1792976433.673956 3172483 spmd_partitioner.cc:1120] Check failed: get_first_valid_element_offset_on_dilated_shard(0) ==
swd->padding_low() (2 vs. -1)
gspmd_pool_manual: fails (matches !target\.IsReplicatedOrSingleDevice\(\))
F0000 00:00:1792975502.587807 3153546 spmd_partitioner.cc:890] Check failed: !target.IsReplicatedOrSingleDevice()
gspmd_mask_manual: fails (matches !IsManualLeaf\(\) && !IsUnknownLeaf\(\))
F0000 00:00:1792975413.377551 3151191 hlo_sharding.cc:1530] Check failed: !IsManualLeaf() && !IsUnknownLeaf()
```

Verification: confirmed, report corrected. **Expected behavior, from:** StableHLO validity per item: bitcast_convert (u32[2] -> u16[2,2] -> u8[2,2,2]) is valid under docs/spec.md 'bitcast_convert' semantics; gather start_indices are 'tensor of integer type' (docs/spec.md gather section, input table); pad allows negative edge padding (docs/spec.md:4703-4709); reduce with the logical

OR body is a valid reduce computation. So each item is XLA's failure on valid StableHLO, except the gspmd_* items, which are XLA GSPMD limits on programs that Shardy compiles.

- zml/Sharding.zig:50-54: 'pub fn fromTarget(target: Target) Partitioner { return switch (target) { .cpu, .cuda, .rocm, .tpu, .oneapi, .neuron, .metal => .shardy, }; }'
- zml/Compiler.zig:777-780: 'const use_shardy_partitioner = switch (partitioning.partitioner) { .shardy => true, .gspmd => false, };'

Suggested fix. Report upstream with these reproducers; in ZML, widen small index dtypes before SPMD gather/scatter.

tools/zmlsmith/regressions/found/xla_bitcast.zig (16 lines)

```
const std = @import("std");
const zml = @import("zml");

// Upstream XLA: two chained narrowing bitcasts fail the HLO verifier after simplification.
test "xla_bitcast_chain" {
    const platform = zml.testing.env();
    const Local = struct {
        fn forward(x: zml.Tensor) zml.Tensor {
            return x.bitCast(.u16).rename(.{ .bitcast = .b }).bitCast(.u8);
        }
    };
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .a = 2 }, .u32));
    var exe = try zml.module.compile(std.testing.allocator, std.testing.io, Local.forward, .{x}, platform, .{});
    defer exe.deinit();
}
```

tools/zmlsmith/regressions/found/xla_spmd_idx.zig (54 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500877.
test "xla_spmd_small_indices" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor, x1: zml.Tensor) [1]zml.Tensor {
            const x2 = x0.gather(.{ .c = x1, .p = x1 }, .{});
            _ = &x2;
            return .{x2};
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_500877", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .c = 4, .p = 23 }, .u16).withPartitioning(.{ .c = .data }));
    const x1: zml.Tensor = .fromShape(zml.Shape.init(.{ .h = 4 }, .u16).withPartitioning(.{ .h = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x0, x1 }, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .h = 4 }, .u16), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var b1: zml.Buffer = try .fromBytes(io, platform, x1.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b1.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ b0, b1 });
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .h = 4 }, .u16), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

tools/zmlsmith/regressions/found/xla_spmd_slice.zig (44 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500483.
test "xla_spmd_strided_slice" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = x0.reshape(zml.Shape.init(.{ .e = 2, .b = 1, .n = 2, .f = 1 }, .f32));
```

```
            _ = &x1;
            const x2 = x1.slices(@as([]const i64, &.{ 2, 3, 0 }), &.{ .{ .start = 1, .end = 2 }, .{ .start = 0, .end = 1, .step = 3 }, .{ .start = -1, .end =
2, .step = 3 } });
            _ = &x2;
            const x3 = x2.clamp(x2, x2);
            _ = &x3;
            return .{x3};
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_500483", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .b = 4 }, .f32).withPartitioning(.{ .b = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .e = 1, .b = 1, .n = 1, .f = 1 }, .f32), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .e = 1, .b = 1, .n = 1, .f = 1 }, .f32), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .exact_match);
    }
}
```

`tools/zmlsmith/regressions/found/xla_spmd_or.zig` (201 lines, first 70 shown)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 501727.
test "xla_spmd_or_reduce" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = zml.ops.reduce(.{x0}, .{zml.Tensor.constant(zml.DataType.zero(.i16))}, &[_]i64{ 1, 2 }, struct {
                fn f(a: zml.ops.ReduceArgs) struct { zml.Tensor } {
                    return .{a.left.logical(.OR, a.right)};
                }
            }.f, .{})[0];
            _ = &x1;
            return .{x1};
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_501727", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .h = 1, .g = 39, .d = 48 }, .i16).withPartitioning(.{ .d = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .h = 1, .g = 1, .d = 1 }, .i16), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
```

tools/zmlsmith/regressions/found/xla_spmd_pad.zig (22 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Upstream XLA SPMD partitioner: a pad with interior padding and negative edge padding
// on a partitioned axis aborts the compiler, CHECK failure in spmd_partitioner.cc
// "get_first_valid_element_offset_on_dilated_shard(0) == ...". Found by zmlsmith
// (seeds 121725 and 141095), where the axis was partitioned by manualComputation.
test "xla_spmd_pad" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const Local = struct {
        fn forward(x: zml.Tensor) zml.Tensor {
            return x.pad(0, .{ .p = zml.Tensor.Pad{ .low = -1, .high = -45, .interior = 2 } }).withPartitioning(.{ .p = .data });
        }
    };
    const mesh = try @constCast(platform).registerSharding("xla_spmd_pad", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .p = 40 }, .i32).withPartitioning(.{ .p = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x}, platform, .{ .shardings = &.{mesh} });
    exe.deinit();
}
```

tools/zmlsmith/regressions/found/gspmd_pool_manual.zig (27 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Upstream XLA GSPMD partitioner: a strided maxPool1d over a size 1 axis, then a
// manualComputation partitioned on the other axis, aborts the compiler with
// "spmd_partitioner.cc Check failed: !target.IsReplicatedOrSingleDevice()". Shardy
// compiles it. Found by zmlsmith with --gspmd (seed 171195).
test "gspmd_pool_manual" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const Local = struct {
        fn body(x: zml.Tensor, out: zml.Shape) zml.Tensor {
            return x.negate().reshape(out);
        }
        fn forward(x: zml.Tensor) zml.Tensor {
            const p = x.maxPool1d(.{ .window_dimensions = 1, .window_strides = 3 }).values;
            const ps = p.withPartitioning(.{ .e = .data });
            return zml.ops.manualComputation(body, ps, ps.shape()).withPartitioning(.{ .e = .replicated });
        }
    };
    const mesh = try @constCast(platform).registerSharding("gspmd_pool_manual", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .e = 4, .n = 1 }, .f64));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x}, platform, .{ .shardings = &.{mesh}, .partitioner = .gspmd });
    exe.deinit();
}
```

tools/zmlsmith/regressions/found/gspmd_mask_manual.zig (40 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Upstream XLA GSPMD partitioner, on a two level mesh (`data` on the bus, `model` on the
// rings): a program returning nn.causalAttnMask next to an unrelated manualComputation
// result aborts the compiler with "hlo_sharding.cc Check failed: !IsManualLeaf() &&
// !IsUnknownLeaf()". Shardy compiles it, and so does GSPMD on the default one level mesh.
// Found by zmlsmith with --gspmd --mesh2d (seed 190135).
fn twoLevelMesh(allocator: std.mem.Allocator, target: zml.Target, devices: []const zml.platform.Device) anyerror!zml.Sharding.PhysicalMesh {
    const half = devices.len / 2;
    const groups = try allocator.alloc(zml.Sharding.PhysicalNode, 2);
    for (groups, 0..) |*grp, gi| {
        const leaves = try allocator.alloc(zml.Sharding.PhysicalNode, half);
        for (leaves, 0..) |*l, j| l.* = .device(devices[gi * half + j]);
        grp.* = .axis(.link, .{ .ring = .linear }, leaves);
    }
    return zml.Sharding.PhysicalMesh.fromTree(allocator, target, .axis(.bus, .tree, groups));
}

test "gspmd_mask_manual" {
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const platform: *zml.Platform = try .auto(allocator, io, .{ .physical_mesh = .{ .custom = twoLevelMesh } });
    defer platform.deinit(allocator, io);
    const Local = struct {
        fn body(x: zml.Tensor, out: zml.Shape) zml.Tensor {
            return x.relu().reshape(out);
        }
        fn forward(x: zml.Tensor) [2]zml.Tensor {
            const xs = x.withPartitioning(.{ .e = .data });
            const m = zml.ops.manualComputation(body, xs, xs.shape()).withPartitioning(.{ .e = .replicated });
            return .{ zml.nn.causalAttnMask(.{ .q = 5, .k = 2 }, .f32, null), m };
        }
    };
    const mesh = try platform.registerShardingWithStrategy("m", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }), .parseBindings(.{ .data
= .bus, .model = .link }));
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .m = 5, .e = 4 }, .f32));
    var exe = try platform.compileFn(allocator, io, Local.forward, .{x}, .{ .shardings = &.{mesh}, .partitioner = .gspmd });
    exe.deinit();
}
```

M22 XLA CPU runtime failures: conv with reversal + stride + padding, FFT layout

MEDIUM upstream XLA crash · XLA CPU · risk 6 (impact 3 × exposure 2)

API	conv2d, Tensor.fft
Location	XLA CPU
Platform	CPU
KnownBug	xla_conv_reversal, xla_fft_layout

What happens. A convolution with window reversal, stride 2 and padding has the StableHLO result shape {1,2,2,2} (8 f32; the shape matches the formula at zml/tensor.zig:1046-1054). At run time the device buffer is 16 f32 (64 bytes) and reading it back into a 32-byte destination fails with InvalidArgument ('dst_size must be >= 64, got 32'). Whether the executable writes 16 elements or allocates 16 is not established. An FFT whose input gets a non-default layout in a sharded program fails with RET_CHECK at xla/backends/cpu/runtime/fft_thunk.cc:168 (input layout must be monotonic with dim 0 major), then ZML reports 'PJRT execution failed with: error.Internal'.

Root cause. Upstream XLA CPU. For the FFT: the thunk requires a dim-0-major layout (fft_thunk.cc:168-169), and StableHLO has no layouts, so XLA's layout assignment picks the non-default one. ZML's op builders set no HLO layouts (grep for minor_to_major finds only buffer and PJRT code: zml/mem.zig:73-75, zml/platform.zig, pjrt/pjrt.zig). The conv mechanism is not identified.

> **Risk assessment.** Wrong buffer sizing is a memory-safety concern inside XLA only if the write really overruns the result; not established by this reproducer (the failure seen is a size check on read-back). The FFT case is a hard run-time failure for sharded programs.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_xla_conv_rev --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_xla_fft_layout --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `xla_conv_rev`: *fails* (matches `InvalidArgument`)
   ```
   [pjrt] (warn): [PJRT_Buffer_ToHostBuffer] InvalidArgument: `dst_size` must be >= 64, got 32.
   ```
   `xla_fft_layout`: *fails* (matches `panic: PJRT execution failed with: error\.Internal`)
   ```
   thread 3169174 panic: PJRT execution failed with: error.Internal
   ```

Verification: confirmed, report corrected. **Expected behavior, from:** The conv result shape is the StableHLO convolution shape (docs/spec.md '### convolution'), and the reproducer asserts that shape before execution. The FFT RET_CHECK is XLA's own precondition (fft_thunk.cc:168-169).

- xla/backends/cpu/runtime/fft_thunk.cc:168: 'TF_RET_CHECK(LayoutUtil::IsMonotonicWithDim0Major(input_shape_.layout()));'
- tools/zmlsmith/regressions/found/xla_conv_rev.zig:20: 'try zml.testing.expectEqualShapes(zml.Shape.init(.{ 1, 2, 2, 2 }, .f32), exe.output_shapes[0]);'

Suggested fix. Report upstream.

`tools/zmlsmith/regressions/found/xla_conv_rev.zig` (34 lines)

```
const std = @import("std");
const zml = @import("zml");

// Upstream XLA CPU: a convolution with window reversal, a stride > 1 and padding returns a
// buffer of the wrong size: ZML and the StableHLO type say {1, 2, 2, 2} (8 elements), the
// executable writes 16, and reading it back fails ("`dst_size` must be >= 64, got 32").
// Any two of the three options work; every dtype is affected. Found by zmlsmith from
// seeds 1000183 and 1101637.
test "xla_conv_rev" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x: zml.Tensor, w: zml.Tensor) zml.Tensor {
            return x.conv2d(w, .{ .window_strides = &.{ 2, 2 }, .padding = &.{ 0, 1, 1, 1 }, .window_reversal = &.{ false, true } });
        }
    };
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ 1, 1, 5, 2 }, .f32));
    const w: zml.Tensor = .fromShape(zml.Shape.init(.{ 2, 1, 3, 2 }, .f32));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x, w }, platform, .{});
    defer exe.deinit();
    try zml.testing.expectEqualShapes(zml.Shape.init(.{ 1, 2, 2, 2 }, .f32), exe.output_shapes[0]);

    var xb: zml.Buffer = try .uninitialized(io, platform, x.shape(), .replicated, .{});
    defer xb.deinit();
    var wb: zml.Buffer = try .uninitialized(io, platform, w.shape(), .replicated, .{});
    defer wb.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ xb, wb });
    defer res.deinit();
    const got = try res.toSliceAlloc(allocator, io);
```

```
    defer got.free(allocator);
}
```

tools/zmlsmith/regressions/found/xla_fft_layout.zig (124 lines, first 70 shown)

```
const std = @import("std");
const zml = @import("zml");

// Upstream XLA CPU: an FFT in a sharded program whose input gets a non default layout
// fails at run time, "RET_CHECK failure (xla/backends/cpu/runtime/fft_thunk.cc:168)
// LayoutUtil::IsMonotonicWithDim0Major(input_shape_.layout())", and ZML panics with
// "PJRT execution failed with: error.Internal". Found by zmlsmith (seed 321385), reduced
// from 30 nodes.
test "xla_fft_layout" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [2]zml.Tensor {
            const x1 = x0.gelu();
            _ = &x1;
            const x2 = zml.Tensor.stack(&.{x1}, 2, .b);
            _ = &x2;
            const x3 = x2.stutter(&[_]u32{ 1, 1, 3, 1 });
            _ = &x3;
            const x4 = zml.Tensor.complex(x0, x1).fft(.{ .kind = .FFT, .length = &[_]i64{ 2, 4 } }).imag();
            _ = &x4;
            const x5 = x4.pad(-3, .{ .k = zml.Tensor.Pad{ .low = 3, .high = 3, .interior = 1 } });
            _ = &x5;
            const x6 = blk: {
                const p = x3.withPartitioning(.{ .k = .data });
                const r = zml.ops.manualComputation(struct {
                    x: zml.Tensor,
                    fn run(c: @This(), out: zml.Shape) zml.Tensor {
                        _ = &c;
                        return c.x.abs().reshape(out);
                    }
                }.run, .{ .x = p }, p.shape());
                break :blk r.withPartitioning(.{ .k = .replicated });
            };
            _ = &x6;
            const x7 = x5.reshape(.{ @as(i64, 104), .auto });
            _ = &x7;
            const x8 = x7.transpose(.{});
            _ = &x8;
            return .{ x8, x6 };
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_321385", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .hd = 4, .h = 2, .k = 4 }, .f32));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ 1, 104 }, .f32), exe.output_shapes[0]);
    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .hd = 4, .h = 2, .b = 3, .k = 4 }, .f32), exe.output_shapes[1]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 64, 192, 0, 0, 64, 192, 0, 0, 64, 192, 0, 0, 0,  0,   0, 0, 64, 192, 0, 0, 0,  0,
            0, 0, 64, 192, 0, 0, 0,  0,   0, 0, 64, 192, 0, 0, 0,  0,   0, 0, 64, 192, 0, 0, 64, 192,
            0, 0, 64, 192, 0, 0, 64, 192, 0, 0, 64, 192, 0, 0, 64, 192, 0, 0, 0,  0,   0, 0, 64, 192,
```

M23 HF revisions containing a slash are mis-parsed

MEDIUM ZML parse · VFS · risk 6 (impact 2 × exposure 3)

API vfs HF Repo.parse
Location vfs/hf.zig:221-233, :160-161
Platform CPU

What happens. Opening `hf://org/model@refs/pr/1/config.json` requests the tree of revision `refs` and looks for `pr/1/config.json`; the open fails.

Root cause. The URI is split on '/' before looking for '@', and the revision is put into URLs without percent-encoding.

> **Risk assessment.** Pull request revisions (`refs/pr/N`) and converted branches (`refs/convert/...`) can't be loaded, which is how unmerged model fixes are usually tested.

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed). These reproducers start found/hf_fixture.zig, a local stand-in for huggingface.co (a python3 https proxy with a test certificate trusted only by the test's client), so python3 must be on the host.
2. Run bazel test //tools/zmlsmith/regressions:found_hf_rev_slash --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   hf_rev_slash: *fails* (matches Failed to fetch tree: status=\.not_found)
   1/1 hf_rev_slash.test.hf_rev_slash...[zml/vfs/hf] (err): Failed to fetch tree: status=.not_found

Verification: confirmed as reported. **Expected behavior, from:** Hub revisions can contain slashes (refs/pr/N, refs/convert/...). Parsing the '@' revision after splitting on '/' is wrong by the URI shape in hf://. Percent-encoding the revision in the tree URL matches the reproducer's route; the Hub's exact expectation was not verified offline.
- vfs/hf.zig:221-233: pub fn parse(uri: []const u8) !Repo { var parts = std.mem.splitScalar(u8, uri, '/'); ... .model = parts.next() ... if (std.mem.findScalar(u8, repo.model, '@')) |at_index| { repo.rev = repo.model[at_index + 1 ..]};
- vfs/hf.zig:160: TREE_URL_TEMPLATE ... /tree/{[rev]s}/{[path]s}?... (rev inserted without encoding)

Suggested fix. Parse the revision up to the next path segment the Hub can't contain, or accept an encoded refs%2Fpr%2F1, and percent-encode the revision in URLs.

tools/zmlsmith/regressions/found/hf_rev_slash.zig (25 lines)

```
const std = @import("std");
const fx = @import("hf_fixture.zig");

// vfs HF Repo.parse splits the URI on '/' before looking for '@' (vfs/hf.zig:221-233 at
// 74c590b0): a revision containing a slash, such as the Hub's pull request refs
// "refs/pr/1" or "refs/convert/parquet", becomes revision "refs" and the rest is taken as
// the file path, and the revision is put in URLs without percent-encoding. Opening
// hf://org/model@refs/pr/1/config.json requests the tree of revision "refs" and fails.
// Found by zmlsmith's HF stand-in.
test "hf_rev_slash" {
    const allocator = std.testing.allocator;
    const tree = try fx.treeJson(allocator, &.{.{ "config.json", 4 }});
    defer allocator.free(tree);
    const resp = try std.fmt.allocPrint(allocator, "HTTP/1.1 200 OK\r\nContent-Length: {d}\r\nConnection: close\r\n\r\n{s}", .{ tree.len, tree });
    defer allocator.free(resp);
    const routes = try std.json.Stringify.valueAlloc(allocator, .{.{ "/tree/refs%2Fpr%2F1", "raw", resp }}, .{});
    defer allocator.free(routes);
    var f: fx.Fixture = undefined;
    try f.init(routes);
    defer f.deinit();
    const vio = f.vfs.io();
    var file = try std.Io.Dir.openFile(.cwd(), vio, "hf://org/model@refs/pr/1/config.json", .{ .mode = .read_only });
    file.close(vio);
}
```

M24 RoPE scaling configs using type are rejected

MEDIUM ZML parse · nn · risk 6 (impact 2 × exposure 3)

API RopeOpts.Scaling.jsonParse
Location zml/nn.zig:529
Platform CPU

What happens. {"type": "linear", "factor": 2.0}, the pre-rename HF spelling that transformers still accepts, fails with MissingField.

Root cause. Only rope_type is read.

> **Risk assessment.** Older checkpoints fail to load.

Reproduction.
1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run bazel test //tools/zmlsmith/regressions:found_rope_type_key --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   rope_type_key: *fails* (matches MissingField)
   1/1 rope_type_key.test.rope_type_key...FAIL (MissingField)

Verification: confirmed as reported. **Expected behavior, from:** HF transformers src/transformers/modeling_rope_utils.py: validate_rope and standardize_rope_params read rope_parameters.get("rope_type", rope_parameters.get("type", "default")), so the legacy type key is accepted (fetched from main in this run).
- zml/nn.zig:529: const impl = obj.get("rope_type") orelse return error.MissingField;

Suggested fix. Accept `type` as an alias.

tools/zmlsmith/regressions/found/rope_type_key.zig (16 lines)

```
const std = @import("std");
const zml = @import("zml");

// RopeOpts.Scaling.jsonParse only reads `rope_type`. HF configs written before
// transformers renamed the key use `type` (e.g. {"type": "linear", "factor": 2.0}), and
// transformers still accepts it; ZML fails with MissingField (zml/nn.zig:529 at 74c590b0).
// Found by reading the code while auditing zmlsmith.
test "rope_type_key" {
    const json =
        \\{"type": "linear", "factor": 2.0}
    ;
    const parsed = try std.json.parseFromSlice(zml.nn.RopeOpts.Scaling, std.testing.allocator, json, .{});
    defer parsed.deinit();
    try std.testing.expectEqual(@as(f32, 2.0), parsed.value.linear.factor);
}
```

M25 Crashes on valid inputs in small Tensor helpers

MEDIUM ZML crash · Tensor · risk 6 (impact 2 × exposure 3)

API chunkAllowTrailing, dynamicUpdateSlice1d, unflatten, cartesianProduct, outer, fmod
Location zml/tensor.zig:3803-3811 (chunkAllowTrailing), zml/tensor.zig:3921 (dynamicUpdateSlice1d placeholder), zml/tensor.zig:1898-1900 (unflatten), zml/tensor.zig:4429-4437 (cartesianProduct rank-0 path), zml/tensor.zig:2608-2609 (outer), zml/tensor.zig:502 (fmod)
Platform CPU
KnownBug chunk_trailing, dus1d_offset_dtype, unflatten_negative_axis, cartesian_scalar, outer_untagged, fmod_non_f32

What happens. chunkAllowTrailing: the tail size is wrong, so with dim 5 and n 3 it returns 3 chunks and drops the 2 remaining items; the trailing chunk end is clamped to start+chunk_size (zml/tensor.zig:3811), so a trailing part longer than one chunk is cut. With n > dim, chunk_size is 0 and @rem(d, 0) is evaluated (zml/tensor.zig:3804). dynamicUpdateSlice1d: "start indices must have same element type" for non-i32 offsets. unflatten: a negative axis is pushed one past the end. cartesianProduct: a rank-0 input asserts. outer: untagged inputs fail ("Can't broadcast"). fmod: asserts on non-f32 dtypes.

Root cause. Per helper: (a) chunkAllowTrailing computes `tail_chunk_size = @rem(d, chunk_size)` instead of `d - n_chunks * chunk_size` (zml/tensor.zig:3804) and clamps the trailing end to start+chunk_size (zml/tensor.zig:3811). (b) dynamicUpdateSlice1d fills the non-target start indices with an i32 placeholder (zml/tensor.zig:3921) and never converts the offset to match. (c) unflatten adds 1 to the already-normalized axis for negative input (zml/tensor.zig:1898). (d) the rank-0 branch of _cartesianProduct calls broadcast with a one-element axes list (zml/tensor.zig:4436-4437) while broadcast requires axes_.len == self.rank() (zml/tensor.zig:2621). (e) outer goes through broad, which maps axes by tag (zml/tensor.zig:2665, 2681-2684), while Shape.outer happily appends untagged dims (zml/shape.zig:1609-1616). (f) fmod hard-codes the divisor type to f32 (zml/tensor.zig:502).

> **Risk assessment.** Each crashes on a valid call; individually small, collectively a sign these helpers have no tests.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_chunk_trailing --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_dus1d_offset --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_unflatten_neg --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_cartesian_scalar --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_outer_untagged --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_fmod_non_f32 --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

```
chunk_trailing: fails (matches found/chunk_trailing\.zig:24:)
/home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/5272/execroot/_main/
tools/zmlsmith/regressions/found/chunk_trailing.zig:24:5: 0x640427743034 in test.chunk_trail
dus1d_offset: fails (matches Failed to create MLIR operation)
thread 3150009 panic: Failed to create MLIR operation
unflatten_neg: fails (matches index out of bounds: index 2, len 2)
1/1 unflatten_neg.test.unflatten_negative_axis...thread 3170805 panic: index out of bounds: index 2, len 2
cartesian_scalar: fails (matches broadcast expects axes_ to map all axes from self)
1/1 cartesian_scalar.test.cartesian_scalar...thread 3151918 panic: broadcast expects axes_ to map all axes from self to axes of the
output shape, got broadcast(Tensor({f64}), {1,f=3,1,f64}, { 0 })
outer_untagged: fails (matches Can't broadcast Tensor\(\{1,f16\}\) to \{1,4,f16\})
1/1 outer_untagged.test.outer_untagged...thread 3156493 panic: Can't broadcast Tensor({1,f16}) to {1,4,f16}
```

`fmod_non_f32`: *fails* (matches `remainder expects tensor to be of same type`)

```
1/1 fmod_non_f32.test.fmod_non_f32...thread 3150800 panic: remainder expects tensor to be of same type, got Tensor({k=4,f16}) and
Tensor({k=4,f32})
```

Verification: confirmed, report corrected. **Expected behavior, from:** (a) The doc comment at zml/tensor.zig:3794-3795 says n equal chunks plus one part with the remaining items; the test at zml/tensor.zig:3832-3848 checks the trailing shape. (b) dynamicUpdateSlice derives the index dtype from the first offset (zml/tensor.zig:3984), so a non-i32 offset is explicitly supported. (c) axis() normalizes negative axes (zml/shape.zig:435-443). (d) _cartesianProduct explicitly accepts rank-0 vectors (zml/tensor.zig:4423). (e) untagged dims are handled by Shape.outer (zml/shape.zig:1609); outer has no unit test in zml/tensor.zig (grep finds none). (f) remainderConst uses self.dtype() (zml/tensor.zig:1201-1202), and torch.fmod, which the doc links (zml/tensor.zig:500), is not f32-only.

- zml/tensor.zig:3804: const tail_chunk_size: i64 = @rem(d, chunk_size);
- zml/tensor.zig:3811: chunk.* = self.slice(a, .{ .start = start, .end = @min(start + chunk_size, d) });
- zml/tensor.zig:3921: const placeholder = Tensor.scalar(0, .i32);
- zml/tensor.zig:1898: const a = if (axis_ >= 0) self.axis(axis_) else self.axis(axis_) + 1;
- zml/tensor.zig:4437: o.* = x.broadcast(res_shape, &[1]i64{@intCast(i)});
- zml/tensor.zig:2621: broadcast asserts axes_.len == self.rank()
- zml/tensor.zig:502: return self.remainder(scalar(divisor, .f32).broadcast(self._shape, &.{}));
- zml/tensor.zig:1202: return self.remainder(.scalar(b, self.dtype()));

Suggested fix. Fix each; zmlsmith has a reduced reproducer per helper.

tools/zmlsmith/regressions/found/chunk_trailing.zig (26 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// chunkAllowTrailing(axis, n) on a dim of 5 with n = 3 must return 3 chunks of 1 item
// plus one trailing chunk of the remaining 2 items (see the doc comment above
// Tensor.chunkAllowTrailing and its test). The tail is computed as @rem(d, chunk_size)
// instead of d - n * chunk_size, so no trailing chunk is produced.
test "chunk_trailing" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const chunks = x0.chunkAllowTrailing(1, 3);
            return .{chunks[chunks.len - 1]};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ 2, 5 }, .f32));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ 2, 2 }, .f32), exe.output_shapes[0]);
}
```

tools/zmlsmith/regressions/found/dus1d_offset.zig (65 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500000.
test "dus1d_offset_dtype" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor, x1: zml.Tensor, x2: zml.Tensor) [1]zml.Tensor {
            const x3 = x0.dynamicUpdateSlice1d(x1, 1, x2);
            _ = &x3;
            return .{x3};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .k = 1, .e = 6, .d = 6, .n = 1 }, .u32));
    const x1: zml.Tensor = .fromShape(zml.Shape.init(.{ .k = 1, .e = 5, .d = 6, .n = 1 }, .u32));
    const x2: zml.Tensor = .fromShape(zml.Shape.init(.{}, .i8));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x0, x1, x2 }, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .k = 1, .e = 6, .d = 6, .n = 1 }, .u32), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var b1: zml.Buffer = try .fromBytes(io, platform, x1.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
```

```
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b1.deinit();
    var b2: zml.Buffer = try .fromBytes(io, platform, x2.shape(), .replicated, &[_]u8{
        0,
    });
    defer b2.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ b0, b1, b2 });
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .k = 1, .e = 6, .d = 6, .n = 1 }, .u32), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

tools/zmlsmith/regressions/found/unflatten_neg.zig (41 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500260.
test "unflatten_negative_axis" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = x0.unflatten(-1, 1);
            _ = &x1;
            return .{x1};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .n = 1, .c = 2 }, .i16));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ 1, 1, 2 }, .i16).withTags(.{ .n, .c, ._ }), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ 1, 1, 2 }, .i16).withTags(.{ .n, .c, ._ }), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

tools/zmlsmith/regressions/found/cartesian_scalar.zig (44 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500205.
test "cartesian_scalar" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor, x1: zml.Tensor) [1]zml.Tensor {
            const x2 = zml.Tensor.cartesianProduct(3, .{ x0, x1, x0 })[2];
            _ = &x2;
            return .{x2};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{}, .f64));
    const x1: zml.Tensor = .fromShape(zml.Shape.init(.{ .f = 3 }, .f64));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x0, x1 }, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ 1, 3, 1 }, .f64).withTags(.{ ._, .f, ._ }), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var b1: zml.Buffer = try .fromBytes(io, platform, x1.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b1.deinit();
```

```
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ b0, b1 });
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ 1, 3, 1 }, .f64).withTags(.{ ._, .f, ._ }), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .exact_match);
    }
}
```

tools/zmlsmith/regressions/found/outer_untagged.zig (41 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500030.
test "outer_untagged" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x1: zml.Tensor) [1]zml.Tensor {
            const x0 = zml.Tensor.zeroes(zml.Shape.init(.{1}, .f16));
            _ = &x0;
            const x2 = x0.outer(x1);
            _ = &x2;
            return .{x2};
        }
    };

    const x1: zml.Tensor = .fromShape(zml.Shape.init(.{4}, .f16));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x1}, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ 1, 4 }, .f16), exe.output_shapes[0]);

    var b1: zml.Buffer = try .fromBytes(io, platform, x1.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b1.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b1});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ 1, 4 }, .f16), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .exact_match);
    }
}
```

tools/zmlsmith/regressions/found/fmod_non_f32.zig (39 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500203.
test "fmod_non_f32" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = x0.fmod(2);
            _ = &x1;
            return .{x1};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .k = 4 }, .f16));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .k = 4 }, .f16), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .k = 4 }, .f16), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .exact_match);
    }
}
```

M26 Tensor.any, Tensor.all and pad with a slice don't compile

MEDIUM ZML compile · Tensor · risk 6 (impact 2 × exposure 3)

API	Tensor.any, Tensor.all, Tensor.pad
Location	zml/tensor.zig:4371 (any: Tensor.constant with two args), zml/tensor.zig:4378 (any: reduce body returns bare Tensor), zml/tensor.zig:4394 and :4398 (all: bare return, then [0] index), zml/shape.zig:1560-1565 (parseAxesOptions slice branch has no return)
Platform	CPU
KnownBug	any_compile, all_compile

What happens. any: Tensor.constant is called with two arguments (zml/tensor.zig:4371) and the reduce body returns a bare Tensor (zml/tensor.zig:4378). all: the reduce body returns a bare Tensor (zml/tensor.zig:4394), so `[0]` (zml/tensor.zig:4398) indexes a Tensor. pad with a slice of Pad: parseAxesOptions' slice branch falls through into the compile error (zml/shape.zig:1560-1578).

Root cause. any: wrong arity for Tensor.constant (signature at zml/tensor.zig:2564 takes one value) and the reduce callback must return a tuple. all: the same bare-return mistake. pad: a missing `return res;` at the end of the slice branch (zml/shape.zig:1565).

> **Risk assessment.** Public API that can't be used at all.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_any_compile --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_all_compile --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_pad_slice --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `any_compile`: *fails to build* (matches `expected 1 argument\(s\), found 2`)
   `zml/tensor.zig:4371:42: error: expected 1 argument(s), found 2`
   `all_compile`: *fails to build* (matches `does not support indexing`)
   `zml/ops.zig:249:38: error: type 'tensor.Tensor' does not support indexing`
   `pad_slice`: *fails to build* (matches `parseStruct expects struct or tuple`)
   `stdx/debug.zig:26:5: error: parseStruct expects struct or tuple, got []const tensor.Tensor.Pad`

Verification: confirmed, report corrected. **Expected behavior, from:** ops.reduce returns stdx.meta.FnReturn(func) and indexes the result with [0] (zml/ops.zig:210; zml/tensor.zig:4372-4382). Other reduce callbacks return struct { Tensor } (zml/tensor.zig:1764, 3366). Tensor.constant takes one value (zml/tensor.zig:2564). parseAxesOptions documents a slice form with exactly rank entries (zml/shape.zig:1561).

- zml/tensor.zig:4371: const pred = self.cmp(.NE, Tensor.constant(self.dims(), self.dtype().zero()));
- zml/tensor.zig:4378: return args.left.logical(.OR, args.right);
- zml/tensor.zig:4394: return args.left.logical(.AND, args.right);
- zml/tensor.zig:4398:)[0]; (all: indexes the bare Tensor returned by ops.reduce)
- zml/tensor.zig:1764: pub fn acc(args: ops.ReduceArgs) struct { Tensor } {
- zml/shape.zig:1560: if (comptime stdx.meta.isSliceOf(V, T)) {
- zml/shape.zig:1578: stdx.debug.compileError("parseStruct expects struct or tuple, got {}", .{V});

Suggested fix. any: use Tensor.constant(self.dtype().zero()).broad(self.shape()) (or a scalar) and return `.{ ... }` from the reduce callback. all: return `.{ ... }`. pad: add `return res;` at the end of the slice branch. Add a test per function.

tools/zmlsmith/regressions/found/any_compile.zig (16 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Doesn't compile: see the zmlsmith report.
test "any_compile" {
    const platform = zml.testing.env();
    const Local = struct {
        fn forward(x: zml.Tensor) zml.Tensor {
            return x.any(.b);
        }
    };
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .a = 2, .b = 3 }, .i32));
    var exe = try zml.module.compile(std.testing.allocator, std.testing.io, Local.forward, .{x}, platform, .{});
    defer exe.deinit();
}
```

tools/zmlsmith/regressions/found/all_compile.zig (16 lines)

```zig
const std = @import("std");
const zml = @import("zml");
```

```
// Doesn't compile: see the zmlsmith report.
test "all_compile" {
    const platform = zml.testing.env();
    const Local = struct {
        fn forward(x: zml.Tensor) zml.Tensor {
            return x.all(.b);
        }
    };
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .a = 2, .b = 3 }, .i32));
    var exe = try zml.module.compile(std.testing.allocator, std.testing.io, Local.forward, .{x}, platform, .{});
    defer exe.deinit();
}
```

tools/zmlsmith/regressions/found/pad_slice.zig (16 lines)

```
const std = @import("std");
const zml = @import("zml");

// Doesn't compile: see the zmlsmith report.
test "pad_slice_compile" {
    const platform = zml.testing.env();
    const Local = struct {
        fn forward(x: zml.Tensor) zml.Tensor {
            return x.pad(0, @as([]const zml.Tensor.Pad, &.{ .{ .low = 1 }, .{} }));
        }
    };
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .a = 2, .b = 3 }, .i32));
    var exe = try zml.module.compile(std.testing.allocator, std.testing.io, Local.forward, .{x}, platform, .{});
    defer exe.deinit();
}
```

M27 broad / broadcastLeft / Right and nearest / upsample lose tags

MEDIUM ZML tags · Tensor · risk 6 (impact 2 × exposure 3)

API	Tensor.broad, nn.nearest, nn.upsample
Location	`zml/tensor.zig:2654-2655 (broadcastRight early return, uses Shape.eql which ignores tags, zml/shape.zig:507-509), zml/tensor.zig:2668 (broad early return), zml/nn.zig:1198-1199 (nearest: untagged offsets gathered)`
Platform	CPU
KnownBug	`broad_equal_dims_tags, nearest_drops_tags`

What happens. When dims already match, broad returns self with its own tags instead of the target's; resized axes lose their tags.

Root cause. broad_equal_dims_tags: the reproducer goes through broadcastRight (zml/tensor.zig:2654), which returns self when rank matches and Shape.eql is true; eql compares dims and dtype only (zml/shape.zig:507-509), so self keeps its own tags. broad (zml/tensor.zig:2668) has the same early return and is the other site to fix. nearest: the offsets tensor is untagged (zml/nn.zig:1198), so the gather in zml/nn.zig:1199 gives the resized axis the untagged tag of its indices, and the original tag is lost.

> **Risk assessment.** Tag-based code downstream fails or, worse, picks the wrong axis.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_broad_tags --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_nearest_tags --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `broad_tags`: *fails* (matches `Expected \{k=4,u32\}, got \{c=4/data,u32\}`)
   `Expected {k=4,u32}, got {c=4/data,u32}FAIL (TestExpectedEqual)`
   `nearest_tags`: *fails* (matches `Expected \{h=1,hd=4,k=1,f32\}, got \{h=1,hd=4,1,f32\}`)
   `Expected {h=1,hd=4,k=1,f32}, got {h=1,hd=4,1,f32}FAIL (TestExpectedEqual)`

Verification: confirmed, report corrected. **Expected behavior, from:** The Tensor.broadcast doc says the result takes its tags from output_shape (zml/tensor.zig:2616), and broadcast itself does that when it does run (zml/tensor.zig:2627-2631 builds res_shape from output_shape). The nearest reproducer expects the resized axis to keep the .k tag, which the input shape has.

- zml/tensor.zig:2654: if (self.rank() == output_shape.rank() and self._shape.eql(output_shape)) {
- zml/tensor.zig:2668: if (std.mem.eql(i64, self.dims(), other.dims())) return self;
- zml/shape.zig:507-508: pub fn eql(...) bool { return std.mem.eql(i64, self.dims(), other.dims()) and self.dtype() == other.dtype(); }
- zml/nn.zig:1198: const offsets = Tensor.arange(.{ .end = n }, .f32)...convert(.i32);
- zml/nn.zig:1199: res = ops.gather(res, &.{ax}, &.{offsets}, .{ .indices_are_sorted = true });

Suggested fix. Return the target shape's tags.

tools/zmlsmith/regressions/found/broad_tags.zig (42 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500174.
test "broad_equal_dims_tags" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = x0.broadcastRight(zml.Shape.init(.{ .k = 4 }, .u32));
            _ = &x1;
            return .{x1};
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_500174", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .c = 4 }, .u32).withPartitioning(.{ .c = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .k = 4 }, .u32), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .k = 4 }, .u32), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

tools/zmlsmith/regressions/found/nearest_tags.zig (41 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500052.
test "nearest_drops_tags" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = zml.nn.nearest(x0, &[_]f64{0.5});
            _ = &x1;
            return .{x1};
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_500052", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .h = 1, .hd = 4, .k = 2 }, .f32).withPartitioning(.{ .hd = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .h = 1, .hd = 4, .k = 1 }, .f32), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .h = 1, .hd = 4, .k = 1 }, .f32), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .exact_match);
    }
}
```

M28 Homemade tokenizer loaders don't compile on Zig 0.16

MEDIUM ZML compile · Tokenizer · risk 4 (impact 2 × exposure 2)

API tokenizer homemade fromHfJson, fromTinyLlamaFile
Location `zml/tokenizer/homemade.zig:1057, :1207`
Platform CPU

What happens. Any use fails to compile.

Root cause. They call `std.fs.cwd()`, removed in Zig 0.16. They have no callers, so the build never analyzes them.

Risk assessment. Dead public API; it also means the bugs above stay latent.

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run bazel test //tools/zmlsmith/regressions:found_tok_homemade_load --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   tok_homemade_load: *fails to build* (matches homemade\.zig:1057:\d+: error: root source file struct 'fs' has no member named 'cwd')
   zml/tokenizer/homemade.zig:1057:28: error: root source file struct 'fs' has no member named 'cwd'

Verification: confirmed as reported. **Expected behavior, from:** Zig 0.16.0 std in the bazel toolchain: lib/std/fs.zig contains no cwd (grep count 0), so std.fs.cwd() does not exist in the toolchain the build uses.

- zml/tokenizer/homemade.zig:1057: const file = try std.fs.cwd().openFile(tokenizer_path, .{});
- zml/tokenizer/homemade.zig:1207: const tokenizer_file = try std.fs.cwd().openFile(tokenizer_path, .{});
- external/rules_zig++zig+zig_0.16.0_x86_64-linux/lib/std/fs.zig: no cwd symbol

Suggested fix. Port to std.Io.Dir, add a test that references them.

tools/zmlsmith/regressions/found/tok_homemade_load.zig (13 lines)

```
const std = @import("std");
const homemade = @import("zml").tokenizer.homemade;

// homemade.fromHfJson and homemade.fromTinyLlamaFile, the only loaders of the pure Zig
// tokenizer, call std.fs.cwd(), which Zig 0.16 removed: any use fails to compile
// (zml/tokenizer/homemade.zig:1057, :1207 at 74c590b0). They have no callers, so the build
// never analyzes them. Found by zmlsmith parse-fuzz.
test "tok_homemade_load" {
    if (homemade.fromHfJson(std.testing.allocator, std.testing.io, "/nonexistent/tokenizer.json")) |t| {
        t.deinit();
    } else |_| {}
}
```

M29 HF tree sizes overflow on large listings

MEDIUM ZML crash · VFS · risk 4 (impact 2 × exposure 2) · **untrusted input**

API vfs HF insertTreeNode / getSizeFromTree
Location vfs/hf.zig:634 (insertTreeNode, reached on every open that fetches the tree); vfs/hf.zig:577 (getSizeFromTree, root directory only)
Platform CPU

What happens. A listing whose file sizes sum past u64 panics with "integer overflow" on open.

Root cause. Directory sizes are accumulated with unchecked +=.

Risk assessment. A server-controlled panic; only a broken or hostile server sends such sizes.

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed). These reproducers start found/hf_fixture.zig, a local stand-in for huggingface.co (a python3 https proxy with a test certificate trusted only by the test's client), so python3 must be on the host.
2. Run bazel test //tools/zmlsmith/regressions:found_hf_size_ovf --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   hf_size_ovf: *fails* (matches panic: integer overflow)
   1/1 hf_size_ovf.test.hf_size_ovf...thread 3155851 panic: integer overflow

Verification: confirmed, report corrected. **Expected behavior, from:** A tree listing with file sizes that do not fit in u64 is malformed. Checked addition returning an error is the expected handling; the same file already returns errors for bad listings.

- vfs/hf.zig:633-634: for (parents.items) |parent| { parent.size += item.size; }
- vfs/hf.zig:573-577: fn getSizeFromTree(...) { if (path.len == 0) { var total: u64 = 0; for (tree.items) |*node| { if (node.kind == .file) total += node.size; }
- vfs/hf.zig:742-743: const tree = self.getOrFetchTree(repo) catch return ...; const size = getSizeFromTree(tree, repo.path)

Suggested fix. Use saturating or checked addition and fail the listing.

tools/zmlsmith/regressions/found/hf_size_ovf.zig (22 lines)

```
const std = @import("std");
const fx = @import("hf_fixture.zig");

// vfs HF insertTreeNode adds every file size to its parent directories unchecked
// (`parent.size += item.size`, vfs/hf.zig:634 at 74c590b0), and getSizeFromTree sums the
// top level files the same way (:577): a tree listing whose sizes overflow u64 panics with
// "integer overflow" instead of failing the open. The listing comes from the server.
// Found by zmlsmith's HF stand-in.
test "hf_size_ovf" {
    const allocator = std.testing.allocator;
    const tree = try fx.treeJson(allocator, &.{ .{ "d/a", std.math.maxInt(u64) }, .{ "d/b", 2 } });
    defer allocator.free(tree);
    const routes = try fx.routesJson(allocator, tree, "abcd");
    defer allocator.free(routes);
    var f: fx.Fixture = undefined;
    try f.init(routes);
    defer f.deinit();
    const vio = f.vfs.io();
    var file = std.Io.Dir.openFile(.cwd(), vio, "hf://org/model/d/a", .{ .mode = .read_only }) catch return;
    file.close(vio);
}
```

M30 Tensor.print of an empty tensor doesn't compile

MEDIUM ZML compile · Tensor · risk 4 (impact 2 × exposure 2)

API Tensor.print
Location `zml/tensor.zig:4594-4601 (print emits zml$print custom call with has_side_effect and no sharding; customCall at :4599)`
Platform CPU

What happens. "RET_CHECK failure (spmd_partitioner.cc:7508) Side-effect HLO must have sharding".

Root cause. Tensor.print emits a side-effecting custom call (zml/tensor.zig:4599) with no sharding. The observed HLO in the previous run shows the operand folded to a constant: `custom-call(%constant.3)`. The SPMD partitioner then rejects the side-effecting instruction without a sharding (spmd_partitioner.cc:7508). The mechanism is confirmed for the empty case only; the non-empty case was not in the reproducer.

> **Risk assessment.** Debug print breaks compilation for empty tensors.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_print_empty --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `print_empty`: *fails* (matches `Side-effect HLO must have sharding`)
   `[pjrt] (warn): [PJRT_Client_Compile] Internal: RET_CHECK failure (xla/service/spmd/spmd_partitioner.cc:7508) hlo->has_sharding() Side-effect HLO must have sharding: %custom-call.1 = () custom-call(%constant.3), custom_ca`

Verification: confirmed, report corrected. **Expected behavior, from:** The reproducer's expectation is that print is a pass-through (returns x and the bytes come back unchanged, zml/tensor.zig:4588-4601 and the reproducer), which matches the ZML side-effect design.

- zml/tensor.zig:4599: ops.customCall("zml$print", body_ctx.input, {}, .{ .name = body_ctx.name }, .{ .has_side_effect = true });
- observed (previous run): RET_CHECK failure (xla/service/spmd/spmd_partitioner.cc:7508) hlo->has_sharding() Side-effect HLO must have sharding: %custom-call.1 = () custom-call(%constant.3)

Suggested fix. Skip the custom call for empty tensors, or give it a sharding.

`tools/zmlsmith/regressions/found/print_empty.zig` (36 lines)

```
const std = @import("std");
const zml = @import("zml");

// Tensor.print of an empty tensor doesn't compile: XLA folds the empty operand into a
// constant and the SPMD partitioner rejects the side effecting zml$print custom call,
// "RET_CHECK failure (spmd_partitioner.cc:7508) hlo->has_sharding() Side-effect HLO must
// have sharding". Found by zmlsmith runtime coverage of Tensor.print.
fn run(comptime dt: zml.DataType, comptime dims: anytype) !void {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const Local = struct {
        fn f(x: zml.Tensor) zml.Tensor {
            x.print("x");
            return x;
        }
    };
    const x: zml.Tensor = .init(dims, dt);
    var exe = try platform.compileFn(allocator, io, Local.f, .{x}, .{});
    defer exe.deinit();
    const bytes = try allocator.alloc(u8, x.shape().byteSize());
    defer allocator.free(bytes);
```

```
    for (bytes, 0..) |*b, i| b.* = @truncate(i *% 37 % 3);
    var b = try zml.Buffer.fromBytes(io, platform, x.shape(), .replicated, bytes);
    defer b.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.f, .{b});
    defer res.deinit();
    const got = try res.toSliceAlloc(allocator, io);
    defer got.free(allocator);
    try std.testing.expectEqualSlices(u8, bytes, got.constData());
}

test "print_empty" {
    try run(.f32, .{0});
}
```

M31 GatedDeltaNet.forward only accepts one axis order

MEDIUM ZML crash · nn · risk 4 (impact 2 × exposure 2)

API nn.GatedDeltaNet.forward
Location zml/nn.zig:1717-1718 (v_hat and delta: shape-matching mul/sub with v in .{h,v} order), :1720 (s_new mul), :1738-1740 (validation by tags only)
Platform CPU

What happens. A state given as `.{ .v, .h, .k }` panics ("mul expects tensor shapes to match").

Root cause. Inputs are validated by tags, then combined with shape-matching mul/sub and dynamicUpdateSlice.

> **Risk assessment.** Crash on valid tagged inputs.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_gdn_order --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `gdn_order`: *fails* (matches `mul expects tensor shapes to match`)
   `1/1 gdn_order.test.gdn_order...thread 3151807 panic: mul expects tensor shapes to match, got {h=1,v=2,f32} and {v=2,h=1,f32}`

Verification: confirmed, report corrected. **Expected behavior, from:** Doc of the tag requirement in validateInitialState (zml/nn.zig:1738-1740) only checks tags; the body combines arrays with shape-matching mul/sub, so accepted inputs with different axis order crash. Whether the axis order must be canonical is not documented in the code; the crash on a validated input is the inconsistency.

- zml/nn.zig:1717: const v_hat = state.s.dot(k, .k).mul(alpha.broad(hv));
- zml/nn.zig:1718: const delta = v.sub(v_hat).mul(beta.broad(hv));
- zml/nn.zig:1738: stdx.debug.assert(state.s.shape().hasTags(.{ .h, .v, .k }), ...

Suggested fix. Transpose inputs to the canonical order first.

tools/zmlsmith/regressions/found/gdn_order.zig (27 lines)

```
const std = @import("std");
const zml = @import("zml");

// nn.GatedDeltaNet.forward validates its inputs by tags only (hasTags), but the loop body
// combines them with shape-matching mul/sub and dynamicUpdateSlice, so any axis order
// other than the documented .{ .s, .h, .k } / .{ .h, .v, .k } panics, e.g. a state given as .{ .v, .h, .k }
// (zml/nn.zig:1708-1717, :1734 at 74c590b0). Found by zmlsmith (gdn family).
test "gdn_order" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(q: zml.Tensor, k: zml.Tensor, v: zml.Tensor, a: zml.Tensor, b: zml.Tensor, s: zml.Tensor) zml.Tensor {
            return zml.nn.GatedDeltaNet.forward(q, k, v, a, b, .{ .s = s }, .scalar(2, .i32)).outputs;
        }
    };
    const q: zml.Tensor = .init(.{ .s = 2, .h = 1, .k = 2 }, .f32);
    const k: zml.Tensor = .init(.{ .s = 2, .h = 1, .k = 2 }, .f32);
    const v: zml.Tensor = .init(.{ .s = 2, .h = 1, .v = 2 }, .f32);
    const a: zml.Tensor = .init(.{ .s = 2, .h = 1 }, .f32);
    const b: zml.Tensor = .init(.{ .s = 2, .h = 1 }, .f32);
    const s: zml.Tensor = .init(.{ .v = 2, .h = 1, .k = 2 }, .f32);
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ q, k, v, a, b, s }, platform, .{});
    exe.deinit();
}
```

M32 Integer mean over a long axis panics while tracing

MEDIUM ZML crash · Tensor · risk 4 (impact 2 × exposure 2)

API Tensor.mean (integer dtypes)
Location zml/tensor.zig:1773 (mean: divByConst(self.dim(axis_))), zml/tensor.zig:1217 (divByConst builds the scalar in self.dtype())
Platform CPU
KnownBug mean_int_len

What happens. A u8 axis of 300 or an i8 axis of 129 panics with "integer does not fit in destination type".

Root cause. The axis length is built as a scalar of the tensor's dtype through divByConst.

> **Risk assessment.** Crash on valid input.

Reproduction.
1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run bazel test //tools/zmlsmith/regressions:found_mean_int_len --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   mean_int_len: *fails* (matches integer does not fit in destination type)
   1/1 mean_int_len.test.mean_int_len...thread 3154692 panic: integer does not fit in destination type

Verification: confirmed, report corrected. **Expected behavior, from:** divByConst documents the scalar as self.dtype() (zml/tensor.zig:1217). mean takes the axis length as a Tensor scalar of that dtype, so an axis length that does not fit the integer dtype cannot be represented.
- zml/tensor.zig:1773: return self.sum(axis_).divByConst(self.dim(axis_));
- zml/tensor.zig:1217: return self.div(.scalar(b, self.dtype()));

Suggested fix. Divide in a wider type.

tools/zmlsmith/regressions/found/mean_int_len.zig (20 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Tensor.mean divides the sum by the axis length with divByConst, which builds the length
// as a scalar of the tensor's dtype (zml/tensor.zig:1783 and :1227 at 74c590b0). For an
// integer dtype whose max is below the length, e.g. a u8 axis of 300, tracing panics with
// "integer does not fit in destination type" in DataType.constant. Found by zmlsmith
// (reduce family, seed 20967).
test "mean_int_len" {
    const platform = zml.testing.env();
    const Local = struct {
        fn forward(x: zml.Tensor) zml.Tensor {
            return x.mean(.a);
        }
    };
    const x: zml.Tensor = .init(.{ .a = 300 }, .u8);
    var exe = try zml.module.compile(std.testing.allocator, std.testing.io, Local.forward, .{x}, platform, .{});
    exe.deinit();
}
```

M33 Reading back an empty sharded buffer panics

MEDIUM ZML crash · Sharding · risk 4 (impact 2 × exposure 2)

API Buffer (sharded)
Location zml/Sharding.zig:1862 (shardSlice), reached from Buffer.toSliceAlloc at zml/buffer.zig:315
Platform CPU, 4-device mesh
KnownBug sharded_empty

What happens. toSliceAlloc of a zero-element buffer sharded on an axis (i8 {k=0, c=4, p=1}, c on a 4-device axis) panics with integer overflow at zml/Sharding.zig:1862 instead of returning an empty slice.

Root cause. shardSlice computes the shard offset as the unsigned difference @intFromPtr(shard_ptr) - @intFromPtr(slice.bytes.ptr) (zml/Sharding.zig:1862). shardPtr (zml/Sharding.zig:1844-1856) starts from slice.constData().ptr and advances by start * stride; for a zero-size host slice the result is not at or after slice.bytes.ptr, and the subtraction underflows. The exact pointer values were not inspected.

> **Risk assessment.** Crash on a valid edge shape (an empty sharded tensor) when reading results back.

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run bazel test //tools/zmlsmith/regressions:found_shard_empty --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   shard_empty: *fails* (matches panic: integer overflow)
   thread 3159999 panic: integer overflow

Verification: confirmed, report corrected. **Expected behavior, from:** Debug builds panic on unsigned overflow; a sub-slice pointer must not lie below the base pointer.

- zml/Sharding.zig:1862: `const offset_bytes = @intFromPtr(shard_ptr) - @intFromPtr(slice.bytes.ptr);`
- zml/Sharding.zig:1851-1853: `for (pl.shape.dims(), pl.axis_plans.slice(), slice.byte_strides.slice()) |size, plan, stride| { const start = plan.linearIndex(device) * size; ptr = ptr[@intCast(start * stride)..]; }`
- zml/buffer.zig:324: `sub_slice.copy(shard_slice.constData());`

Suggested fix. Return early for zero-size slices in shardSlice / toSliceAlloc, or compute the offset from start * stride instead of from pointers.

tools/zmlsmith/regressions/found/shard_empty.zig (36 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 501933.
test "sharded_empty" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            return .{x0};
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_501933", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .k = 0, .c = 4, .p = 1 }, .i8).withPartitioning(.{ .c = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .k = 0, .c = 4, .p = 1 }, .i8), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{});
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{};
        const expected: zml.Slice = .init(zml.Shape.init(.{ .k = 0, .c = 4, .p = 1 }, .i8), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

M34 gather / scatter reject valid batching layouts

MEDIUM ZML crash · Tensor · risk 4 (impact 2 × exposure 2)

API	zml.ops.gather, Tensor.scatterSlices
Location	gather_batch: zml/ops.zig:2082-2098 (result shape assumes non-batching indices axes sit at the gathered axis), zml/ops.zig:2142 (assert). scatter_rank: zml/ops.zig:1907-1912 (untagged branch)
Platform	CPU
KnownBug	gather_batching_after, scatter_untagged_rank

What happens. gather: when a batching axis of self comes after the gathered axis, the result shape computed by ZML does not match the StableHLO result, and the assert at zml/ops.zig:2142 rejects a valid call. scatterSlices: with untagged index tensors (two rank-1 indices) and a rank-1 update, the untagged scatter config produces an update shape with one window dim per index tensor, which overflows (panic: integer overflow).

Root cause. gather: StableHLO orders the output's non-offset dims by the indices' batch dims in indices order (batching dims included), but ZML's res_shape (zml/ops.zig:2082-2089) inserts the non-batching index axes at the position of the first gathered axis. The two disagree when a batching axis follows the gathered axis. scatter: the untagged branch appends one window_id per index tensor (zml/ops.zig:1908-1912, using indices_per_axis.len) instead of one per batch dim of the indices, so the updates' window_id count does not match the update rank. The exact overflow site was not located statically.

Risk assessment. Crash on valid inputs.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_gather_batch --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_scatter_rank --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `gather_batch`: *fails* (matches `gather expects that batching indices appear in the same order`)
   `1/1 gather_batch.test.gather_batching_after...thread 3150270 panic: gather expects that batching indices appear in the same order in 'self' and 'indices', got: self=Tensor({e=5,f=4/data,c=1,m=1,u32}), indices=Tensor({e=5`
   `scatter_rank`: *fails* (matches `integer overflow`)
   `1/1 scatter_rank.test.scatter_untagged_rank...thread 3159072 panic: integer overflow`

Verification: confirmed, report corrected. **Expected behavior, from:** gather doc (zml/tensor.zig:2815-2823) describes batching axes by tag and says batching requires tags. The assert message (zml/ops.zig:2142) only requires that batching indices appear in the same order in self and indices; in the reproducer they do (e then m in both), so the rejection contradicts the assert's own criterion. The scatter reproducer is an untagged point update, whose update shape should equal the indices batch shape {1}.

- zml/ops.zig:2082-2089: for (0.., indices_shape.tags(), indices_shape.dims()) |...| { ... res_shape = res_shape.appendDim(id_inserted_dim, id_axis); ... }
- zml/ops.zig:2142: stdx.debug.assert(mlir_shape.eql(res_shape), "gather expects that batching indices appear in the same order ...")
- zml/ops.zig:1908-1912: for (0..indices_per_axis.len) |i| { op_kind...inserted_window; scatter_to_operand_axes...; up_kind.appendAssumeCapacity(.window_id); }

Suggested fix. gather: transpose the gather result (or the indices) into ZML's layout, or compute res_shape in StableHLO's order. scatter: derive the window_id count from the indices batch rank, not from the number of index tensors.

`tools/zmlsmith/regressions/found/gather_batch.zig` (51 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500241.
test "gather_batching_after" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor, x1: zml.Tensor) [1]zml.Tensor {
            const x2 = x0.gather(.{ .f = x1 }, .{});
            _ = &x2;
            return .{x2};
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_500241", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .e = 5, .f = 4, .c = 1, .m = 1 }, .u32).withPartitioning(.{ .f = .data }));
    const x1: zml.Tensor = .fromShape(zml.Shape.init(.{ 5, 1, 2 }, .i8).withTags(.{ .e, .m, ._ }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x0, x1 }, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ 5, 2, 1, 1 }, .u32).withTags(.{ .e, ._, .c, .m }), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var b1: zml.Buffer = try .fromBytes(io, platform, x1.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b1.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ b0, b1 });
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ 5, 2, 1, 1 }, .u32).withTags(.{ .e, ._, .c, .m }), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

`tools/zmlsmith/regressions/found/scatter_rank.zig` (51 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500049.
test "scatter_untagged_rank" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
```

```
    const Local = struct {
        fn forward(x0: zml.Tensor, x1: zml.Tensor, x2: zml.Tensor) [1]zml.Tensor {
            const x3 = x0.scatterSlices(@as([]const zml.Tensor, &.{ x1, x1 }), x2, .{ .update_fn = zml.Tensor.ScatterOpts.increment, .indices_are_unique =
false });
            _ = &x3;
            return .{x3};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .b = 5, .h = 2 }, .f32));
    const x1: zml.Tensor = .fromShape(zml.Shape.init(.{1}, .i64));
    const x2: zml.Tensor = .fromShape(zml.Shape.init(.{1}, .f32));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x0, x1, x2 }, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .b = 5, .h = 2 }, .f32), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var b1: zml.Buffer = try .fromBytes(io, platform, x1.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b1.deinit();
    var b2: zml.Buffer = try .fromBytes(io, platform, x2.shape(), .replicated, &[_]u8{
        0, 0, 0, 0,
    });
    defer b2.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ b0, b1, b2 });
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .b = 5, .h = 2 }, .f32), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .exact_match);
    }
}
```

M35 Bicubic resize of axes shorter than 4 asserts

MEDIUM ZML crash · nn · risk 4 (impact 2 × exposure 2)

API nn.resizeCubic1d / Bicubic
Location `zml/nn.zig:1539-1543 (gatherSlices with slice size 4), assertion at zml/tensor.zig:2961`
Platform CPU
KnownBug `cubic_small_axis`

What happens. gatherSlices asserts (4 neighbors).

Root cause. neighbors use a slice of 4 along _neighbors in gatherSlices (zml/nn.zig:1539-1543). gatherSlices asserts slice size <= axis length (zml/tensor.zig:2961), so any resized axis shorter than 4 trips the assertion. The maximum(0) clamp at zml/nn.zig:1537 only fixes the lower index, not the slice length.

> **Risk assessment.** Crash on small images.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_cubic_small --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

   `cubic_small`: *fails* (matches `gatherSlices expects `slice_shape` to be smaller than `self.shape\(\)``)

   ```
   1/1 cubic_small.test.cubic_small_axis...thread 3151430 panic: gatherSlices expects `slice_shape` to be smaller than `self.shape()`.
   On axis _neighbors, got {_neighbors=4,f32} > {h=4/data,_neighbors=1,c=3,f32}.
   ```

Verification: confirmed, report corrected. **Expected behavior, from:** Assertion text in zml/tensor.zig:2961 is the contract for gatherSlices. A bicubic resize to or from an axis with fewer than 4 samples has no defined value in the code, so the crash is a missing-check issue.

- zml/tensor.zig:2961: stdx.debug.assert(slice_dim <= self._shape.dim(self_ax), "gatherSlices expects `slice_shape` to be smaller than `self.shape()`. On axis {s}, got {f} > {f}."
- zml/nn.zig:1537: const neighbors = scaled.floor().addConstant(-1).convert(.i32).maximum(Tensor.scalar(0, .i32));

Suggested fix. Clamp the neighbor indices.

`tools/zmlsmith/regressions/found/cubic_small.zig` (46 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 500318.
test "cubic_small_axis" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = zml.nn.resizeCubicId(x0, 1, 3, .{});
            _ = &x1;
            return .{x1};
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_500318", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .h = 4, .f = 1, .c = 3 }, .f32).withPartitioning(.{ .h = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .h = 4, .f = 3, .c = 3 }, .f32), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .h = 4, .f = 3, .c = 3 }, .f32), @as([]const u8, &expected_data));
        try zml.testing.expectClose(io, expected, res[0], .{ .absolute_tolerance = 3.0517578125e-5, .relative_tolerance =
2.384185791015625e-7, .minimum_close_fraction = 1 });
    }
}
```

M36 GSPMD: partitionId inside manualComputation doesn't compile

MEDIUM ZML compile · Sharding · risk 4 (impact 2 × exposure 2)

API	Compiler.Options.partitioner = .gspmd
Location	`zml/ops.zig:196-205 (partitionId emits a bare stablehlo.partition_id with no mhlo.sharding), zml/ops.zig:3231-3233 (the {manual} annotation for gspmd is applied only to custom calls)`
Platform	CPU, 4-device mesh
KnownBug	`gspmd_partition_id`

What happens. Compiling with `.partitioner = .gspmd` fails: PJRT compile returns error.Unimplemented. The same program compiles with the default Shardy partitioner (the reproducer compiles both).

Root cause. partitionId creates a stablehlo.partition_id op with no sharding (zml/ops.zig:196-205), and the manualComputation body is only closed by an SPMDShardToFullShape: the module printed by the failing run has no SPMDFullToShardShape before the partition_id, because the body does not use its input. For GSPMD the partition_id is therefore outside any manual region. ZML marks `{manual}` only on custom calls it creates (zml/ops.zig:3231-3233).

> **Risk assessment.** Only with partitioner = .gspmd. The reproducer's region is not dead code: its result is the function output.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_gspmd_part_id --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

   `gspmd_part_id`: *fails* (matches `found/gspmd_part_id\.zig:29:`)

   ```
   /home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/5273/execroot/_main/
   tools/zmlsmith/regressions/found/gspmd_part_id.zig:29:15: 0x6069ebf2b0ee in test.gspmd_part_
   ```

Verification: confirmed, report corrected. **Expected behavior, from:** The partition-id instruction is ambiguous without a manual context under GSPMD (the XLA message quoted in the reproducer). ZML's own GSPMD manual lowering (zml/ops.zig:2687-2700) is the reference for what ZML marks.

- zml/ops.zig:198: const op = mlir.Operation.make(ctx.mlir_ctx, "stablehlo.partition_id", .{
- zml/ops.zig:3231: if (ctx.manual_computation_depth > 0 and ctx.partitioning.partitioner == .gspmd) {

- zml/ops.zig:3232: op.setAttributeByName("mhlo.sharding", .string(ctx.mlir_ctx, "{manual}"));

Suggested fix. Mark the region manual for GSPMD.

tools/zmlsmith/regressions/found/gspmd_part_id.zig (32 lines)

```
const std = @import("std");
const zml = @import("zml");

// With Compiler.Options.partitioner = .gspmd, a manualComputation whose body calls
// ops.partitionId doesn't compile (PJRT returns error.Unimplemented; XLA's message is
// "PartitionId instruction is not supported for SPMD partitioning since the meaning is
// ambiguous"). The emitted module has no SPMDFullToShardShape before the partition_id, so
// the region is not marked manual in the GSPMD lowering. The same program compiles with
// the default Shardy partitioner. Found by zmlsmith with --gspmd (seed 150209).
test "gspmd_part_id" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const Local = struct {
        fn body(x: zml.Tensor, out: zml.Shape) zml.Tensor {
            _ = x;
            return zml.ops.partitionId().convert(.i32).broad(out);
        }
        fn forward(x: zml.Tensor) zml.Tensor {
            const xs = x.withPartitioning(.{ .a = .data });
            return zml.ops.manualComputation(body, xs, xs.shape().withDtype(.i32)).withPartitioning(.{ .a = .replicated });
        }
    };
    const mesh = try @constCast(platform).registerSharding("gspmd_part_id", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .a = 4 }, .f32));
    // Shardy compiles it.
    var shardy = try zml.module.compile(allocator, io, Local.forward, .{x}, platform, .{ .shardings = &.{mesh} });
    shardy.deinit();
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x}, platform, .{ .shardings = &.{mesh}, .partitioner = .gspmd });
    exe.deinit();
}
```

M37 maxPool indices are wrong when the max is the dtype minimum

MEDIUM ZML wrong values · Tensor · risk 4 (impact 2 × exposure 2)

API	Tensor.maxPool1d/2d
Location	zml/tensor.zig:3693-3696 (reduceWindow init: .constant(minValue) and index init .scalar(0, .i32)), zml/tensor.zig:3399-3403 (tie rule keeps the smallest index)
Platform	CPU
KnownBug	pool_min_index

What happens. The returned index points at the init value.

Root cause. The index reduction starts from init index 0 (zml/tensor.zig:3696). The tie rule in the reducer keeps the smaller index on equal values (zml/tensor.zig:3399-3403). When a window element equals dtype minValue, the init pair (minValue, 0) ties with it and wins, so the returned index is 0 instead of the element's index.

> **Risk assessment.** Silent but rare (windows full of the dtype minimum, e.g. -inf masks).

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run bazel test //tools/zmlsmith/regressions:found_pool_min_index --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:

   pool_min_index: *fails* (matches found/pool_min_index\.zig:42:)

   ```
   /home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/4723/execroot/_main/
   tools/zmlsmith/regressions/found/pool_min_index.zig:42:9: 0x64d98e2029cc in test.pool_min_in
   ```

Verification: confirmed, report corrected. **Expected behavior, from:** maxPool with window 1 and dilation 2 selects each element, so the expected index per window is that element's own index. The tie rule is documented in the code comment at zml/tensor.zig:3399.

- zml/tensor.zig:3696: .{ .constant(self.dtype().minValue()), .scalar(0, .i32) },
- zml/tensor.zig:3399: // If values.left == values.right: keep the smallest idx.

Suggested fix. Use an index sentinel that loses ties.

tools/zmlsmith/regressions/found/pool_min_index.zig (45 lines)

```
const std = @import("std");
const zml = @import("zml");
```

```
// Generated by zmlsmith from seed 800369.
test "pool_min_index" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const x1 = x0.maxPool1d(.{ .window_dimensions = 1, .window_strides = null, .base_dilations = 1, .window_dilations = 2, .padding = .{ 0,
0 } }).indices;
            _ = &x1;
            return .{x1};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .k = 6, .n = 2 }, .u64));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .k = 6, .n = 2 }, .i32), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    });
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
            0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .k = 6, .n = 2 }, .i32), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

M38 Rank-0 Slice views print the first element of the buffer

MEDIUM ZML wrong values · Host · risk 4 (impact 2 × exposure 2)

API Slice print (rank 0)
Location `zml/slice.zig:259-261 (rank-0 branch; the offending read is zml/slice.zig:260)`
Platform CPU

What happens. A scalar view taken with subSlice + squeeze prints 10 instead of 30.

Root cause. The rank-0 path reads `bytes.ptr` and ignores `offset_bytes`.

> **Risk assessment.** Misleading debug output.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_slice_print --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `slice_print`: *fails* (matches `\^ \('\\x31'\)`)
   `^ ('\x31')`

Verification: confirmed as reported. **Expected behavior, from:** Slice.constData() (zml/slice.zig:143-146) applies offset_bytes; the rank-0 path must read through the same offset. subSlice sets offset_bytes (zml/slice.zig:119-123).

- zml/slice.zig:260: `const single_ptr: *const dt.toZigType() = @ptrCast(@alignCast(slice.bytes.ptr));`
- zml/slice.zig:146: `return slice.bytes[slice.offset_bytes..end];`

Suggested fix. Apply the offset.

`tools/zmlsmith/regressions/found/slice_print.zig` (16 lines)

```
const std = @import("std");
const zml = @import("zml");

// Slice.prettyPrint of a rank-0 view reads `bytes.ptr` and ignores `offset_bytes`, so a
// scalar taken from a larger slice with subSlice + squeeze prints the first element of
// the buffer (zml/slice.zig:259 at 74c590b0). Found by auditing zmlsmith coverage of
// zml.Slice.
test "slice_print" {
    var data = [_]f32{ 10, 20, 30 };
    const s: zml.Slice = .init(zml.Shape.init(.{3}, .f32), std.mem.sliceAsBytes(&data));
    const last = s.subSlice(0, 2, 1).squeeze(0);
```

```
    const txt = try std.fmt.allocPrint(std.testing.allocator, "{d}", .{last});
    defer std.testing.allocator.free(txt);
    try std.testing.expectEqualStrings("30", txt);
}
```

M39 leakyReLU is wrong for negative slopes

MEDIUM ZML wrong values · Tensor · risk 3 (impact 3 × exposure 1)

API	Tensor.leakyReLU
Location	`zml/tensor.zig:1593`
Platform	CPU
KnownBug	`leaky_negative_slope`

What happens. leakyReLU(-0.5) of [-2, 2] gives [0, 1] instead of [1, 2].

Root cause. Tensor.leakyReLU computes relu(x) + min(slope*x, 0) (tensor.zig:1593-1594), which equals max(0,x)+slope*min(0,x) only for slope >= 0. For a negative slope the min(slope*x,0) term is taken on the wrong side.

Risk assessment. Silent; positive slopes, the usual case, are right.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_leaky_neg --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:

```
leaky_neg: fails (matches found/leaky_neg\.zig:28:)
/home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/4659/execroot/_main/
tools/zmlsmith/regressions/found/leaky_neg.zig:28:5: 0x60d3d96cff14 in test.leaky_neg (tools
```

Verification: confirmed, report corrected. **Expected behavior, from:** The doc comment at zml/tensor.zig:1589-1591: LeakyReLU(x) = max(0,x) + negative_slope * min(0,x). Hand check: x=-2, slope=-0.5 gives 0 (code) vs 1 (formula); x=2 gives 1 (code) vs 2 (formula).

- zml/tensor.zig:1592: pub fn leakyReLU(self: Tensor, negative_slope: f32) Tensor {
- zml/tensor.zig:1593: const below_zero = self.scale(negative_slope).minimum(.scalar(0, self.dtype()));
- zml/tensor.zig:1594: return self.relu().add(below_zero);

Suggested fix. Implement the documented formula.

tools/zmlsmith/regressions/found/leaky_neg.zig (30 lines)

```
const std = @import("std");
const zml = @import("zml");

// Tensor.leakyReLU computes relu(x) + min(slope * x, 0). Its doc formula is
// max(0, x) + slope * min(0, x): the two differ for a negative slope.
test "leaky_neg" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x: zml.Tensor) zml.Tensor {
            return x.leakyReLU(-0.5);
        }
    };
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{ .a = 2 }, .f32));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x}, platform, .{});
    defer exe.deinit();

    const in = [_]f32{ -2, 2 };
    var b: zml.Buffer = try .fromBytes(io, platform, x.shape(), .replicated, std.mem.sliceAsBytes(&in));
    defer b.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b});
    defer res.deinit();
    const got = try res.toSliceAlloc(allocator, io);
    defer got.free(allocator);
    // max(0, -2) - 0.5 * min(0, -2) = 1, max(0, 2) - 0.5 * min(0, 2) = 2.
    try std.testing.expectEqualSlices(f32, &.{ 1, 2 }, got.items(f32));
}
```

M40 Simplifier rewrites ignore IEEE zeros: -0 + 0, and convert(bool) * x with x = -3, inf or NaN

MEDIUM upstream XLA wrong values · XLA · risk 3 (impact 1 × exposure 3)

API	addConstant(0), convert(bool) * negative

Location XLA algebraic simplifier
Platform CPU
KnownBug xla_add_zero_sign, xla_bool_mul_sign

What happens. With a constant zero, x + 0 is simplified to x, so -0 + 0 is -0 instead of +0. Mul(convert(bool), x) is rewritten to select(pred, x, 0): for false, convert(bool) * -3 gives +0 instead of -0, and for false * inf or false * NaN the result is 0 instead of NaN. The NaN/inf case is the larger effect.

Root cause. Simplifier rewrites that ignore the sign of zero.

> **Risk assessment.** Sign-of-zero differences only matter to code that inspects zero signs (1/x, atan2, bit-exact references). The masking pattern mask(bool) * x with x = inf or NaN gives 0 instead of NaN, which hides NaN propagation in masked code paths.

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run bazel test //tools/zmlsmith/regressions:found_xla_bool_mul --test_output=errors, then bazel test //tools/zmlsmith/regressions:found_add_zero_sign --test_output=errors, then bazel test //tools/zmlsmith/regressions:found_bool_mul_inf --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:

```
xla_bool_mul: fails (matches got \{ 0, -3 \})
got { 0, -3 }
add_zero_sign: fails (matches found/add_zero_sign\.zig:28:)
/home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/5275/execroot/_main/
tools/zmlsmith/regressions/found/add_zero_sign.zig:28:5: 0x5d7fcdc79128 in test.add_zero_sig
bool_mul_inf: fails (matches found/bool_mul_inf\.zig:28:)
/home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/5304/execroot/_main/
tools/zmlsmith/regressions/found/bool_mul_inf.zig:28:5: 0x5921964bc080 in test.bool_mul_inf
```

Verification: confirmed, report corrected. **Expected behavior, from:** StableHLO multiply for floats is IEEE-754 multiplication (docs/spec.md:4458) and add is IEEE-754 addition (docs/spec.md:639), so 0 * inf = NaN and -0 + +0 = +0. XLA's rewrites are unguarded: 'A + 0 => A' (algebraic_simplifier.cc:875-877) and 'Mul(Convert(Pred), operand) => select(pred, operand, 0)' (algebraic_simplifier.cc:5008-5014).

- xla/hlo/transforms/simplifiers/algebraic_simplifier.cc:875-877: '// A + 0 => A' ... 'if (IsAll(rhs, 0) && ReplaceInstructionIfCompatible(add, lhs))'
- xla/hlo/transforms/simplifiers/algebraic_simplifier.cc:5008: '// Mul(Convert(Pred), operand) => select(pred, operand, 0)'
- docs/spec.md:4458: '* For floats: multiplication from IEEE-754.'
- tools/zmlsmith/regressions/found/xla_bool_mul.zig: 'try std.testing.expect(v[0] == 0 and std.math.signbit(v[0]));'

Suggested fix. Report upstream with xla_bool_mul and the new bool_mul_inf and add_zero_sign reproducers. ZML-side: none that does not change the program semantics (a select-based mask in ZML would already match XLA's output).

tools/zmlsmith/regressions/found/xla_bool_mul.zig (30 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Upstream XLA CPU: convert(bool) times a negative constant gives +0 for false, where
// IEEE multiplication gives 0 * -3 = -0; XLA probably rewrites the product of a converted
// predicate into a select with a literal 0. Found by zmlsmith (seed 310246).
test "xla_bool_mul" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const Local = struct {
        fn f(x: zml.Tensor) zml.Tensor {
            return x.convert(.f64).scale(-3);
        }
    };
    const x: zml.Tensor = .init(.{2}, .bool);
    var exe = try platform.compileFn(allocator, io, Local.f, .{x}, .{});
    defer exe.deinit();
    var b = try zml.Buffer.fromBytes(io, platform, x.shape(), .replicated, &.{ 0, 1 });
    defer b.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.f, .{b});
    defer res.deinit();
    const got = try res.toSliceAlloc(allocator, io);
    defer got.free(allocator);
    const v = got.constItems(f64);
    std.debug.print("got {any}\n", .{v});
    try std.testing.expect(v[0] == 0 and std.math.signbit(v[0]));
    try std.testing.expectEqual(@as(f64, -3), v[1]);
}
```

tools/zmlsmith/regressions/found/add_zero_sign.zig (31 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// IEEE-754 addition gives -0 + +0 = +0 (StableHLO add is IEEE addition, docs/spec.md "### add").
// XLA's algebraic simplifier rewrites A + 0 => A unconditionally (algebraic_simplifier.cc
// HandleAdd, not gated by fast math), so the sign of a negative zero input is kept.
test "add_zero_sign" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x: zml.Tensor) zml.Tensor {
            return x.addConstant(0);
        }
    };
    const x: zml.Tensor = .init(.{2}, .f32);
    var exe = try platform.compileFn(allocator, io, Local.forward, .{x}, .{});
    defer exe.deinit();

    var b: zml.Buffer = try .fromBytes(io, platform, x.shape(), .replicated, std.mem.sliceAsBytes(&[_]f32{ -0.0, 1.0 }));
    defer b.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b});
    defer res.deinit();
    const got = try res.toSliceAlloc(allocator, io);
    defer got.free(allocator);
    const v = got.constItems(f32);
    try std.testing.expect(!std.math.signbit(v[0]));
    try std.testing.expectEqual(@as(f32, 1.0), v[1]);
}
```

tools/zmlsmith/regressions/found/bool_mul_inf.zig (31 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// IEEE-754 multiplication gives 0 * inf = NaN (StableHLO multiply is IEEE multiplication,
// docs/spec.md "### multiply"). XLA's algebraic simplifier rewrites Mul(Convert(pred), x)
// into select(pred, x, 0) unconditionally (algebraic_simplifier.cc HandleMultiply, the
// "Mul(Convert(Pred), operand)" rule), so false * inf is 0 and false * NaN is 0.
test "bool_mul_inf" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn f(x: zml.Tensor) zml.Tensor {
            return x.convert(.f32).scale(std.math.inf(f32));
        }
    };
    const x: zml.Tensor = .init(.{2}, .bool);
    var exe = try platform.compileFn(allocator, io, Local.f, .{x}, .{});
    defer exe.deinit();
    var b: zml.Buffer = try .fromBytes(io, platform, x.shape(), .replicated, &.{ 0, 1 });
    defer b.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.f, .{b});
    defer res.deinit();
    const got = try res.toSliceAlloc(allocator, io);
    defer got.free(allocator);
    const v = got.constItems(f32);
    try std.testing.expect(std.math.isNan(v[0]));
    try std.testing.expect(std.math.isInf(v[1]));
}
```

7 Low findings

L01 Constant additions are reassociated

LOW upstream XLA wrong values · XLA CPU · risk 8 (impact 2 × exposure 4)

API subConstant / addConstant chains (f32)
Location XLA algebraic simplifier
Platform CPU
KnownBug xla_const_reassoc

What happens. `(x - 0.5) + 0.5` returns x bit for bit, and `(10000001 - 0.5) + 0.25` gives 10000001 instead of 10000000.

Root cause. `x.subConstant(c1).addConstant(c2)` is rounded once as `x + (c2 - c1)`; StableHLO rounds after each op.

Risk assessment. One rounding less than StableHLO prescribes (values differ by 1 ulp at most in the examples). Visible only to bit-exact references or code that relies on rounding (for example the 1.5*2^23 round-to-integer trick). The rewrite is not behind a fast-math flag.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_xla_const_fold --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `xla_const_fold`: *fails* (matches `expected 0\.099999994, found 0\.1`)
   `expected 0.099999994, found 0.1`

Verification: confirmed, report corrected. **Expected behavior, from:** StableHLO add is IEEE-754 addition (docs/spec.md:639), so each op rounds. XLA's rule (A + C1) + C2 => A + (C1 + C2) has no fast-math check (algebraic_simplifier.cc:895-906; the comment there says it is a deliberate constant-folding reassociation with the limitation 'This is not general').

- xla/hlo/transforms/simplifiers/algebraic_simplifier.cc:895-896: '// Reassociate to allow constant folding.'
- xla/hlo/transforms/simplifiers/algebraic_simplifier.cc:901-902: 'VLOG(10) << "trying transform [(A + C1) + C2 => A + (C1 + C2)]";'
- docs/spec.md:639: '* For floats: `addition` from IEEE-754.'

Suggested fix. Report upstream: the reassociation is an ungated rewrite in XLA's algebraic simplifier. No ZML-side flag disables it.

tools/zmlsmith/regressions/found/xla_const_fold.zig (41 lines)

```
const std = @import("std");
const zml = @import("zml");

// Upstream XLA CPU (probably the algebraic simplifier): chained float additions of
// constants are folded, `x.subConstant(c1).addConstant(c2)` is computed as
// `x + (c2 - c1)` with one rounding instead of two, so `(x - 0.5) + 0.5` returns x
// bit for bit. StableHLO rounds after each op. Found by zmlsmith (seed 120975).
fn check(c1: f32, c2: f32) !void {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;
    const Local = struct {
        fn forward(x: zml.Tensor, a: f32, b: f32) zml.Tensor {
            return x.subConstant(a).addConstant(b);
        }
    };
    const x: zml.Tensor = .init(.{4}, .f32);
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x, c1, c2 }, platform, .{});
    defer exe.deinit();
    const xs = [_]f32{ -1.5498009, 0.1, 1.0e7 + 1, 3.3333333 };
    var b: zml.Buffer = try .fromBytes(io, platform, x.shape(), .replicated, std.mem.sliceAsBytes(&xs));
    defer b.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b});
    defer res.deinit();
    const got = try res.toSliceAlloc(allocator, io);
    defer got.free(allocator);
    for (xs, got.constItems(f32)) |v, g| {
        const want = (v - c1) + c2;
        if (want != g) std.debug.print("c1={d} c2={d} x={d}: want {d}, got {d}\n", .{ c1, c2, v, want, g });
        try std.testing.expectEqual(want, g);
    }
}

test "xla_const_fold_cancel" {
    try check(0.5, 0.5);
}

test "xla_const_fold_other" {
    try check(0.5, 0.25);
}
```

L02 reuseBuffer doesn't check that input and output share a sharding

LOW ZML compile · Sharding · risk 4 (impact 2 × exposure 2)

API Tensor.reuseBuffer
Location zml/tensor.zig:361-373 (reuseBuffer records the donation without comparing sharding)
Platform CPU, 4-device mesh

What happens. A sharded input aliased to a replicated output fails in XLA ("aliased input ... same size").

Root cause. No sharding check when recording the alias.

> **Risk assessment.** Late, unclear compile error.

Reproduction.

1. Get ZML at 74c590b0 and add tools/zmlsmith/ to the tree (it is not committed).
2. Run bazel test //tools/zmlsmith/regressions:found_reuse_sharding --test_output=errors.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at 74c590b0 the log matched the pattern given for each reproducer:
   reuse_sharding: *fails* (matches aliased input 1 at index)
   ```
   [pjrt] (warn): [PJRT_Client_Compile] Internal: Expected aliased input 1 at index {} and output at index {} to have the same size.
   Input sub-shape is u8[1]{0} with size 1, output sub-shape is u8[4]{0} with size 4
   ```

Verification: confirmed, report corrected. **Expected behavior, from:** The reuseBuffer doc (zml/tensor.zig:354-360) says nothing about sharding, so there is no documented contract for a sharding-mismatched alias. The rejection itself comes from XLA; ZML does not check it.

- zml/tensor.zig:368-370: scope.id_to_donation.put(scope.arena.allocator(), self.id, donation) catch @panic("OOM");
- zml/tensor.zig:372: return self;

Suggested fix. Check and report in ZML.

tools/zmlsmith/regressions/found/reuse_sharding.zig (49 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 800494.
test "reuse_buffer_sharding" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor, x2: zml.Tensor) [1]zml.Tensor {
            const x1 = x0.bytes();
            _ = &x1;
            const x3 = x1.reuseBuffer(x2);
            _ = &x3;
            return .{x3};
        }
    };

    const mesh = try @constCast(platform).registerSharding("zmlsmith_800494", .mesh(.{ .data = .low_bandwidth, .model = .high_bandwidth }));
    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .m = 1 }, .i32));
    const x2: zml.Tensor = .fromShape(zml.Shape.init(.{ .bytes = 4 }, .u8).withPartitioning(.{ .bytes = .data }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x0, x2 }, platform, .{ .shardings = &.{mesh} });
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ .bytes = 4 }, .u8), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), mesh, &[_]u8{
        0, 0, 0, 0,
    });
    defer b0.deinit();
    var b2: zml.Buffer = try .fromBytes(io, platform, x2.shape(), mesh, &[_]u8{
        0, 0, 0, 0,
    });
    defer b2.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ b0, b2 });
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ .bytes = 4 }, .u8), @as([]const u8, &expected_data));
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

L03 Shape helpers: reshape(-1) of an empty shape, and ._N past the rank

LOW ZML crash · Host · risk 4 (impact 2 × exposure 2)

API Shape.reshape, axisFromTagMaybe, axisFromInt
Location zml/shape.zig:594 (inferMissingAxis: `if (tmp_count > n) return error.TooBig;`, reached by reshape(-1) on a zero-size input); zml/shape.zig:437 (axisFromInt: `if (a < -rk or a > rk)`); zml/shape.zig:449 and :470-474 (axisFromTagMaybe / axisFromLiteralInt: `._N` returned without a rank check)
Platform CPU

What happens. reshape(-1) on a shape with a zero dim fails with TooBig (`Can't reshape { 0, 3 } to { -1 }: TooBig`) where numpy gives {0}. hasTag(._5) on a rank-2 shape returns 5 instead of null. (axisFromInt also accepts a == rank, zml/shape.zig:437, but that path has no standalone reproducer.)

Root cause. reshape(-1): inferMissingAxis compares the product of the explicit target dims against the input count; with a zero-size input (n = 0) the product 1 is > 0, so it reports TooBig instead of inferring 0. axisFromInt accepts a == rank (`a > rk` should be `a >= rk` for non-negative a), so axis(rank) returns an index past the dims. hasTag(._N) returns N regardless of rank because axisFromLiteralInt (shape.zig:470-474) is never checked against the shape.

> **Risk assessment.** Empty-tensor reshapes are rejected with a misleading error; out-of-range axes reach later code (index past dims) instead of a clear error.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_reshape_zero --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_literal_axis --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `reshape_zero`: *fails* (matches `Can't reshape \{ 0, 3 \} to \{ -1 \}: TooBig`)
   `1/1 reshape_zero.test.Shape.reshape(-1) with a zero dim...thread 3160572 panic: Can't reshape { 0, 3 } to { -1 }: TooBig`
   `literal_axis`: *fails* (matches `expected null, found 5`)
   `1/1 literal_axis.test.literal_axis...expected null, found 5`

Verification: confirmed, report corrected. **Expected behavior, from:** numpy: np.zeros((0, 3)).reshape(-1).shape == (0,), so -1 is inferred as 0 when the input has no elements (the target product is 1 and the input count 0 is divisible by it). hasTag returns ?u3 and is documented by its name as null when the tag is absent; a `._5` tag on a rank-2 shape is absent.

- zml/shape.zig:594: `if (tmp_count > n) return error.TooBig;`
- zml/shape.zig:437-438: `if (a < -rk or a > rk) { stdx.debug.panic("Tensor {f} doesn't have dimension: {d}", ...`
- zml/shape.zig:449: `if (axisFromLiteralInt(t)) |ax| return ax;`

Suggested fix. Add the checks.

tools/zmlsmith/regressions/found/reshape_zero.zig (8 lines)

```zig
const std = @import("std");
const zml = @import("zml");

test "Shape.reshape(-1) with a zero dim" {
    const sh: zml.Shape = .init(.{ 0, 3 }, .f32);
    try zml.testing.expectEqualShapes(zml.Shape.init(.{0}, .f32), sh.reshape(.{-1}));
}
```

tools/zmlsmith/regressions/found/literal_axis.zig (13 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// Shape.axisFromTagMaybe resolves the `._N` tag syntax to axis N without checking it
// against the rank: hasTag(._5) on a rank 2 shape returns 5 instead of null, and APIs that
// take the tag as an axis (sum, slice, gather, ...) index past the dims instead of
// reporting a missing axis. Found by zmlsmith host checks.
test "literal_axis" {
    const sh: zml.Shape = .init(.{ .a = 2, .b = 3 }, .f32);
    try std.testing.expectEqual(@as(?u3, 1), sh.hasTag(._1));
    try std.testing.expectEqual(@as(?u3, null), sh.hasTag(._5));
}
```

L04 DataType.Value.as and mantissaSize miss f8 / f4 / 2-bit dtypes

LOW ZML crash · Host · risk 4 (impact 2 × exposure 2)

API DataType.Value.as, mantissaSize; Rng.uniform
Location zml/dtype.zig:355-370 (DataType.Value.as switch), zml/dtype.zig:375-384 (mantissaSize), zml/tensor.zig:767 (Rng.uniform calls mantissaSize).

Platform CPU

What happens. DataType.Value.as panics ('Unsupported conversion') for f8e3m4, f8e4m3, f8e8m0, f4e2m1, i2 and u2, because those tags have no arm in its switch. Rng.uniform panics for every float dtype that mantissaSize does not list: f8e3m4, f8e4m3, f8e8m0 and f4e2m1 (mantissaSize lists only f8e4m3b11fnuz, f8e4m3fn, f8e4m3fnuz, f8e5m2, f8e5m2fnuz, f16, bf16, f32 and f64). The panic message 'Can't get mantissa size for a non-float dtype' is misleading for these float types.

Root cause. Incomplete switches.

> **Risk assessment.** Crash on valid dtypes.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_uniform_f8e4m3 --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_value_as_f8 --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `uniform_f8e4m3`: *fails* (matches `Can't get mantissa size for a non-float dtype`)
   `1/1 uniform_f8e4m3.test.uniform_f8e4m3...thread 3175652 panic: Can't get mantissa size for a non-float dtype`
   `value_as_f8`: *fails* (matches `Unsupported conversion .*f32`)
   `1/1 value_as_f8.test.value_as_f8...thread 3174701 panic: Unsupported conversion .f8e4m3 -> f32`

Verification: confirmed, report corrected. **Expected behavior, from:** uniform is documented as sampling 'floating point numbers from an interval' for any float tensor (zml/tensor.zig:766-769), so every float dtype is a valid target. as() is documented with 'TODO allow more lossless conversions' (dtype.zig:356) and is expected to convert values that are exactly representable in f32.

- zml/dtype.zig:375-384: 'pub fn mantissaSize(dtype: DataType) usize { return switch (dtype) { .f8e4m3b11fnuz, .f8e4m3fn, .f8e4m3fnuz => 3, .f8e5m2, .f8e5m2fnuz => 2, ... else => @panic("Can't get mantissa size for a non-float dtype") } }'
- zml/tensor.zig:767: 'const mantissa_bit_count = @import("dtype.zig").mantissaSize(dt);'
- zml/dtype.zig:370: 'std.debug.panic("Unsupported conversion {} -> {s}", .{ self.dtype(), @typeName(T) });'

Suggested fix. Complete the switches.

`tools/zmlsmith/regressions/found/uniform_f8e4m3.zig` (36 lines)

```
const std = @import("std");
const zml = @import("zml");

// Rng.uniform on an f8e4m3 (non-FN) tensor panics: mantissaSize (zml/dtype.zig:375) has no
// case for .f8e4m3, so the bit-masking step cannot pick a mantissa width. Every float dtype
// is a valid uniform() target, and the samples must lie in [min, max).
test "uniform_f8e4m3" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor) [1]zml.Tensor {
            const r: zml.Tensor.Rng = .{ ._state = x0 };
            _, const x = r.uniform(zml.Shape.init(.{ .h = 4 }, .f8e4m3), .{ .min = 0, .max = 1 });
            return .{x};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{2}, .u64));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x0}, platform, .{});
    defer exe.deinit();

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{0} ** 16);
    defer b0.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{b0});
    defer for (&res) |*r| r.deinit();

    const got = try res[0].toSliceAlloc(allocator, io);
    defer got.free(allocator);
    for (got.constItems(u8)) |bits| {
        const v = zml.floats.Float8E4M3.toF32(@bitCast(bits));
        try std.testing.expect(v >= 0 and v <= 1);
    }
}
```

`tools/zmlsmith/regressions/found/value_as_f8.zig` (11 lines)

```
const std = @import("std");
const zml = @import("zml");

// DataType.Value.as converts a value to a host type. Its switch (zml/dtype.zig:355-370) has
// no f8e4m3 arm, so as(f32) on an f8e4m3 value panics with "Unsupported conversion" even
// though the value is representable in f32. Expected: 1.0 round-trips.
test "value_as_f8" {
```

```
    const v = zml.DataType.Value.init(.f8e4m3, 1.0);
    try std.testing.expectEqual(@as(f32, 1.0), v.as(f32));
}
```

L05 zml.testing.expectClose panics on bool slices and on identical f4e2m1, f8e3m4, f8e4m3, f8e8m0 slices

LOW ZML crash · Testing · risk 3 (impact 1 × exposure 3)

API zml.testing.expectClose
Location zml/testing.zig:85-91 (left dtype list accepts the four float types), :95-104 (right dtype list, lacks them), :113 (unreachable), :116-118 (bool routed to integer comparison), :295 (cmpIntSlices panic)
Platform CPU

What happens. Panics on bool slices. Identical f4e2m1, f8e3m4, f8e4m3 and f8e8m0 slices hit `unreachable` at zml/testing.zig:113 instead of comparing equal.

Root cause. Bool is routed to the integer comparison, which only accepts int or vector types (zml/testing.zig:116-118, :295). The right-hand dtype switch lacks the four f8/f4 types that the left-hand switch accepts (:85-91 vs :95-104), so comparing two tensors of one of those dtypes reaches `else => unreachable`.

> **Risk assessment.** Test helper only; it makes these dtypes hard to test.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_close_bool --test_output=errors`, then `bazel test //tools/zmlsmith/regressions:found_close_f8 --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `close_bool`: *fails* (matches `cmpIntSlices expect int or vector of int`)
   `1/1 close_bool.test.zml.testing.expectClose on bool slices...thread 3150860 panic: cmpIntSlices expect int or vector of int`
   `close_f8`: *fails* (matches `zml/testing\.zig:113:`)
   `/home/pollivie/.cache/bazel/_bazel_pollivie/db33e5240b55bbdfdc37d8cd1827c2ab/sandbox/processwrapper-sandbox/5300/execroot/_main/zml/testing.zig:113:25: 0x58563af2abd0 in expectClose__anon_49865 (tools_Szmlsmith_Sregressi`

Verification: confirmed, report corrected. **Expected behavior, from:** A tensor compared with itself must be close (identical bytes). For bool, the comparison should be exact equality. The float dtypes are listed on the left, so the right-hand list is incomplete.

- zml/testing.zig:85: .f8e4m3, (left list; also :83, :84, :86-91 for the other types)
- zml/testing.zig:95-103: inline .bf16, .f16, .f32, .f64, .f8e4m3b11fnuz, .f8e4m3fn, .f8e4m3fnuz, .f8e5m2, .f8e5m2fnuz
- zml/testing.zig:113: else => unreachable,
- zml/testing.zig:116-118: inline .bool, .u2, ...i64 => |t| { const T = t.toZigType(); const report = try .compareSlices(false, allocator, T, T, left, right, opts);
- zml/testing.zig:295: else => @panic("cmpIntSlices expect int or vector of int"),
- zml/testing.zig:156: return !report.nan_or_inf and report.close_fraction >= opts.minimum_close_fraction;
- zml/testing.zig:266-268: if (!std.math.isFinite(l_f32) or !std.math.isFinite(r_f32)) { self.nan_or_inf = true; continue; }
- zml/testing.zig:303-310: if (self.processed_count == 0) { return .{ ... .close_fraction = std.math.nan(f32), ...}

Suggested fix. Handle each case.

tools/zmlsmith/regressions/found/close_bool.zig (10 lines)

```
const std = @import("std");
const zml = @import("zml");

test "zml.testing.expectClose on bool slices" {
    const sh: zml.Shape = .init(.{2}, .bool);
    var data = [_]bool{ true, false };
    const s: zml.Slice = .init(sh, std.mem.sliceAsBytes(&data));
    try zml.testing.expectClose(std.testing.io, s, s, .{});
}
```

tools/zmlsmith/regressions/found/close_f8.zig (14 lines)

```
const std = @import("std");
const zml = @import("zml");

// zml.testing.expectClose on identical f8e4m3 slices hits `unreachable` (testing.zig:113).
// The left dtype switch accepts .f8e4m3 (testing.zig:85) but the right dtype switch
// (testing.zig:95-104) has no .f8e4m3 arm, so a tensor compared with itself panics instead
// of passing. Basis: a slice compared with itself is close under any tolerance.
test "close_f8" {
    const sh: zml.Shape = .init(.{2}, .f8e4m3);
    var data = [_]u8{ 0x38, 0x40 };
    const s: zml.Slice = .init(sh, std.mem.sliceAsBytes(&data));
```

```
    try zml.testing.expectClose(std.testing.io, s, s, .{});
}
```

L06 DataType.resolvePeerType is not commutative and can lose values

LOW ZML wrong values · Host · risk 2 (impact 2 × exposure 1)

API DataType.resolvePeerType
Location `zml/dtype.zig:179-190`
Platform CPU

What happens. DataType.resolvePeerType returns the first argument when both dtypes have the same size, so it is not commutative: f16 with bf16 gives f16 and bf16 with f16 gives bf16, and likewise for the f8 formats; u8 with i8 resolves to u8 and u32 with i16 to u32. No ZML code calls resolvePeerType outside its own test (zml/dtype.zig:192-203); binaryOp asserts equal dtypes (zml/tensor.zig:4557).

Root cause. Size-only comparison.

> **Risk assessment.** A public helper with a non-symmetric and lossy answer; no ZML arithmetic path reaches it, so the risk is to downstream callers of the helper only.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_peer_type --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `peer_type`: *fails* (matches `expected \.f16, found \.bf16`)
   `1/1 peer_type.test.peer_type...expected .f16, found .bf16`

Verification: confirmed, report corrected. **Expected behavior, from:** A peer-type resolver is expected to be symmetric (the reproducer compares both orders); this follows the function's own doc comment ('Try to find a type compatible with both dtype') and the test cases already in dtype.zig.

- zml/dtype.zig:189: 'return if (a.sizeOf() >= b.sizeOf()) a else b;'
- zml/tensor.zig:4557: 'stdx.debug.assert(self.dtype() == other.dtype(), "{s} expects tensor to be of same type, ...")'
- grep for resolvePeerType across zml/ finds only zml/dtype.zig:179 and its test

Suggested fix. Follow numpy-like promotion, or reject ambiguous pairs.

`tools/zmlsmith/regressions/found/peer_type.zig` (15 lines)

```
const std = @import("std");
const zml = @import("zml");

// DataType.resolvePeerType returns the first argument when both have the same size: it
// isn't commutative (f16 with bf16 gives f16, bf16 with f16 gives bf16, likewise for
// the f8 formats) and it resolves u8 with i8 to u8, or u32 with i16 to u32, types that
// can't hold the other's values (zml/dtype.zig:179-190 at 74c590b0). Found by reading the
// code while adding dtype coverage to zmlsmith.
test "peer_type" {
    const D = zml.DataType;
    try std.testing.expectEqual(D.f16.resolvePeerType(.bf16), D.bf16.resolvePeerType(.f16));
    try std.testing.expect(D.u8.resolvePeerType(.i8) != D.u8);
    try std.testing.expect(D.u32.resolvePeerType(.i16) != D.u32);
}
```

L07 customCall array outputs don't compile

LOW ZML compile · Runtime · risk 2 (impact 2 × exposure 1)

API zml.ops.customCall
Location `zml/ops.zig:2505-2509 (customCall result mapping for an array output spec; the bug is `array_info.fields.len` at zml/ops.zig:2507). Output type spec: zml/ops.zig:2279-2283.`
Platform CPU

What happens. customCall with a fixed-size array of output shapes ([2]zml.Shape) does not compile (Zig error at zml/ops.zig:2507: no field named 'fields'). Tuple and single-shape outputs are unaffected.

Root cause. In the result mapping of customCall, the `.array` prong uses `array_info.fields.len` (zml/ops.zig:2507). std.builtin.Type.Array has `len` and `child`, not `fields`, so any [N]Shape output spec with N != 1 is a Zig compile error. The

comptime type (CustomCallResultTypeFromOutputSpec, zml/ops.zig:2279-2283) is already correct ([N]Tensor), so only the value mapping is wrong. Tuples and slices take other prongs and work.

> **Risk assessment.** Only for custom kernels.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_custom_call_arr --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `custom_call_arr`: *fails to build* (matches `ops\.zig:2507:\d+: error`)
   `zml/ops.zig:2507:39: error: no field named 'fields' in struct 'builtin.Type.Array'`

Verification: confirmed as reported. **Expected behavior, from:** Zig std Type.Array has fields {len, child, sentinel_ptr}; the intended result is [array_info.len]Tensor, as in the output-spec function at zml/ops.zig:2283.

- zml/ops.zig:2505-2509: `.array => |array_info| b: { if (array_info.len == 1) break :b outputs_flat[0]; var outputs_: [array_info.fields.len]Tensor = undefined; @memcpy(&outputs_, outputs_flat); break :b outputs_; },`
- zml/ops.zig:2283: `[array_info.len]Tensor;`

Suggested fix. Support arrays like tuples.

`tools/zmlsmith/regressions/found/custom_call_arr.zig` (23 lines)

```zig
const std = @import("std");
const zml = @import("zml");

// ops.customCall with a fixed-size array of output shapes ([2]zml.Shape) does not compile.
// The result mapping in zml/ops.zig:2507 reads `array_info.fields.len`, but
// std.builtin.Type.Array has `len` and no `fields`, so the array branch is a Zig compile error.
test "custom_call_arr" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        const out_shapes = [2]zml.Shape{ zml.Shape.init(.{4}, .f32), zml.Shape.init(.{4}, .f32) };
        fn forward(x: zml.Tensor) [2]zml.Tensor {
            return zml.ops.customCall("zml$custom_call_arr_probe", .{x}, out_shapes, .{}, .{ .has_side_effect = false });
        }
    };
    const x: zml.Tensor = .fromShape(zml.Shape.init(.{4}, .f32));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{x}, platform, .{});
    defer exe.deinit();
    try std.testing.expectEqual(@as(usize, 2), exe.output_shapes.len);
}
```

8 Design notes

These behaviors are surprising but may be intended. Each needs a decision rather than a fix, and they are not counted as bugs.

D01 .coord is a reserved tag in gather

DESIGN ZML design · Design · risk 1 (impact 1 × exposure 1)

Location zml/ops.zig:2122 (indices stacked on an internal .coord axis), zml/ops.zig:2135 (index_vector_dim = indices.axis(.coord)), zml/ops.zig:2060-2063 (a self axis with the same tag as an index axis becomes batching)
Platform CPU
KnownBug gather_coord_tag

What happens. When the operand also has a .coord axis, the indices' .coord becomes a batching dim and MLIR rejects the gather.

Root cause. gather stacks its index tensors on an internal `.coord` axis (zml/ops.zig:2122). A user-supplied indices tensor with its own `.coord` axis collides with it, and the collision turns the user's coord into a batching dim (zml/ops.zig:2060-2063) and makes index_vector_dim ambiguous, so MLIR rejects the op.

> **Risk assessment.** Needs a decision rather than a fix.

Reproduction.

1. Get ZML at `74c590b0` and add `tools/zmlsmith/` to the tree (it is not committed).
2. Run `bazel test //tools/zmlsmith/regressions:found_gather_coord --test_output=errors`.
3. The test fails while the bug is present (it is written to pass once the bug is fixed). In the run at `74c590b0` the log matched the pattern given for each reproducer:
   `gather_coord`: *fails* (matches `Failed to create MLIR operation`)
   `thread 3151242 panic: Failed to create MLIR operation`

Verification: confirmed, report corrected. **Expected behavior, from:** gatherSlices documents `.coord` as the coordinate axis of its indices (zml/tensor.zig:2905-2906), so the tag has a public meaning there. gather's doc (zml/tensor.zig:2815-2826) does not mention .coord, so the contract for gather is undocumented.

- zml/ops.zig:2122: const indices = Tensor.stack(indices_per_axis.constSlice(), .last, .coord);
- zml/ops.zig:2060-2063: if (indices_shape.hasTag(t)) |id_ax| { self_kind.buffer[self_ax] = .batching; ...
- zml/tensor.zig:2906: /// The coordinates are read from the `.coord` axis, or last axis if `.coord` is not found.

Suggested fix. Decide and document.

tools/zmlsmith/regressions/found/gather_coord.zig (46 lines)

```
const std = @import("std");
const zml = @import("zml");

// Generated by zmlsmith from seed 501638.
test "gather_coord_tag" {
    const platform = zml.testing.env();
    const allocator = std.testing.allocator;
    const io = std.testing.io;

    const Local = struct {
        fn forward(x0: zml.Tensor, x1: zml.Tensor) [1]zml.Tensor {
            const x2 = zml.ops.gather(x0, &.{1}, &.{x1}, .{});
            _ = &x2;
            return .{x2};
        }
    };

    const x0: zml.Tensor = .fromShape(zml.Shape.init(.{ .d = 1, .f = 1, .coord = 1 }, .u32));
    const x1: zml.Tensor = .fromShape(zml.Shape.init(.{ 1, 1, 4 }, .i8).withTags(.{ .coord, .p, ._ }));
    var exe = try zml.module.compile(allocator, io, Local.forward, .{ x0, x1 }, platform, .{});
    defer exe.deinit();

    try zml.testing.expectEqualShapes(zml.Shape.init(.{ 1, 1, 4, 1 }, .u32).withTags(.{ .d, .p, ._, .coord }), exe.output_shapes[0]);

    var b0: zml.Buffer = try .fromBytes(io, platform, x0.shape(), .replicated, &[_]u8{
        0, 0, 0, 0,
    });
    defer b0.deinit();
    var b1: zml.Buffer = try .fromBytes(io, platform, x1.shape(), .replicated, &[_]u8{
        0, 0, 0, 0,
    });
    defer b1.deinit();
    var res = try zml.testing.autoCall(allocator, io, &exe, Local.forward, .{ b0, b1 });
    defer for (&res) |*r| r.deinit();

    {
        const expected_data align(16) = [_]u8{
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        };
        const expected: zml.Slice = .init(zml.Shape.init(.{ 1, 1, 4, 1 }, .u32).withTags(.{ .d, .p, ._, .coord }), @as([]const u8, &expected_data));
```

```
        const got = try res[0].toSliceAlloc(allocator, io);
        defer got.free(allocator);
        try std.testing.expectEqualSlices(u8, expected.constData(), got.constData());
    }
}
```

D02 Quantized scales are tagged differently

DESIGN ZML design · Design · risk 1 (impact 1 × exposure 1)

Location zml/quantization.zig:171 (quantizeBlockFp8 scales keep the quantized axis tag via setDim); zml/quantization.zig:189 (quantizeNvfp4 splitAxis names the outer axis .sc); zml/quantization.zig:208 (scales.squeeze(.blk) keeps .sc); zml/nn.zig:79-88 (unpackFp4)

Platform CPU

What happens. quantizeBlockFp8 keeps the quantized axis' tag on its scales (quantization.zig:171), quantizeNvfp4 renames it .sc (:208). nn.unpackFp4 only works when the packed axis is last, which its doc doesn't say.

> **Risk assessment.** Needs a decision rather than a fix.

Reproduction. `no reproducer (API behavior)`

Verification: confirmed as reported. **Expected behavior, from:** Design observation: tag consistency across quantization schemes.

- zml/quantization.zig:171: `.scales = scales.reshape(x.shape().setDim(axis, @divExact(x.dim(axis), block_size)).withDtype(.f32)).convert(scale_dtype),`
- zml/quantization.zig:189: `const grouped = scaled.splitAxis(axis, .{ .sc = -1, .blk = nvfp4_block_size });`
- zml/nn.zig:85-87: `return w.bitCast(.f4e2m1) ... .merge(.{ .kb = .{ source_tag, .bitcast } }) .renameTag(.kb, result_tag);`

Suggested fix. Decide and document.

D03 maxPool2d indices hold only the column of the max

DESIGN ZML design · Design · risk 1 (impact 1 × exposure 1)

Location zml/tensor.zig:3722 (a = self.axis(-1)), :3735 (iota(self._shape, a)); TODO at :3717 and :3720 documents the last-axis limitation

Platform CPU

What happens. It reduces an iota along the last axis (tensor.zig:3745), so the row is lost and ties across rows keep the smallest column. PyTorch returns the flat index in the plane.

> **Risk assessment.** Needs a decision rather than a fix.

Reproduction. `no reproducer (API behavior)`

Verification: confirmed as reported. **Expected behavior, from:** PyTorch max_pool2d(return_indices=True) returns the flat index h * W + w inside each (H, W) plane (PyTorch docs for torch.nn.functional.max_pool2d). ZML's indices are the column only.

- zml/tensor.zig:3720-3722: `// TODO: support maxPool on non last axis` / `const a = self.axis(-1);`
- zml/tensor.zig:3735: `.{ self, iota(self._shape, a) },`

Suggested fix. Decide and document.

D04 Buffer.toSlice fails on buffers sharded on any axis but the first

DESIGN ZML design · Design · risk 1 (impact 1 × exposure 1)

Location zml/buffer.zig:283-290 (toSlice; error at :290) and zml/buffer.zig:305-328 (toSliceAlloc, which copies the same shards with Slice.copy at :324); TODOs at :288 and :322

Platform CPU

What happens. error.NonContiguousShardRead (buffer.zig:290), while toSliceAlloc copies the same shards; the source has a TODO.

> **Risk assessment.** Needs a decision rather than a fix.

Reproduction. `no reproducer (API behavior)`

Verification: confirmed as reported. **Expected behavior, from:** Source TODOs at zml/buffer.zig:288 and :322 acknowledge the inconsistency; the two methods are documented to read the same buffer.

- zml/buffer.zig:290: `if (!sub_slice.isContiguous()) return error.NonContiguousShardRead;`
- zml/buffer.zig:322: `// TODO: why is this using Slice.copy while toSlice errors out ?`

Suggested fix. Decide and document.

9 Appendix A: robustness audit

`zmlsmith robust` runs each invalid use of the API in its own process and records how it fails. A clean rejection is a ZML error or panic with a message that names the problem. 38 of 83 cases are not cleanly rejected.

Outcome	Cases
accepted silently	`with_tags_duplicate`, `convert_complex`, `dynamic_slice_float_offset`, `triangular_same_axis`, `argmax_empty_axis`, `mean_empty_axis`, `sum_empty_tensor`, `pow_int`, `arange_empty`, `linspace_one_step`, `slice_squeeze_non_unit`
not validated: invalid MLIR	`axis_equal_rank`, `concat_mismatched_other_dim`, `concat_mismatched_dtype`, `slice_start_below_minus_dim`, `reshape_wrong_count`, `transpose_repeated_axis`, `gather_float_indices`, `pad_negative_beyond_dim`, `dynamic_slice_too_long`, `exp_int`, `shift_float`, `not_float`, `if_branch_shapes_differ`, `composite_unnamespaced_name`, `conv2d_repeated_spatial_dim`
not validated: safety panic or crash	`insert_axes_full_rank`, `chunk_exact_not_divisible`, `chunk_trailing_more_than_dim`, `unflatten_last_negative`, `subbyte_odd_last_dim`, `call_too_many_args`, `call_too_few_args`, `fill_too_many_results`, `bake_too_many`, `fixed_pool_not_divisible`, `fixed_pool_zero_blocks`, `fixed_pool_empty_buffer`
rejected with a message	`axis_below_minus_rank`, `unknown_tag`, `add_mismatched_dims`, `add_mismatched_dtype`, `dot_mismatched_contracting`, `dot_missing_tag`, `slice_start_after_end`, `slice_end_past_dim`, `slice_zero_step`, `slice_negative_step`, `reshape_two_auto`, `transpose_wrong_len`, `squeeze_non_one`, `with_tags_wrong_count`, `broad_incompatible`, `broadcast_axes_out_of_range`, `split_wrong_sum`, `topk_k_zero`, `topk_k_too_large`, `repeat_wrong_len`, `stutter_zero`, `select_non_bool`, `cmp_mismatched_dims`, `bitcast_incompatible_last_dim`, `unflatten_not_divisible`, `dynamic_update_larger_update`, `conv_mismatched_features`, `cholesky_rank3`, `softmax_int`, `iota_tag_missing`, `scalar_nan`, `scalar_out_of_range`, `subbyte_scalar`, `reuse_buffer_twice`, `while_cond_not_bool`, `while_body_changes_shape`, `if_pred_not_scalar`, `unpack_fp4_packed_not_last`, `uniform_f8e4m3`, `quantize_block_fp8_e5m2`, `quantize_nvfp4_block_remainder`, `scaled_dot_narrow_output`, `call_wrong_arg_shape`, `call_wrong_arg_dtype`, `read_donated_argument`

Some of the silently accepted cases are fine: empty reductions and integer `pow` are well defined. They are listed so that each one is a deliberate choice.

10 Appendix B: withdrawn after verification

These entries were in the unverified report and were removed because the review showed they are not bugs, or not established. They are listed so the decision can be checked.

Was	Title	Reason
M20	FnExe with a single Tensor result doesn't compile	Not a bug: FnExe requires a struct with named fields as output (zml/exe.zig:518-532), and every forward in the repository returns one. At most the compile error could be clearer.
L04	x / broadcast(y) is computed with a reciprocal; f32 to e8m0 rounds ties up	Not a bug on the default path: the reciprocal rewrite needs xla_cpu_enable_fast_math (default false, xla/debug_options_flags.cc:238), and f32 to e8m0 rounding is TBD in the spec (docs/spec.md:2191-2195).
L07	FixedBufferPool.init panics on sizes it should reject	Caller precondition violations with no documented error contract (FixedBufferPool.init sizes); no reproducer.
D02	Returning one reuseBuffer result twice panics	Not a bug: the panic at zml/Compiler.zig:662 states the contract (`reuseBuffer` at most once per input).
D03	Duplicate tags are accepted by withTags	Not established: withTags accepting duplicate tags is an inconsistency with Shape.rename (zml/shape.zig:1146), but no wrong result was shown.
D04	Float offsets in dynamicSlice and triangular on the same axis twice are accepted	Not a bug: float offsets are converted like every other convert (zml/tensor.zig:2063), and nothing specifies triangular applied twice to one axis.
D07	reducePrecision flushes the reduced format's subnormals on CPU	Not a bug: the StableHLO reduce_precision spec (docs/spec.md:5037-5054) does not say subnormals of the reduced format are kept.
D09	softmax computes in f32 for every dtype, f64 included	Intended: softmax converts to f32 explicitly (zml/tensor.zig:1735). Only a documentation note.

11 Appendix C: coverage and limits

- **Graph API.** The op families call 160 of the 186 `pub fn` declared in `zml/tensor.zig` (the others are internal helpers such as `_result` or `binaryOpHelper`), 27 of the 40 in `zml/nn.zig` (not `forwardQuantized`, the `jsonParse` hooks or the buffer helpers), and 16 of the 40 in `zml/ops.zig` (not the custom call and kernel entry points, nor the generic `ops.scatter`). Counted by name at `74c590b0`. Quantization helpers are emulated bit for bit, and every f8/f4 dtype is generated. Ranks go up to 8, with size-0 and size-1 axes, dims around tile widths, programs of 30 to 80 nodes, and swarm testing.
- **Sharding.** A 4-device mesh, and a 2-level mesh with `data` and `model` on distinct axes. Both Shardy and GSPMD.
- **Runtime.** Donation, repeated and unawaited calls, 6 threads calling one executable, 4 threads compiling, FnExe with a baked prefix, and every Rng algorithm.
- **IO.** safetensors, `tokenizer.json` (IREE and homemade), sentencepiece models, rope configs, and the HTTP, S3 and GCS backends against a local server; the HF backend against a local https stand-in for huggingface.co (random repository layouts, listings, stats and ranged reads); the Loader round trip.
- **Known bugs.** The generator avoids each known bug by default, and 15% of seeds lift one at random so its input space is still explored.
- **Not covered.**
  - GPU and TPU platforms, custom kernels (Triton, CUDA, NKI) and MoE.